LOONGARCH ISA REFERENCE

龙芯架构
指令速查表

LoongArch Instruction Quick Reference

卷册　卷一 · 基础架构

范围　A – Z 字母

条目　357 条

排序　助记符字典序

A4 · 按指令连续排列 · 含编码方框位图

目录

A 6

add.d 6
add.w 6
addi.d 7
addi.w 7
addu16i.d 7
alsl.d 8
alsl.w 8
alsl.wu 9
amadd.b 9
amadd.d 10
amadd.h 10
amadd.w 11
amadd_db.b 11
amadd_db.d 12
amadd_db.h 12
amadd_db.w 13
amand.d 13
amand.w 14
amand_db.d 14
amand_db.w 15
amcas.b 15
amcas.d 16
amcas.h 16
amcas.w 17
amcas_db.b 17
amcas_db.d 18
amcas_db.h 18
amcas_db.w 19
ammax.d 19
ammax.du 20
ammax.w 20
ammax.wu 21
ammax_db.d 21
ammax_db.du 22
ammax_db.w 22
ammax_db.wu 23
ammin.d 23
ammin.du 24
ammin.w 24
ammin.wu 25
ammin_db.d 25
ammin_db.du 26
ammin_db.w 26
ammin_db.wu 27
amor.d 27
amor.w 28
amor_db.d 28
amor_db.w 29
amswap.b 29
amswap.d 30
amswap.h 30
amswap.w 31
amswap_db.b 31
amswap_db.d 32
amswap_db.h 32
amswap_db.w 33
amxor.d 33
amxor.w 34
amxor_db.d 34
amxor_db.w 35
and 35
andi 35
andn 36
asrtgt.d 36
asrtle.d 36

B 36

b 37
bceqz 37
bcnez 37
beq 38
beqz 38
bge 38
bgeu 39
bitrev.4b 39
bitrev.8b 40
bitrev.d 40
bitrev.w 40
bl 41
blt 41
bltu 41
bne 42
bnez 42
break 42
bstrins.d 43
bstrins.w 43
bstrpick.d 43
bstrpick.w 44
bytepick.d 44
bytepick.w 45

C 45

cacop 45
clo.d 45
clo.w 46
clz.d 46
clz.w 46
cpucfg 47
crc.w.b.w 47
crc.w.d.w 47
crc.w.h.w 48

crc.w.w.w 48
crcc.w.b.w 49
crcc.w.d.w 49
crcc.w.h.w 50
crcc.w.w.w 50
csrrd 50
csrwr 51
csrxchg 51
cto.d 51
cto.w 52
ctz.d 52
ctz.w 52

D 53

dbar 53
dbcl 53
div.d 53
div.du 54
div.w 54
div.wu 54

E 55

ertn 55
ext.w.b 55
ext.w.h 55

F 56

fabs.d 56
fabs.s 56
fadd.d 56
fadd.s 57
fclass.d 57
fclass.s 57
fcmp.cond.d 58
fcmp.cond.s 58
fcopysign.d 59
fcopysign.s 59
fcvt.d.s 59
fcvt.s.d 60
fdiv.d 60
fdiv.s 60
ffint.d.l 61
ffint.d.w 61
ffint.s.l 61
ffint.s.w 62
fld.d 62
fld.s 62
fldgt.d 63
fldgt.s 63
fldle.d 63
fldle.s 64
fldx.d 64
fldx.s 64
flogb.d 65
flogb.s 65
fmadd.d 65
fmadd.s 66
fmax.d 66
fmax.s 66
fmaxa.d 67
fmaxa.s 67
fmin.d 67
fmin.s 68
fmina.d 68
fmina.s 68
fmov.d 69
fmov.s 69
fmsub.d 69
fmsub.s 70
fmul.d 70
fmul.s 70
fneg.d 71
fneg.s 71
fnmadd.d 71
fnmadd.s 72
fnmsub.d 72
fnmsub.s 72
frecip.d 73
frecip.s 73
frecipe.d 73
frecipe.s 74
frint.d 74
frint.s 74
frsqrt.d 75
frsqrt.s 75
frsqrte.d 75
frsqrte.s 76
fscaleb.d 76
fscaleb.s 76
fsel 77
fsqrt.d 77
fsqrt.s 77
fst.d 78
fst.s 78
fstgt.d 78
fstgt.s 79
fstle.d 79
fstle.s 79
fstx.d 80
fstx.s 80
fsub.d 80
fsub.s 81
ftint.l.d 81
ftint.l.s 81
ftint.w.d 82
ftint.w.s 82
ftintrm.l.d 82
ftintrm.l.s 83
ftintrm.w.d 83
ftintrm.w.s 83
ftintrne.l.d 84

ftintrne.l.s 84
ftintrne.w.d 84
ftintrne.w.s 85
ftintrp.l.d 85
ftintrp.l.s 85
ftintrp.w.d 86
ftintrp.w.s 86
ftintrz.l.d 86
ftintrz.l.s 87
ftintrz.w.d 87
ftintrz.w.s 87

I 88

ibar 88
idle 88
invtlb 88
iocsrrd.b 89
iocsrrd.d 89
iocsrrd.h 89
iocsrrd.w 90
iocsrwr.b 90
iocsrwr.d 90
iocsrwr.h 91
iocsrwr.w 91

J 91

jirl 91

L 91

ld.b 92
ld.bu 92
ld.d 92
ld.h 93
ld.hu 93
ld.w 93
ld.wu 94
lddir 94
ldgt.b 94
ldgt.d 95
ldgt.h 95
ldgt.w 95
ldle.b 96
ldle.d 96
ldle.h 97
ldle.w 97
ldpte 97
ldptr.d 98
ldptr.w 98
ldx.b 98
ldx.bu 99
ldx.d 99
ldx.h 99
ldx.hu 100
ldx.w 100
ldx.wu 100
ll.d 101
ll.w 101
llacq.d 101
llacq.w 102
lu12i.w 102
lu32i.d 102
lu52i.d 103

M 103

maskeqz 103
masknez 103
mod.d 104
mod.du 104
mod.w 104
mod.wu 105
movcf2fr 105
movcf2gr 105
movfcsr2gr 106
movfr2cf 106
movfr2gr.d 106
movfr2gr.s 107
movfrh2gr.s 107
movgr2cf 107
movgr2fcsr 108
movgr2fr.d 108
movgr2fr.w 108
movgr2frh.w 109
mul.d 109
mul.w 109
mulh.d 110
mulh.du 110
mulh.w 110
mulh.wu 111
mulw.d.w 111
mulw.d.wu 111

N 112

nor 112

O 112

or 112
ori 112
orn 113

P 113

pcaddi 113
pcaddu12i 113
pcaddu18i 114
pcalau12i 114
preld 114
preldx 115

R 115

rdtime.d 115

rdtimeh.w 115
rdtimel.w 116
revb.2h 116
revb.2w 116
revb.4h 117
revb.d 117
revh.2w 117
revh.d 118
rotr.d 118
rotr.w 118
rotri.d 119
rotri.w 119

S 119

sc.d 120
sc.q 120
sc.w 121
screl.d 121
screl.w 122
sll.d 122
sll.w 122
slli.d 123
slli.w 123
slt 123
slti 124
sltu 124
sltui 124
sra.d 125
sra.w 125
srai.d 125
srai.w 126
srl.d 126
srl.w 126
srli.d 127
srli.w 127
st.b 127
st.d 128
st.h 128
st.w 128
stgt.b 129
stgt.d 129
stgt.h 129
stgt.w 130
stle.b 130
stle.d 130
stle.h 131
stle.w 131
stptr.d 131
stptr.w 132
stx.b 132
stx.d 132
stx.h 133
stx.w 133
sub.d 133
sub.w 134
syscall 134

T 134

tlbclr 134
tlbfill 135
tlbflush 135
tlbrd 135
tlbsrch 136
tlbwr 136

X 136

xor 136
xori 137

A · 算术、逻辑、原子与边界检查指令

本章收录全部以 A 开头的指令。约定：`GR[x]` 表示第 x 号通用寄存器；`GRLEN` 在 LA32 中为 32，在 LA64 中为 64；位域编号从 31 到 0。位图中固定编码字段以浅灰底标示，寄存器号与立即数字段为白底。

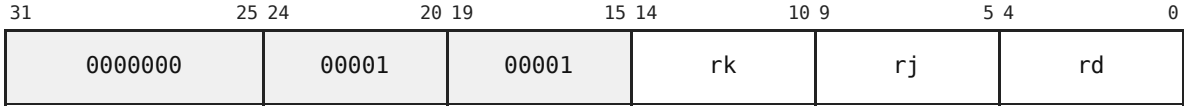
add.d `rd, rj, rk`

`GR[rd] = GR[rj][63:0] + GR[rk][63:0]`

双字整数加法（Add Doubleword） · 3R 型 · LA64

将通用寄存器 `rj`、`rk` 中的 64 位数据相加，结果写入 `rd`。不进行溢出检测。

编码



编码值

```
0x00108000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
add.d $a0, $a1, $a2      # $a0 = $a1 + $a2
```

语义伪代码

```
tmp = GR[rj][63:0] + GR[rk][63:0]
GR[rd] = tmp[63:0]
```

---

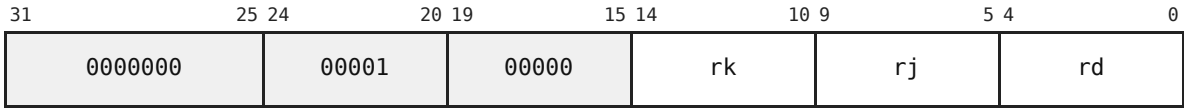
add.w `rd, rj, rk`

`GR[rd] = SignExtend(GR[rj][31:0] + GR[rk][31:0], GRLEN)`

字整数加法（Add Word） · 3R 型 · LA32 / LA64

将通用寄存器 `rj`、`rk` 中的低 32 位数据相加，结果符号扩展至 GRLEN 位后写入 `rd`。不进行溢出检测。

编码



编码值

```
0x00100000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
add.w $a0, $a1, $a2      # $a0 = $a1 + $a2
```

语义伪代码

```
tmp = GR[rj][31:0] + GR[rk][31:0]
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

---

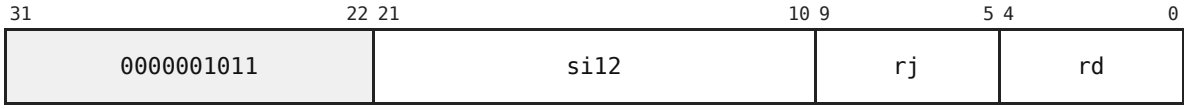
addi.d rd, rj, si12

GR[rd] = GR[rj][63:0] + SignExtend(si12, 64)

双字立即数加法（Add Immediate Doubleword） · 2RI12 型 · LA64

将 `rj` 的 64 位数据与符号扩展后的 12 位立即数 `si12` 相加，结果写入 `rd`。`si12` 的取值范围为 −2048～2047。不进行溢出检测。

编码



编码值

```
0x02c00000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
addi.d $a0, $a1, -16      # $a0 = $a1 - 16
```

语义伪代码

```
tmp = GR[rj][63:0] + SignExtend(si12, 64)
GR[rd] = tmp[63:0]
```

---

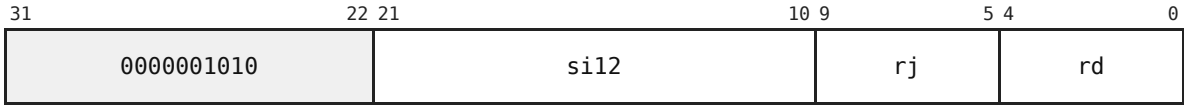
addi.w rd, rj, si12

GR[rd] = SignExtend(GR[rj][31:0] + SignExtend(si12, 32), GRLEN)

字立即数加法（Add Immediate Word） · 2RI12 型 · LA32 / LA64

将 `rj` 的低 32 位与符号扩展后的 12 位立即数 `si12` 相加，结果符号扩展至 GRLEN 位后写入 `rd`。`si12` 的取值范围为 −2048～2047。不进行溢出检测。

编码



编码值

```
0x02800000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
addi.w $a0, $a1, -16      # $a0 = $a1 - 16
```

语义伪代码

```
tmp = GR[rj][31:0] + SignExtend(si12, 32)
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

---

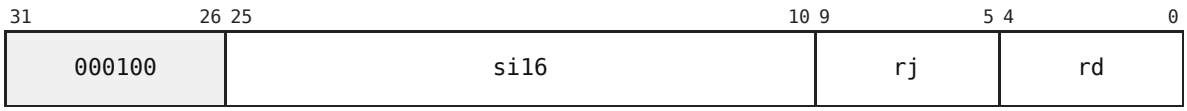
addu16i.d rd, rj, si16

GR[rd] = GR[rj] + SignExtend({si16, 16'b0}, 64)

双字 16 位立即数加法（Add 16-bit Immediate Doubleword） · 2RI16 型 · LA64

将 16 位立即数 `si16` 逻辑左移 16 位后符号扩展至 64 位，再与 `rj` 相加，结果写入 `rd`。`si16` 的取值范围为 −32768～32767。该指令常与 `ldptr.w/d`、`stptr.w/d` 配合，用于位置无关代码的 GOT 表访问。不进行溢出检测。

编码



编码值

```
0x10000000 | ((si16 & 0xffff) << 10) | (rj << 5) | rd
```

汇编示例

```
addu16i.d $a0, $a1, 1      # $a0 = $a1 + 0x10000
```

语义伪代码

```
tmp = GR[rj][63:0] + SignExtend({si16, 16'b0}, 64)
GR[rd] = tmp[63:0]
```

---

alsl.d `rd, rj, rk, sa`

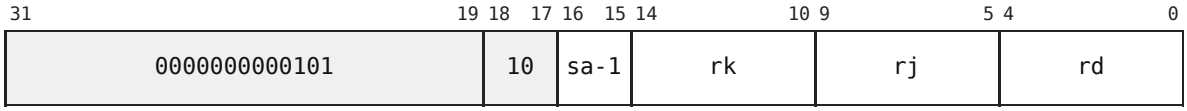
`GR[rd] = (GR[rj][63:0] << sa) + GR[rk][63:0]`

双字左移加法（Add Shifted Left Doubleword） · 3R 型 · LA64

将 `rj` 的 64 位数据左移 `sa` 位，加上 `rk` 的 64 位数据，结果写入 `rd`。`sa` 的取值范围为 1～4。不进行溢出检测。

> 编码字段为 `sa-1`（2 位）。官方手册把该 2 位编码字段记为 `sa2`，并规定实际移位量为 `sa2+1`；汇编时直接填写实际移位量（1～4）。本手册语法用 `sa` 表示该实际移位量。

编码



编码值

```
0x002c0000 | (((sa - 1) & 0x3) << 15) | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
alsl.d $a0, $a1, $a2, 2      # $a0 = ($a1 << 2) + $a2
```

语义伪代码

```
tmp = (GR[rj][63:0] << sa) + GR[rk][63:0]
GR[rd] = tmp[63:0]
```

---

alsl.w `rd, rj, rk, sa`

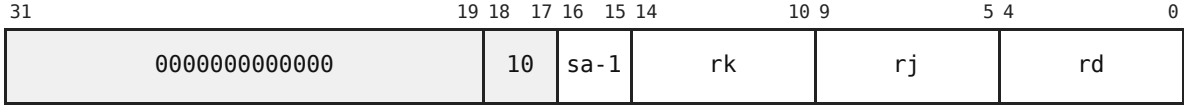
`GR[rd] = SignExtend((GR[rj][31:0] << sa) + GR[rk][31:0], GRLEN)`

字左移加法（Add Shifted Left Word） · 3R 型 · LA32 / LA64

将 `rj` 的低 32 位左移 `sa` 位，加上 `rk` 的低 32 位，结果符号扩展至 GRLEN 位后写入 `rd`。`sa` 的取值范围为 1～4。不进行溢出检测。

> 编码字段为 `sa-1`（2 位）。官方手册把该 2 位编码字段记为 `sa2`，并规定实际移位量为 `sa2+1`；汇编时直接填写实际移位量（1～4）。本手册语法用 `sa` 表示该实际移位量。

编码



编码值

```
0x00040000 | (((sa - 1) & 0x3) << 15) | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
alsl.w $a0, $a1, $a2, 2      # $a0 = ($a1 << 2) + $a2
```

语义伪代码

```
tmp = (GR[rj][31:0] << sa) + GR[rk][31:0]
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

---

alsl.wu `rd, rj, rk, sa`

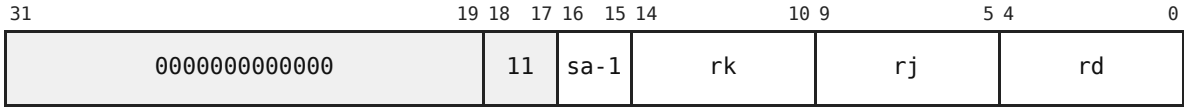
`GR[rd] = ZeroExtend((GR[rj][31:0] << sa) + GR[rk][31:0], GRLEN)`

字左移加法（零扩展）（Add Shifted Left Word Unsigned） · 3R 型 · LA64

将 `rj` 的低 32 位左移 `sa` 位，加上 `rk` 的低 32 位，结果零扩展至 GRLEN 位后写入 `rd`。`sa` 的取值范围为 1～4。不进行溢出检测。

> 编码字段为 `sa-1`（2 位）。官方手册把该 2 位编码字段记为 `sa2`，并规定实际移位量为 `sa2+1`；汇编时直接填写实际移位量（1～4）。本手册语法用 `sa` 表示该实际移位量。

编码



编码值

```
0x00060000 | (((sa - 1) & 0x3) << 15) | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
alsl.wu $a0, $a1, $a2, 2      # $a0 = ($a1 << 2) + $a2
```

语义伪代码

```
tmp = (GR[rj][31:0] << sa) + GR[rk][31:0]
GR[rd] = ZeroExtend(tmp[31:0], GRLEN)
```

> **原子指令通用规则**
>
> AM* 指令对 `GR[rj]` 指向的内存单元执行原子的“读—改—写”：旧值符号扩展至 GRLEN 位后写入 `GR[rd]`，新值写回该内存单元。下文中 `[rj]` 表示 `GR[rj]` 指向的内存单元。
>
> - 地址必须自然对齐，否则触发非对齐异常；字节访问（`.b`）无此限制。
> - `_db` 变体在操作前后附加数据屏障（DB）。
> - 官方规定：`amswap`/`amadd`/`amand`/`amor`/`amxor`/`ammax`/`ammin` 的 W/D 类要求 `rd ≠ rj`，否则触发指令未定义异常；`amswap`/`amadd` 的 B/H 变体允许 `rd = rj`。`rd = rk` 时结果不确定，软件应避免。

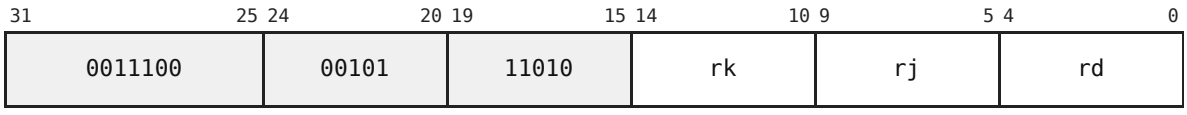
amadd.b `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = old + GR[rk]`

字节原子加法（Atomic Add Byte） · 3R 型 · LA64

原子加法：读出 `[rj]` 中的 8 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 8 位相加，结果写回 `[rj]`。整个“读—改—写”过程原子完成。

编码



编码值

```
0x385d0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amadd.b $a0, $a1, $a2      # [mem] += $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 8] = old + GR[rk][7:0]
```

---

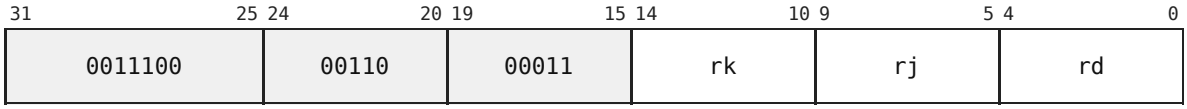
amadd.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

双字原子加法（Atomic Add Doubleword） · 3R 型 · LA64

原子加法：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38618000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amadd.d $a0, $a1, $a2      # [mem] += $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old + GR[rk][63:0]
```

---

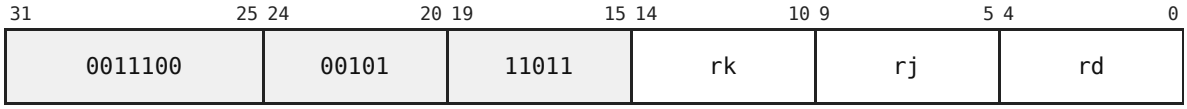
amadd.h rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

半字原子加法（Atomic Add Halfword） · 3R 型 · LA64

原子加法：读出 `[rj]` 中的 16 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 16 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x385d8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amadd.h $a0, $a1, $a2      # [mem] += $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 16] = old + GR[rk][15:0]
```

---

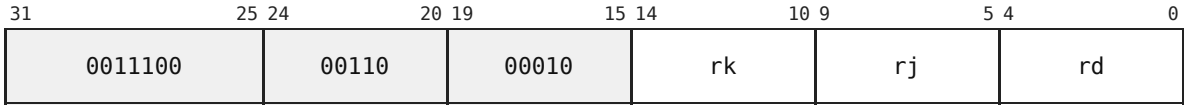
amadd.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

字原子加法（Atomic Add Word） · 3R 型 · LA64

原子加法：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38610000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amadd.w $a0, $a1, $a2      # [mem] += $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old + GR[rk][31:0]
```

---

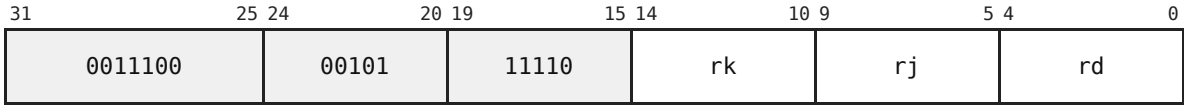
amadd_db.b rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

字节原子加法（Atomic Add Byte） · 3R 型 · LA64 · DB

原子加法：读出 `[rj]` 中的 8 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 8 位相加，结果写回 `[rj]`。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x385f0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amadd_db.b $a0, $a1, $a2      # [mem] += $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 8] = old + GR[rk][7:0]
DataBarrier()
```

---

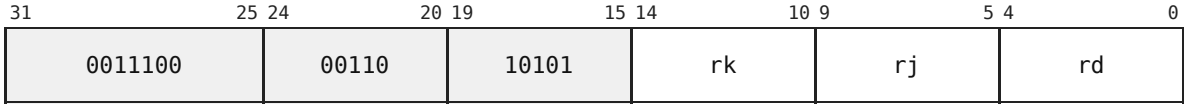
amadd_db.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

双字原子加法（Atomic Add Doubleword） · 3R 型 · LA64 · DB

原子加法：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386a8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amadd_db.d $a0, $a1, $a2      # [mem] += $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old + GR[rk][63:0]
DataBarrier()
```

---

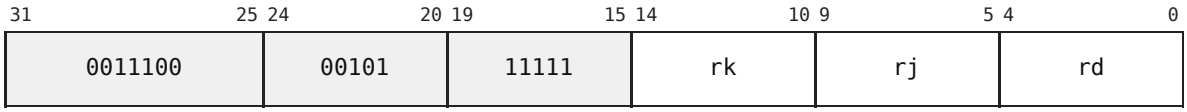
amadd_db.h rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

半字原子加法（Atomic Add Halfword） · 3R 型 · LA64 · DB

原子加法：读出 `[rj]` 中的 16 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 16 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385f8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amadd_db.h $a0, $a1, $a2      # [mem] += $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 16] = old + GR[rk][15:0]
DataBarrier()
```

---

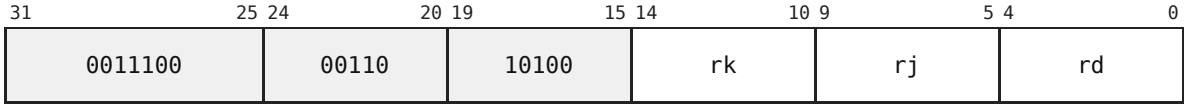
amadd_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old + GR[rk]

字原子加法（Atomic Add Word） · 3R 型 · LA64 · DB

原子加法：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位相加，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x386a0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amadd_db.w $a0, $a1, $a2      # [mem] += $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old + GR[rk][31:0]
DataBarrier()
```

---

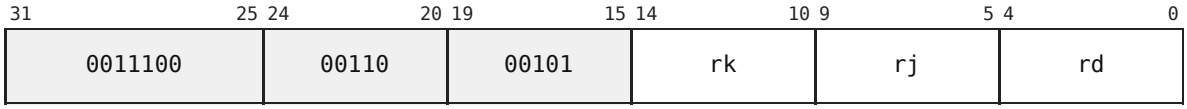
amand.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old & GR[rk]

双字原子按位与（Atomic AND Doubleword） · 3R 型 · LA64

原子按位与：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位与，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38628000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amand.d $a0, $a1, $a2      # [mem] &= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old & GR[rk][63:0]
```

---

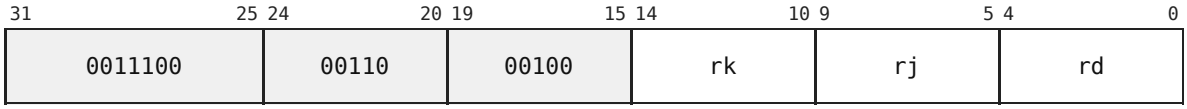
amand.w `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = old & GR[rk]`

字原子按位与 （Atomic AND Word） · 3R 型 · LA64

原子按位与：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位与，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38620000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amand.w $a0, $a1, $a2      # [mem] &= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old & GR[rk][31:0]
```

---

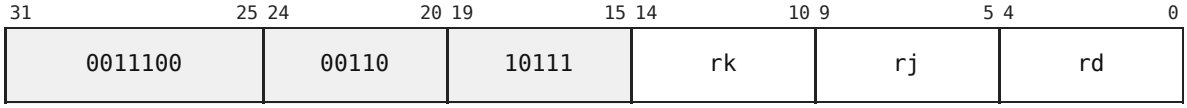
amand_db.d `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = old & GR[rk]`

双字原子按位与 （Atomic AND Doubleword） · 3R 型 · LA64 · DB

原子按位与：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位与，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x386b8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amand_db.d $a0, $a1, $a2      # [mem] &= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old & GR[rk][63:0]
DataBarrier()
```

---

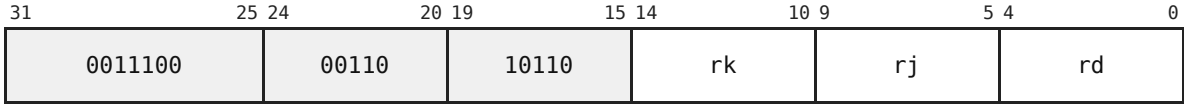
amand_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old & GR[rk]

字原子按位与 （Atomic AND Word） · 3R 型 · LA64 · DB

原子按位与：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位与，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386b0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amand_db.w $a0, $a1, $a2      # [mem] &= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old & GR[rk][31:0]
DataBarrier()
```

---

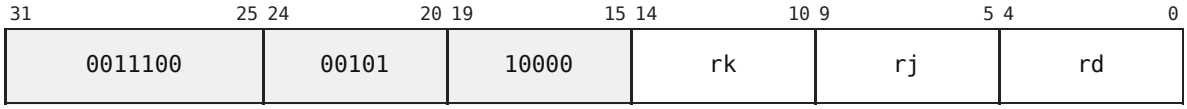
amcas.b rd, rk, rj

if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old

字节原子比较交换 （Atomic Compare and Swap Byte） · 3R 型 · LA64

原子比较交换：读出 `[rj]` 中的 8 位旧值，将旧值与 `rd` 的低 8 位比较；若相等，将 `rk` 的低 8 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。比较成功时，整个“读—改—写”过程原子完成。

编码



编码值

```
0x38580000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas.b $a0, $a1, $a2      # if ([mem] == $a0[7:0]) [mem] = $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
expected = GR[rd][7:0]
if (old == expected)
    Memory[GR[rj], 8] = GR[rk][7:0]
GR[rd] = SignExtend(old, GRLEN)
```

---

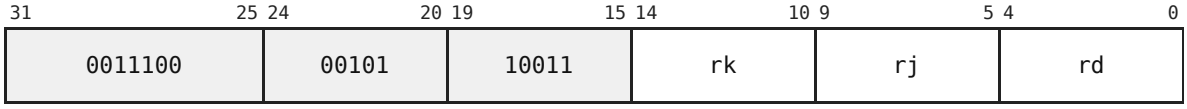
amcas.d rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

双字原子比较交换 （Atomic Compare and Swap Doubleword） · 3R 型 · LA64

原子比较交换：读出 `[rj]` 中的 64 位旧值，将旧值与 `rd` 的低 64 位比较；若相等，将 `rk` 的低 64 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。

编码



编码值

```
0x38598000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas.d $a0, $a1, $a2      # if ([mem] == $a0[63:0]) [mem] = $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
expected = GR[rd][63:0]
if (old == expected)
    Memory[GR[rj], 64] = GR[rk][63:0]
GR[rd] = SignExtend(old, GRLEN)
```

---

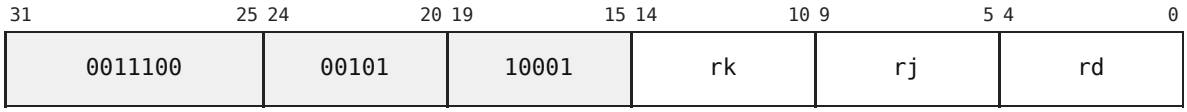
amcas.h rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

半字原子比较交换 （Atomic Compare and Swap Halfword） · 3R 型 · LA64

原子比较交换：读出 `[rj]` 中的 16 位旧值，将旧值与 `rd` 的低 16 位比较；若相等，将 `rk` 的低 16 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。

编码



编码值

```
0x38588000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas.h $a0, $a1, $a2      # if ([mem] == $a0[15:0]) [mem] = $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
expected = GR[rd][15:0]
if (old == expected)
    Memory[GR[rj], 16] = GR[rk][15:0]
GR[rd] = SignExtend(old, GRLEN)
```

---

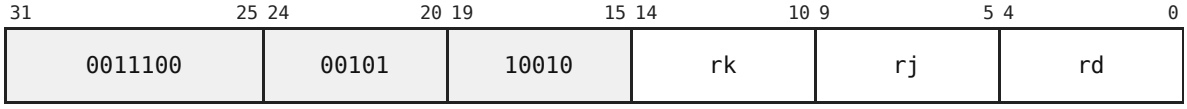
amcas.w rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

字原子比较交换（Atomic Compare and Swap Word） · 3R 型 · LA64

原子比较交换：读出 `[rj]` 中的 32 位旧值，将旧值与 `rd` 的低 32 位比较；若相等，将 `rk` 的低 32 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。

编码



编码值

```
0x38590000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas.w $a0, $a1, $a2      # if ([mem] == $a0[31:0]) [mem] = $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
expected = GR[rd][31:0]
if (old == expected)
    Memory[GR[rj], 32] = GR[rk][31:0]
GR[rd] = SignExtend(old, GRLEN)
```

---

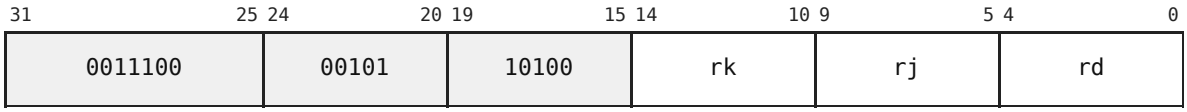
amcas_db.b rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

字节原子比较交换（Atomic Compare and Swap Byte） · 3R 型 · LA64 · DB

原子比较交换：读出 `[rj]` 中的 8 位旧值，将旧值与 `rd` 的低 8 位比较；若相等，将 `rk` 的低 8 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。比较成功时，整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385a0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas_db.b $a0, $a1, $a2      # if ([mem] == $a0[7:0]) [mem] = $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
expected = GR[rd][7:0]
if (old == expected)
    Memory[GR[rj], 8] = GR[rk][7:0]
GR[rd] = SignExtend(old, GRLEN)
DataBarrier()
```

---

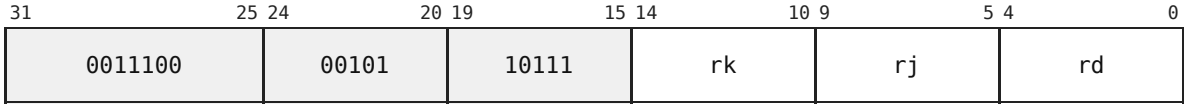
amcas_db.d rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

双字原子比较交换 （Atomic Compare and Swap Doubleword） · 3R 型 · LA64 · DB

原子比较交换：读出 `[rj]` 中的 64 位旧值，将旧值与 `rd` 的低 64 位比较；若相等，将 `rk` 的低 64 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385b8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas_db.d $a0, $a1, $a2      # if ([mem] == $a0[63:0]) [mem] = $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
expected = GR[rd][63:0]
if (old == expected)
    Memory[GR[rj], 64] = GR[rk][63:0]
GR[rd] = SignExtend(old, GRLEN)
DataBarrier()
```

---

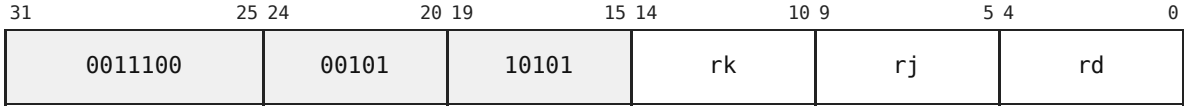
amcas_db.h rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

半字原子比较交换 （Atomic Compare and Swap Halfword） · 3R 型 · LA64 · DB

原子比较交换：读出 `[rj]` 中的 16 位旧值，将旧值与 `rd` 的低 16 位比较；若相等，将 `rk` 的低 16 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385a8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas_db.h $a0, $a1, $a2      # if ([mem] == $a0[15:0]) [mem] = $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
expected = GR[rd][15:0]
if (old == expected)
    Memory[GR[rj], 16] = GR[rk][15:0]
GR[rd] = SignExtend(old, GRLEN)
DataBarrier()
```

---

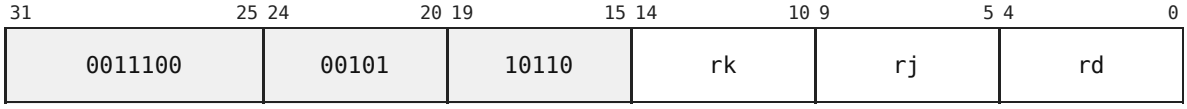
amcas_db.w rd, rk, rj

`if (old == GR[rd]) Mem[GR[rj]] = GR[rk]; GR[rd] = old`

字原子比较交换 （Atomic Compare and Swap Word） · 3R 型 · LA64 · DB

原子比较交换：读出 `[rj]` 中的 32 位旧值，将旧值与 `rd` 的低 32 位比较；若相等，将 `rk` 的低 32 位写入 `[rj]`。无论是否相等，旧值都符号扩展后写入 `rd`。地址必须自然对齐，否则触发非对齐异常。比较成功时，整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385b0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amcas_db.w $a0, $a1, $a2      # if ([mem] == $a0[31:0]) [mem] = $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
expected = GR[rd][31:0]
if (old == expected)
    Memory[GR[rj], 32] = GR[rk][31:0]
GR[rd] = SignExtend(old, GRLEN)
DataBarrier()
```

---

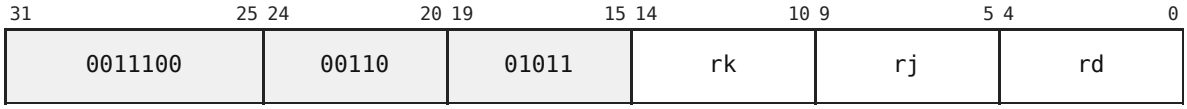
ammax.d rd, rk, rj

`GR[rd] = old; Mem[GR[rj]] = max(old, GR[rk])`

双字原子最大值 （Atomic Maximum Doubleword） · 3R 型 · LA64

原子最大值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 64 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38658000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax.d $a0, $a1, $a2      # [mem] = max([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (signed(old) > signed(GR[rk][63:0])) ? old : GR[rk][63:0]
```

---

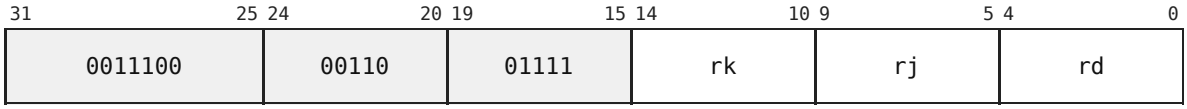
ammax.du rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = maxu(old, GR[rk])

双字原子无符号最大值 （Atomic Maximum Doubleword Unsigned） · 3R 型 · LA64

原子最大值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 64 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38678000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax.du $a0, $a1, $a2      # [mem] = maxu([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (unsigned(old) > unsigned(GR[rk][63:0])) ? old : GR[rk][63:0]
```

---

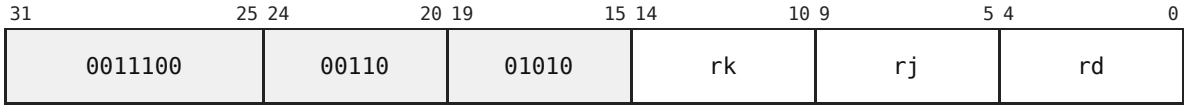
ammax.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = max(old, GR[rk])

字原子最大值 （Atomic Maximum Word） · 3R 型 · LA64

原子最大值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 32 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38650000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax.w $a0, $a1, $a2      # [mem] = max([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (signed(old) > signed(GR[rk][31:0])) ? old : GR[rk][31:0]
```

---

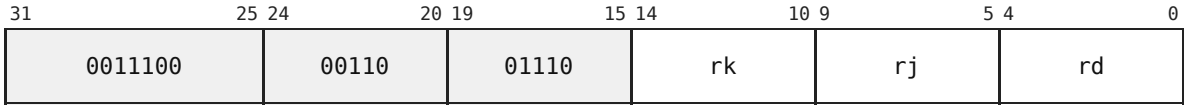
ammax.wu rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = maxu(old, GR[rk])

字原子无符号最大值 （Atomic Maximum Word Unsigned） · 3R 型 · LA64

原子最大值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 32 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38670000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax.wu $a0, $a1, $a2      # [mem] = maxu([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (unsigned(old) > unsigned(GR[rk][31:0])) ? old : GR[rk][31:0]
```

---

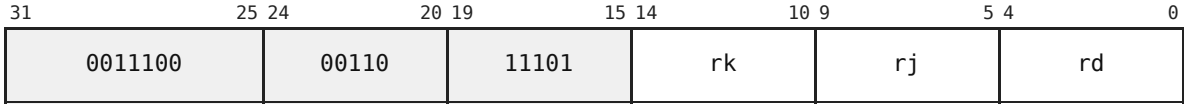
ammax_db.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = max(old, GR[rk])

双字原子最大值 （Atomic Maximum Doubleword） · 3R 型 · LA64 · DB

原子最大值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 64 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386e8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax_db.d $a0, $a1, $a2      # [mem] = max([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (signed(old) > signed(GR[rk][63:0])) ? old : GR[rk][63:0]
DataBarrier()
```

---

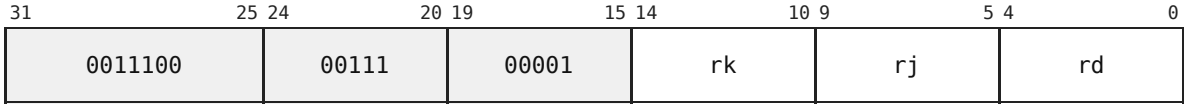
ammax_db.du rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = maxu(old, GR[rk])

双字原子无符号最大值（Atomic Maximum Doubleword Unsigned） · 3R 型 · LA64 · DB

原子最大值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 64 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x38708000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax_db.du $a0, $a1, $a2      # [mem] = maxu([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (unsigned(old) > unsigned(GR[rk][63:0])) ? old : GR[rk][63:0]
DataBarrier()
```

---

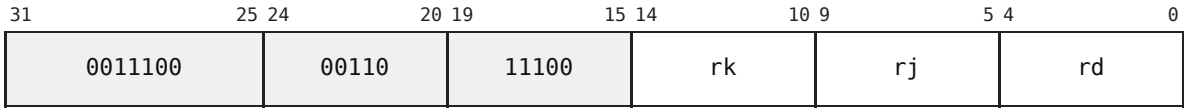
ammax_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = max(old, GR[rk])

字原子最大值（Atomic Maximum Word） · 3R 型 · LA64 · DB

原子最大值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 32 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386e0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax_db.w $a0, $a1, $a2      # [mem] = max([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (signed(old) > signed(GR[rk][31:0])) ? old : GR[rk][31:0]
DataBarrier()
```

---

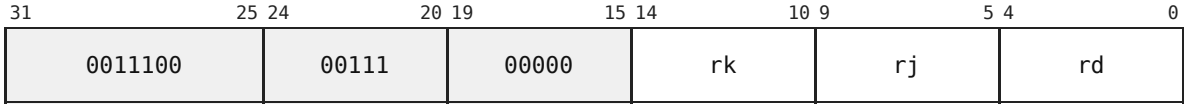
ammax_db.wu rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = maxu(old, GR[rk])

字原子无符号最大值 （Atomic Maximum Word Unsigned） · 3R 型 · LA64 · DB

原子最大值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 32 位，将较大值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x38700000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammax_db.wu $a0, $a1, $a2      # [mem] = maxu([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (unsigned(old) > unsigned(GR[rk][31:0])) ? old : GR[rk][31:0]
DataBarrier()
```

---

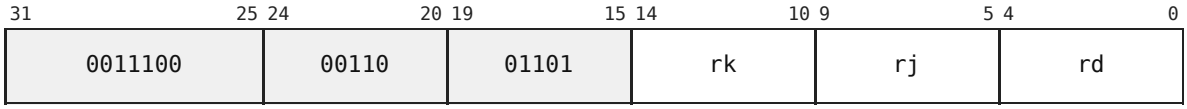
ammin.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = min(old, GR[rk])

双字原子最小值 （Atomic Minimum Doubleword） · 3R 型 · LA64

原子最小值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 64 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38668000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin.d $a0, $a1, $a2      # [mem] = min([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (signed(old) < signed(GR[rk][63:0])) ? old : GR[rk][63:0]
```

---

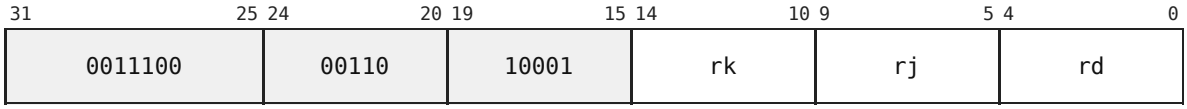
ammin.du rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = minu(old, GR[rk])

双字原子无符号最小值 （Atomic Minimum Doubleword Unsigned） · 3R 型 · LA64

原子最小值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 64 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38688000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin.du $a0, $a1, $a2      # [mem] = minu([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (unsigned(old) < unsigned(GR[rk][63:0])) ? old : GR[rk][63:0]
```

---

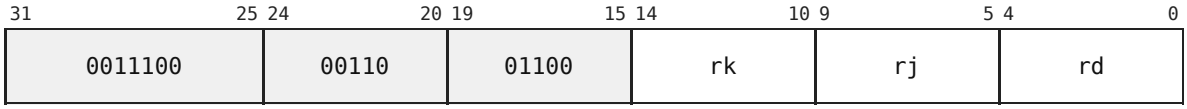
ammin.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = min(old, GR[rk])

字原子最小值 （Atomic Minimum Word） · 3R 型 · LA64

原子最小值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 32 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38660000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin.w $a0, $a1, $a2      # [mem] = min([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (signed(old) < signed(GR[rk][31:0])) ? old : GR[rk][31:0]
```

---

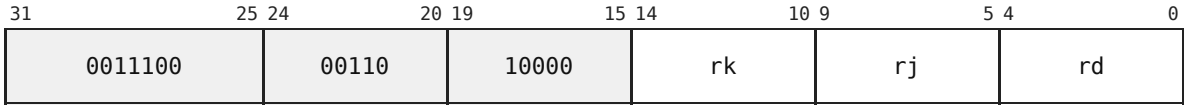
ammin.wu `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = minu(old, GR[rk])`

字原子无符号最小值 （Atomic Minimum Word Unsigned） · 3R 型 · LA64

原子最小值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 32 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38680000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin.wu $a0, $a1, $a2      # [mem] = minu([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (unsigned(old) < unsigned(GR[rk][31:0])) ? old : GR[rk][31:0]
```

---

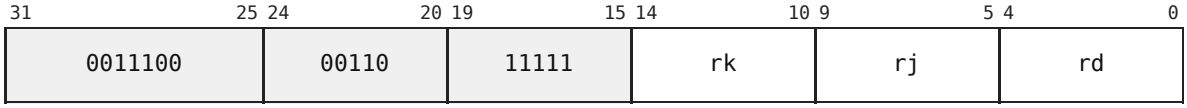
ammin_db.d `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = min(old, GR[rk])`

双字原子最小值 （Atomic Minimum Doubleword） · 3R 型 · LA64 · DB

原子最小值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 64 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386f8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin_db.d $a0, $a1, $a2      # [mem] = min([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (signed(old) < signed(GR[rk][63:0])) ? old : GR[rk][63:0]
DataBarrier()
```

---

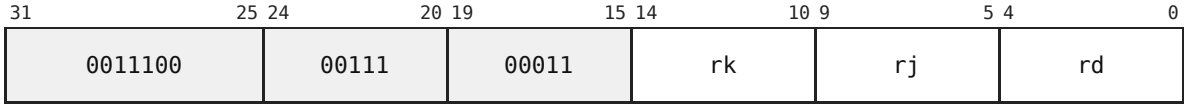
ammin_db.du rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = minu(old, GR[rk])

双字原子无符号最小值（Atomic Minimum Doubleword Unsigned） · 3R 型 · LA64 · DB

原子最小值：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 64 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x38718000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin_db.du $a0, $a1, $a2      # [mem] = minu([mem], $a1[63:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = (unsigned(old) < unsigned(GR[rk][63:0])) ? old : GR[rk][63:0]
DataBarrier()
```

---

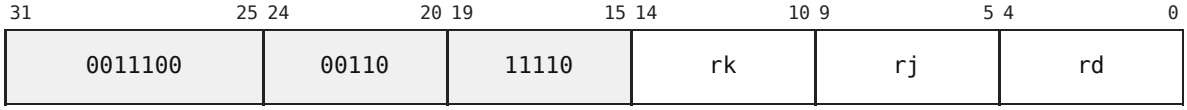
ammin_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = min(old, GR[rk])

字原子最小值（Atomic Minimum Word） · 3R 型 · LA64 · DB

原子最小值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按有符号数比较旧值与 `rk` 的低 32 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386f0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ammin_db.w $a0, $a1, $a2      # [mem] = min([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (signed(old) < signed(GR[rk][31:0])) ? old : GR[rk][31:0]
DataBarrier()
```

---

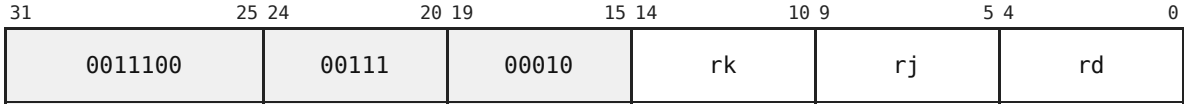
ammin_db.wu rd, rk, rj

`GR[rd] = old; Mem[GR[rj]] = minu(old, GR[rk])`

字原子无符号最小值 （Atomic Minimum Word Unsigned） · 3R 型 · LA64 · DB

原子最小值：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；按无符号数比较旧值与 `rk` 的低 32 位，将较小值写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x38710000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ammin_db.wu $a0, $a1, $a2      # [mem] = minu([mem], $a1[31:0]); $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = (unsigned(old) < unsigned(GR[rk][31:0])) ? old : GR[rk][31:0]
DataBarrier()
```

---

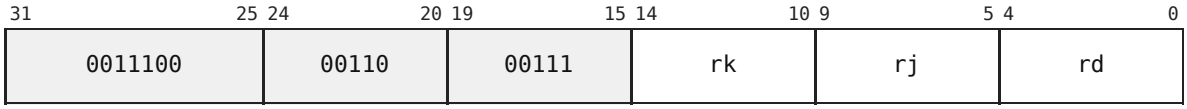
amor.d rd, rk, rj

`GR[rd] = old; Mem[GR[rj]] = old | GR[rk]`

双字原子按位或 （Atomic OR Doubleword） · 3R 型 · LA64

原子按位或：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38638000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amor.d $a0, $a1, $a2      # [mem] |= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old | GR[rk][63:0]
```

---

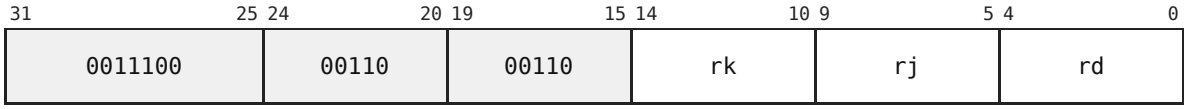
amor.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old | GR[rk]

字原子按位或（Atomic OR Word） · 3R 型 · LA64

原子按位或：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38630000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amor.w $a0, $a1, $a2      # [mem] |= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old | GR[rk][31:0]
```

---

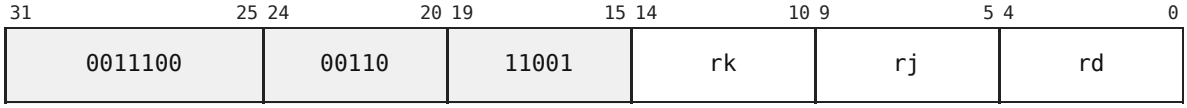
amor_db.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old | GR[rk]

双字原子按位或（Atomic OR Doubleword） · 3R 型 · LA64 · DB

原子按位或：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386c8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amor_db.d $a0, $a1, $a2      # [mem] |= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old | GR[rk][63:0]
DataBarrier()
```

---

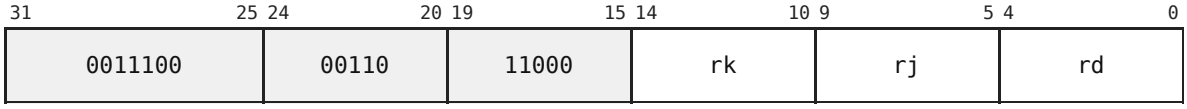
amor_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old | GR[rk]

字原子按位或（Atomic OR Word） · 3R 型 · LA64 · DB

原子按位或：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386c0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amor_db.w $a0, $a1, $a2      # [mem] |= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old | GR[rk][31:0]
DataBarrier()
```

---

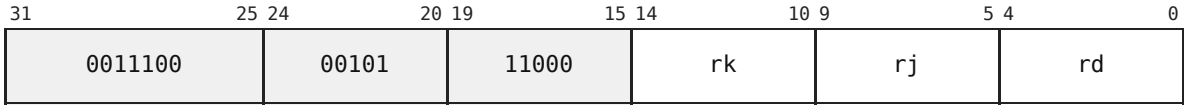
amswap.b rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

字节原子交换（Atomic Swap Byte） · 3R 型 · LA64

原子交换：读出 `[rj]` 中的 8 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 8 位写入 `[rj]`。整个“读—改—写”过程原子完成。

编码



编码值

```
0x385c0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amswap.b $a0, $a1, $a2      # [mem] = $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 8] = GR[rk][7:0]
```

---

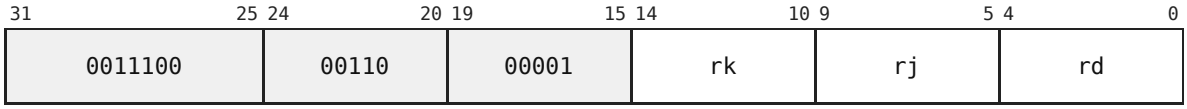
amswap.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

双字原子交换（Atomic Swap Doubleword） · 3R 型 · LA64

原子交换：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 64 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x38608000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amswap.d $a0, $a1, $a2      # [mem] = $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = GR[rk][63:0]
```

---

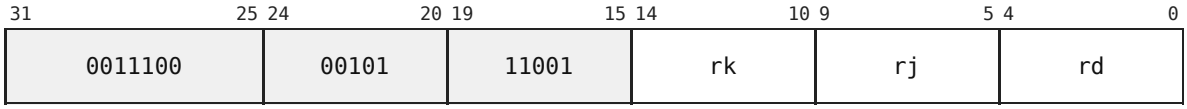
amswap.h rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

半字原子交换（Atomic Swap Halfword） · 3R 型 · LA64

原子交换：读出 `[rj]` 中的 16 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 16 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值 `0x385c8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amswap.h $a0, $a1, $a2      # [mem] = $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 16] = GR[rk][15:0]
```

---

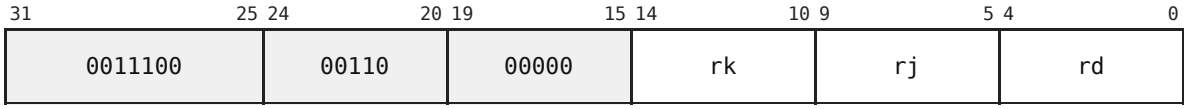
amswap.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

字原子交换 （Atomic Swap Word） · 3R 型 · LA64

原子交换：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 32 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38600000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amswap.w $a0, $a1, $a2      # [mem] = $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = GR[rk][31:0]
```

---

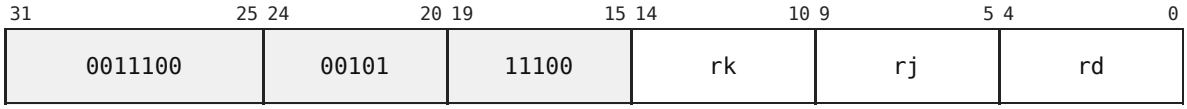
amswap_db.b rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

字节原子交换 （Atomic Swap Byte） · 3R 型 · LA64 · DB

原子交换：读出 `[rj]` 中的 8 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 8 位写入 `[rj]`。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x385e0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amswap_db.b $a0, $a1, $a2      # [mem] = $a1[7:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 8]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 8] = GR[rk][7:0]
DataBarrier()
```

---

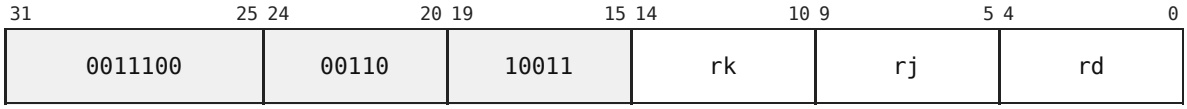
amswap_db.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

双字原子交换 （Atomic Swap Doubleword） · 3R 型 · LA64 · DB

原子交换：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 64 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x38698000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amswap_db.d $a0, $a1, $a2      # [mem] = $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = GR[rk][63:0]
DataBarrier()
```

---

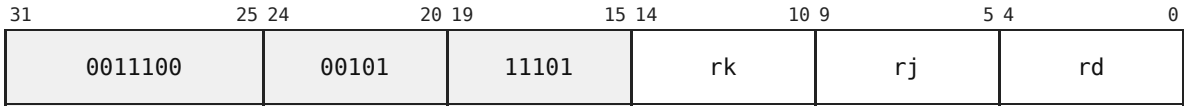
amswap_db.h rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

半字原子交换 （Atomic Swap Halfword） · 3R 型 · LA64 · DB

原子交换：读出 `[rj]` 中的 16 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 16 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值 `0x385e8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
amswap_db.h $a0, $a1, $a2      # [mem] = $a1[15:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 16]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 16] = GR[rk][15:0]
DataBarrier()
```

---

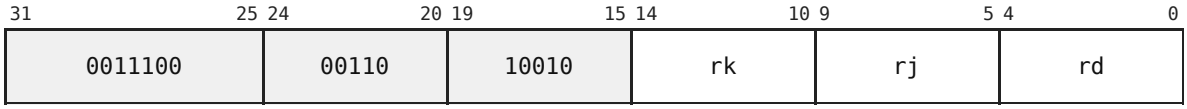
amswap_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = GR[rk]

字原子交换 （Atomic Swap Word） · 3R 型 · LA64 · DB

原子交换：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将 `rk` 的低 32 位写入 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x38690000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amswap_db.w $a0, $a1, $a2      # [mem] = $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = GR[rk][31:0]
DataBarrier()
```

---

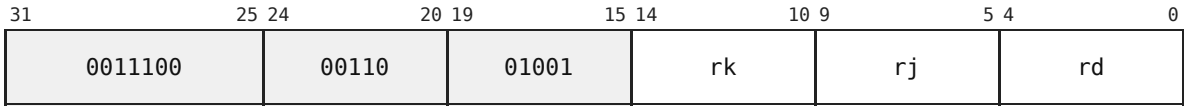
amxor.d rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old ^ GR[rk]

双字原子按位异或 （Atomic XOR Doubleword） · 3R 型 · LA64

原子按位异或：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位异或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38648000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amxor.d $a0, $a1, $a2      # [mem] ^= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old ^ GR[rk][63:0]
```

---

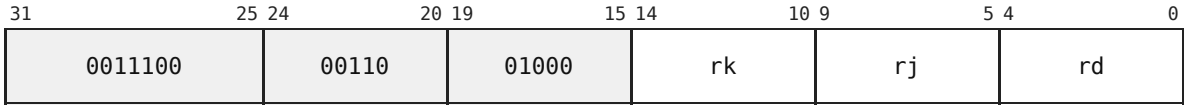
amxor.w `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = old ^ GR[rk]`

字原子按位异或（Atomic XOR Word） · 3R 型 · LA64

原子按位异或：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位异或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。

编码



编码值

```
0x38640000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amxor.w $a0, $a1, $a2      # [mem] ^= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old ^ GR[rk][31:0]
```

---

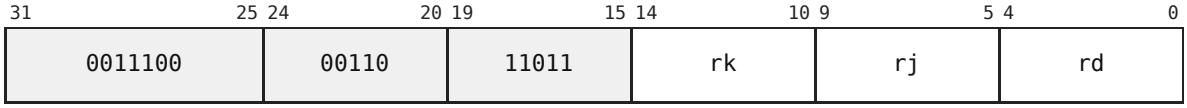
amxor_db.d `rd, rk, rj`

`GR[rd] = old; Mem[GR[rj]] = old ^ GR[rk]`

双字原子按位异或（Atomic XOR Doubleword） · 3R 型 · LA64 · DB

原子按位异或：读出 `[rj]` 中的 64 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 64 位按位异或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386d8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amxor_db.d $a0, $a1, $a2      # [mem] ^= $a1[63:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 64]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 64] = old ^ GR[rk][63:0]
DataBarrier()
```

---

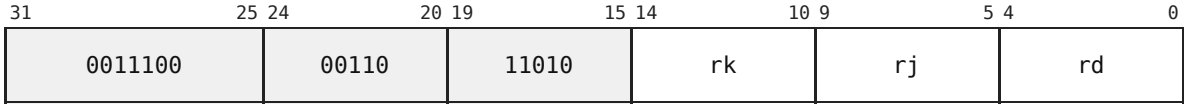
amxor_db.w rd, rk, rj

GR[rd] = old; Mem[GR[rj]] = old ^ GR[rk]

字原子按位异或 （Atomic XOR Word） · 3R 型 · LA64 · DB

原子按位异或：读出 `[rj]` 中的 32 位旧值，符号扩展至 GRLEN 位后写入 `rd`；将旧值与 `rk` 的低 32 位按位异或，结果写回 `[rj]`。地址必须自然对齐，否则触发非对齐异常。整个“读—改—写”过程原子完成。该指令附加数据屏障（DB）语义。

编码



编码值

```
0x386d0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
amxor_db.w $a0, $a1, $a2      # [mem] ^= $a1[31:0]; $a0 = old
```

语义伪代码

```
old = Memory[GR[rj], 32]
GR[rd] = SignExtend(old, GRLEN)
Memory[GR[rj], 32] = old ^ GR[rk][31:0]
DataBarrier()
```

---

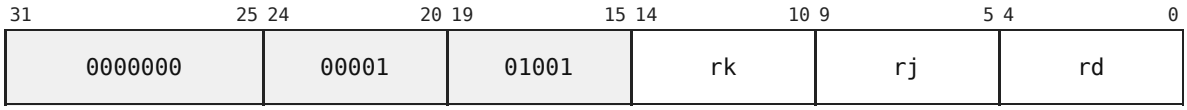
and rd, rj, rk

GR[rd] = GR[rj] & GR[rk]

按位与 （Bitwise AND） · 3R 型 · LA32 / LA64

将 `rj` 与 `rk` 按位与，结果写入 `rd`。操作数长度为 GRLEN 位。

编码



编码值

```
0x00148000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
and $a0, $a1, $a2      # $a0 = $a1 & $a2
```

语义伪代码

```
GR[rd] = GR[rj] & GR[rk]
```

---

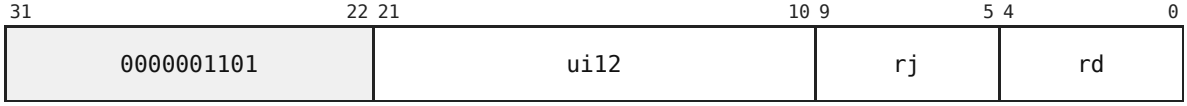
andi rd, rj, ui12

GR[rd] = GR[rj] & ZeroExtend(ui12, GRLEN)

立即数按位与 （Bitwise AND Immediate） · 2RI12 型 · LA32 / LA64

将 `rj` 与零扩展后的 12 位无符号立即数 `ui12` 按位与，结果写入 `rd`。`ui12` 的取值范围为 0～4095。

编码



编码值

```
0x03400000 | ((ui12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
andi $a0, $a1, 0xff      # $a0 = $a1 & 0xff
```

语义伪代码

```
GR[rd] = GR[rj] & ZeroExtend(ui12, GRLEN)
```

---

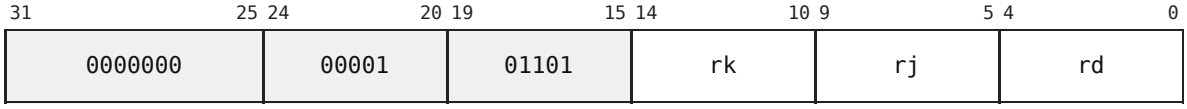
andn `rd, rj, rk`

`GR[rd] = GR[rj] & ~GR[rk]`

按位与非 （Bitwise AND NOT） · 3R 型 · LA32 / LA64

将 `rk` 按位取反后与 `rj` 按位与，结果写入 `rd`。操作数长度为 GRLEN 位。

编码



编码值

```
0x00168000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
andn $a0, $a1, $a2      # $a0 = $a1 & ~$a2
```

语义伪代码

```
GR[rd] = GR[rj] & (~GR[rk])
```

---

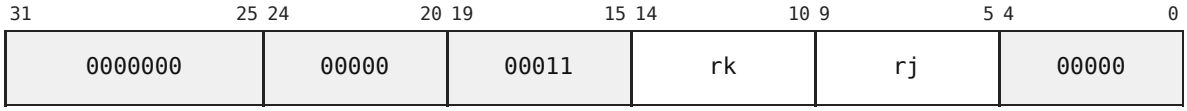
asrtgt.d `rj, rk`

`if (signed(GR[rj]) <= signed(GR[rk])) raise BCE`

地址下界检查（大于） （Assert Greater Than Doubleword） · 2R 型 · LA64

将 `rj`、`rk` 作为有符号数比较；若 `rj <= rk`，触发边界检查异常（BCE）。用于确认地址 `rj` 高于下界 `rk`。

编码



编码值

```
0x00018000 | (rk << 10) | (rj << 5)
```

汇编示例

```
asrtgt.d $a0, $a1      # if ($a0 <= $a1) raise BCE
```

语义伪代码

```
if (signed(GR[rj]) <= signed(GR[rk]))
    RaiseException(BCE)
```

---

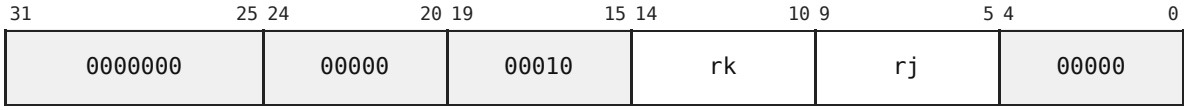
asrtle.d `rj, rk`

`if (signed(GR[rj]) > signed(GR[rk])) raise BCE`

地址上界检查（小于等于） （Assert Less Than or Equal Doubleword） · 2R 型 · LA64

将 `rj`、`rk` 作为有符号数比较；若 `rj > rk`，触发边界检查异常（BCE）。用于确认地址 `rj` 不超过上界 `rk`。

编码



编码值

```
0x00010000 | (rk << 10) | (rj << 5)
```

汇编示例

```
asrtle.d $a0, $a1      # if ($a0 > $a1) raise BCE
```

语义伪代码

```
if (signed(GR[rj]) > signed(GR[rk]))
    RaiseException(BCE)
```

B · 位操作、分支与断点指令

本章收录 23 条以 B 开头的指令。分支指令的 `offs` 为汇编中的字节偏移；位图中的 `offs[...]` 表示编码后的字偏移字段（`offs >> 2`）。`offs16`、`offs21`、`offs26` 的字节范围分别约为 ±128 KB、±4 MB、±128 MB。`bceqz/bcnez` 的 `cj`

为条件标志寄存器号 0～7，汇编名为 `$fcc0`～`$fcc7`。

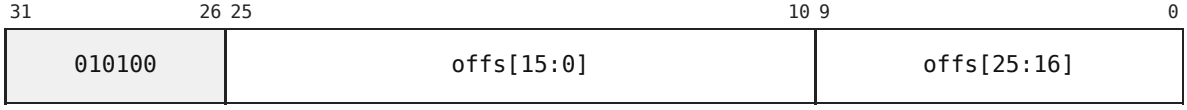
b `offs`

`PC = PC + SignExtend(offs, GRLEN)`

无条件跳转 （Unconditional Branch） · I26 型 · LA32 / LA64

无条件跳转到 `PC + offs`。`offs` 为 26 位有符号字偏移 `offs26 << 2`，即字节偏移。

编码



编码值

```
0x50000000 | (((offs >> 2) & 0xffff) << 10) | (((offs >> 2) >> 16) & 0x3ff)
```

汇编示例

```
b 8      # PC = PC + 8
```

语义伪代码

```
PC = PC + SignExtend(offs, GRLEN)
```

---

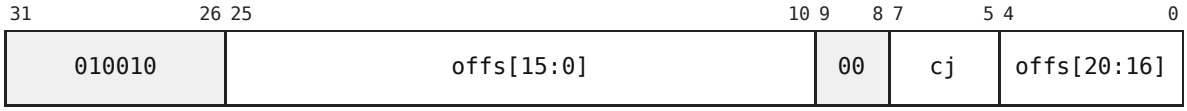
bceqz `cj, offs`

`if (CFR[cj] == 0) PC = PC + SignExtend(offs, GRLEN)`

条件标志为零跳转 （Branch if Condition Flag Equal to Zero） · 1RI21 型 · LA32 / LA64 · F

当条件标志寄存器 `cj` 的值为 0 时跳转到 `PC + offs`，否则顺序执行。`cj` 为 0～7，汇编名为 `$fcc0`～`$fcc7`。

编码



编码值

```
0x48000000 | (((offs >> 2) & 0xffff) << 10) | (cj << 5) | (((offs >> 2) >> 16) & 0x1f)
```

汇编示例

```
bceqz $fcc0, 8      # if ($fcc0 == 0) PC += 8
```

语义伪代码

```
if (CFR[cj] == 0)
    PC = PC + SignExtend(offs, GRLEN)
```

---

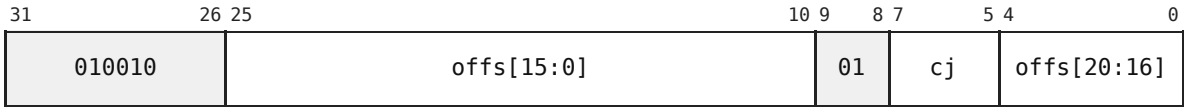
bcnez `cj, offs`

`if (CFR[cj] != 0) PC = PC + SignExtend(offs, GRLEN)`

条件标志非零跳转 （Branch if Condition Flag Not Equal to Zero） · 1RI21 型 · LA32 / LA64 · F

当条件标志寄存器 `cj` 的值不为 0 时跳转到 `PC + offs`，否则顺序执行。`cj` 为 0～7，汇编名为 `$fcc0`～`$fcc7`。

编码



编码值

```
0x48000100 | (((offs >> 2) & 0xffff) << 10) | (cj << 5) | (((offs >> 2) >> 16) & 0x1f)
```

汇编示例

```
bcnez $fcc0, 8      # if ($fcc0 != 0) PC += 8
```

语义伪代码

```
if (CFR[cj] != 0)
    PC = PC + SignExtend(offs, GRLEN)
```

---

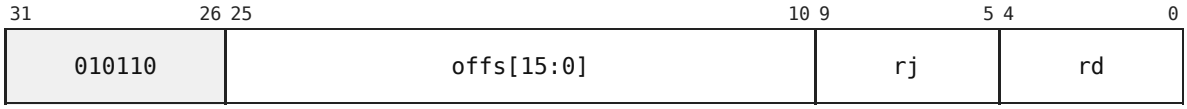
beq `rj, rd, offs`

`if (GR[rj] == GR[rd]) PC = PC + SignExtend(offs, GRLEN)`

相等跳转 （Branch if Equal） · 2RI16 型 · LA32 / LA64

比较 `rj` 与 `rd`，相等时跳转到 `PC + offs`，否则顺序执行。`offs` 为 16 位有符号字偏移 `offs16 << 2`，即字节偏移。

编码



编码值 `0x58000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd`

汇编示例

```
beq $a0, $a1, 8      # if ($a0 == $a1) PC += 8
```

语义伪代码

```
if (GR[rj] == GR[rd])
    PC = PC + SignExtend(offs, GRLEN)
```

---

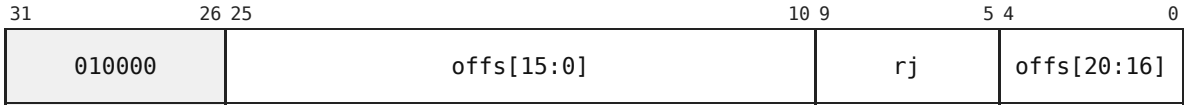
beqz `rj, offs`

`if (GR[rj] == 0) PC = PC + SignExtend(offs, GRLEN)`

等于零跳转 （Branch if Equal to Zero） · 1RI21 型 · LA32 / LA64

检查 `rj` 的值，等于 0 时跳转到 `PC + offs`，否则顺序执行。`offs` 为 21 位有符号字偏移 `offs21 << 2`，即字节偏移。

编码



编码值 `0x40000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | (((offs >> 2) >> 16) & 0x1f)`

汇编示例

```
beqz $a0, 8      # if ($a0 == 0) PC += 8
```

语义伪代码

```
if (GR[rj] == 0)
    PC = PC + SignExtend(offs, GRLEN)
```

---

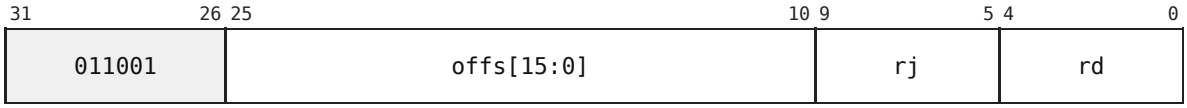
bge `rj, rd, offs`

`if (signed(GR[rj]) >= signed(GR[rd])) PC = PC + SignExtend(offs, GRLEN)`

有符号大于等于跳转 （Branch if Greater Than or Equal） · 2RI16 型 · LA32 / LA64

将 `rj`、`rd` 作为有符号数比较，前者大于等于后者时跳转到 `PC + offs`，否则顺序执行。`offs` 为字节偏移。

编码



编码值 `0x64000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd`

汇编示例

```
bge $a0, $a1, 8      # if ($a0 >= $a1) PC += 8
```

语义伪代码

```
if (signed(GR[rj]) >= signed(GR[rd]))
    PC = PC + SignExtend(offs, GRLEN)
```

---

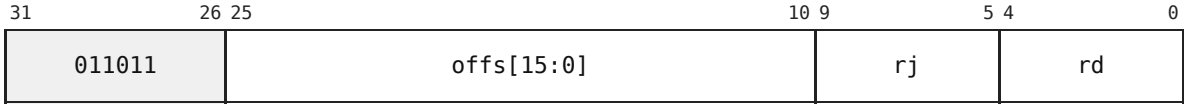
bgeu `rj, rd, offs`

`if (unsigned(GR[rj]) >= unsigned(GR[rd])) PC = PC + SignExtend(offs, GRLEN)`

无符号大于等于跳转 （Branch if Greater Than or Equal Unsigned） · 2RI16 型 · LA32 / LA64

将 `rj`、`rd` 作为无符号数比较，前者大于等于后者时跳转到 `PC + offs`，否则顺序执行。`offs` 为字节偏移。

编码



编码值

```
0x6c000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd
```

汇编示例

```
bgeu $a0, $a1, 8      # if ((unsigned)$a0 >= (unsigned)$a1) PC += 8
```

语义伪代码

```
if (unsigned(GR[rj]) >= unsigned(GR[rd]))
    PC = PC + SignExtend(offs, GRLEN)
```

---

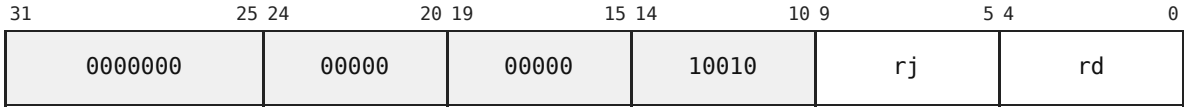
bitrev.4b `rd, rj`

`GR[rd] = SignExtend(BITREV_EACH_BYTE(GR[rj][31:0]), GRLEN)`

4 字节位反转 （Bit Reverse 4 Bytes） · 2R 型 · LA32 / LA64

将 `rj` 低 32 位中的 4 个字节分别按位反转，得到的 32 位结果符号扩展至 GRLEN 位后写入 `rd`。

编码



编码值

```
0x00004800 | (rj << 5) | rd
```

汇编示例

```
bitrev.4b $a0, $a1      # 每个字节内按位反转
```

语义伪代码

```
bstr32[31:24] = BITREV(GR[rj][31:24])
bstr32[23:16] = BITREV(GR[rj][23:16])
bstr32[15:8]  = BITREV(GR[rj][15:8])
bstr32[7:0]   = BITREV(GR[rj][7:0])
GR[rd] = SignExtend(bstr32, GRLEN)
```

---

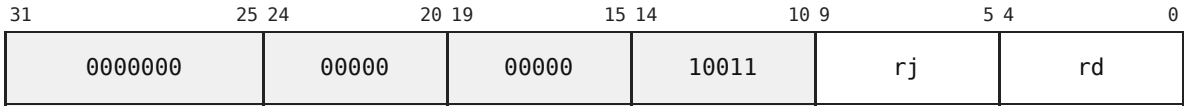
bitrev.8b rd, rj

GR[rd] = BITREV_EACH_BYTE(GR[rj][63:0])

8 字节位反转 （Bit Reverse 8 Bytes） · 2R 型 · LA64

将 `rj` 中的 8 个字节分别按位反转，64 位结果写入 `rd`。

编码



编码值 `0x00004c00 | (rj << 5) | rd`

汇编示例

```
bitrev.8b $a0, $a1      # 8 个字节分别按位反转
```

语义伪代码

```
GR[rd][63:56] = BITREV(GR[rj][63:56])
GR[rd][55:48] = BITREV(GR[rj][55:48])
GR[rd][47:40] = BITREV(GR[rj][47:40])
GR[rd][39:32] = BITREV(GR[rj][39:32])
GR[rd][31:24] = BITREV(GR[rj][31:24])
GR[rd][23:16] = BITREV(GR[rj][23:16])
GR[rd][15:8]  = BITREV(GR[rj][15:8])
GR[rd][7:0]   = BITREV(GR[rj][7:0])
```

---

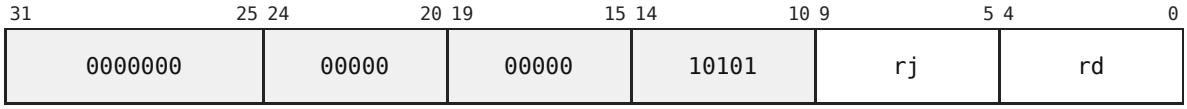
bitrev.d rd, rj

GR[rd] = BITREV(GR[rj][63:0])

双字位反转 （Bit Reverse Doubleword） · 2R 型 · LA64

将 `rj` 的 64 位数据整体按位反转，结果写入 `rd`。

编码



编码值 `0x00005400 | (rj << 5) | rd`

汇编示例

```
bitrev.d $a0, $a1      # $a0 = bitrev($a1)
```

语义伪代码

```
GR[rd] = BITREV(GR[rj][63:0])
```

---

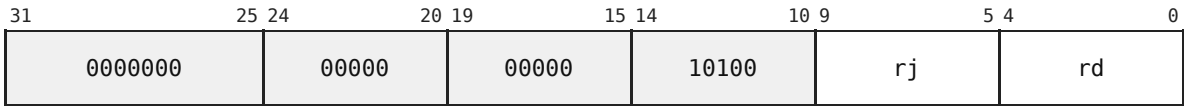
bitrev.w rd, rj

GR[rd] = SignExtend(BITREV(GR[rj][31:0]), GRLEN)

字位反转 （Bit Reverse Word） · 2R 型 · LA32 / LA64

将 `rj` 的低 32 位整体按位反转，32 位结果符号扩展至 GRLEN 位后写入 `rd`。

编码



编码值 `0x00005000 | (rj << 5) | rd`

汇编示例

```
bitrev.w $a0, $a1      # $a0 = bitrev($a1[31:0])
```

语义伪代码

```
bstr32 = BITREV(GR[rj][31:0])
GR[rd] = SignExtend(bstr32, GRLEN)
```

---

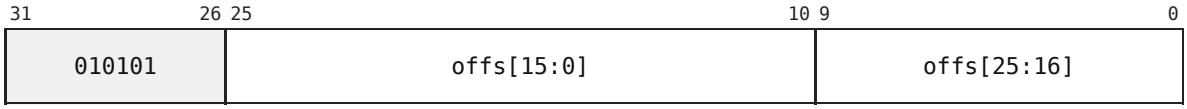
bl `offs`

`GR[1] = PC + 4; PC = PC + SignExtend(offs, GRLEN)`

无条件跳转并链接 （Branch and Link） · I26 型 · LA32 / LA64

无条件跳转到 `PC + offs`，并将 `PC + 4` 写入 `r1`（返回地址寄存器 `ra`）。`offs` 为 26 位有符号字偏移 `offs26 << 2`，即字节偏移。

编码



编码值

```
0x54000000 | (((offs >> 2) & 0xffff) << 10) | (((offs >> 2) >> 16) & 0x3ff)
```

汇编示例

```
bl 8      # r1 = PC + 4; PC += 8
```

语义伪代码

```
GR[1] = PC + 4
PC = PC + SignExtend(offs, GRLEN)
```

---

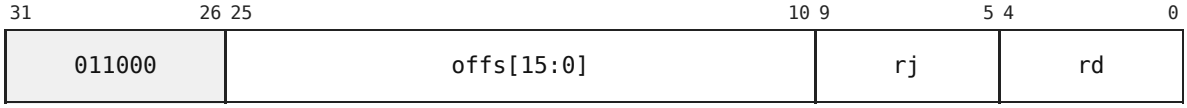
blt `rj, rd, offs`

`if (signed(GR[rj]) < signed(GR[rd])) PC = PC + SignExtend(offs, GRLEN)`

有符号小于跳转 （Branch if Less Than） · 2RI16 型 · LA32 / LA64

将 `rj`、`rd` 作为有符号数比较，前者小于后者时跳转到 `PC + offs`，否则顺序执行。`offs` 为字节偏移。

编码



编码值

```
0x60000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd
```

汇编示例

```
blt $a0, $a1, 8      # if ($a0 < $a1) PC += 8
```

语义伪代码

```
if (signed(GR[rj]) < signed(GR[rd]))
    PC = PC + SignExtend(offs, GRLEN)
```

---

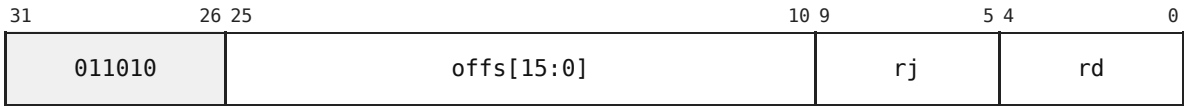
bltu `rj, rd, offs`

`if (unsigned(GR[rj]) < unsigned(GR[rd])) PC = PC + SignExtend(offs, GRLEN)`

无符号小于跳转 （Branch if Less Than Unsigned） · 2RI16 型 · LA32 / LA64

将 `rj`、`rd` 作为无符号数比较，前者小于后者时跳转到 `PC + offs`，否则顺序执行。`offs` 为字节偏移。

编码



编码值

```
0x68000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd
```

汇编示例

```
bltu $a0, $a1, 8      # if ((unsigned)$a0 < (unsigned)$a1) PC += 8
```

语义伪代码

```
if (unsigned(GR[rj]) < unsigned(GR[rd]))
    PC = PC + SignExtend(offs, GRLEN)
```

---

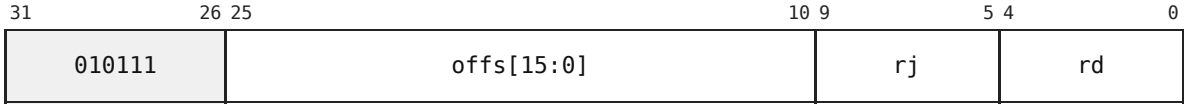
bne `rj, rd, offs`

`if (GR[rj] != GR[rd]) PC = PC + SignExtend(offs, GRLEN)`

不相等跳转 （Branch if Not Equal） · 2RI16 型 · LA32 / LA64

比较 `rj` 与 `rd`，不相等时跳转到 `PC + offs`，否则顺序执行。`offs` 为字节偏移。

编码



编码值

```
0x5c000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd
```

汇编示例

```
bne $a0, $a1, 8      # if ($a0 != $a1) PC += 8
```

语义伪代码

```
if (GR[rj] != GR[rd])
    PC = PC + SignExtend(offs, GRLEN)
```

---

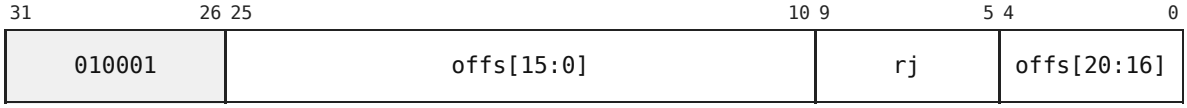
bnez `rj, offs`

`if (GR[rj] != 0) PC = PC + SignExtend(offs, GRLEN)`

不等于零跳转 （Branch if Not Equal to Zero） · 1RI21 型 · LA32 / LA64

检查 `rj` 的值，不等于 0 时跳转到 `PC + offs`，否则顺序执行。`offs` 为 21 位有符号字偏移 `offs21 << 2`，即字节偏移。

编码



编码值

```
0x44000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | (((offs >> 2) >> 16) & 0x1f)
```

汇编示例

```
bnez $a0, 8      # if ($a0 != 0) PC += 8
```

语义伪代码

```
if (GR[rj] != 0)
    PC = PC + SignExtend(offs, GRLEN)
```

---

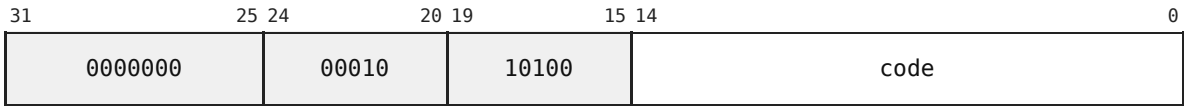
break `code`

`raise BRK(code)`

断点异常 （Breakpoint） · I15 型 · LA32 / LA64

无条件触发断点异常（BRK）。`code` 为 15 位立即数，可作为异常处理程序读取的参数。

编码



编码值

```
0x002a0000 | ((code & 0x7fff) << 0)
```

汇编示例

```
break 1      # 触发断点异常，code = 1
```

语义伪代码

```
RaiseException(BRK, code)
```

---

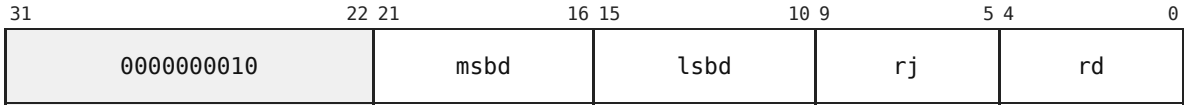
bstrins.d rd, rj, msbd, lsbd

GR[rd][msbd:lsbd] = GR[rj][msbd-lsbd:0]

位串插入（双字）（Bit String Insert Doubleword） · 2RI6 型 · LA64

将 `rd` 的 `[msbd:lsbd]` 位替换为 `rj` 的低 `(msbd-lsbd+1)` 位，其余位保持不变。`msbd`、`lsbd` 的范围为 0～63，且 `msbd ≥ lsbd`。

编码



编码值

```
0x00800000 | ((msbd & 0x3f) << 16) | ((lsbd & 0x3f) << 10) | (rj << 5) | rd
```

汇编示例

```
bstrins.d $a0, $a1, 39, 8      # $a0[39:8] = $a1[31:0]
```

语义伪代码

```
GR[rd][msbd:lsbd] = GR[rj][msbd-lsbd:0]
```

---

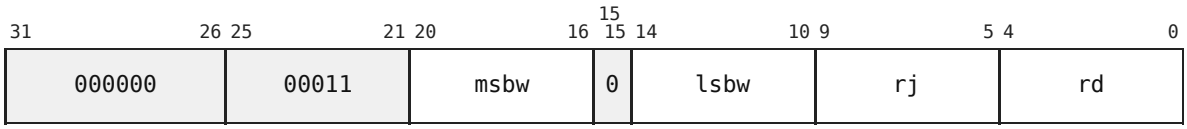
bstrins.w rd, rj, msbw, lsbw

GR[rd][msbw:lsbw] = GR[rj][msbw-lsbw:0]

位串插入（字）（Bit String Insert Word） · 2RI5 型 · LA32 / LA64

将 `rd` 低 32 位的 `[msbw:lsbw]` 位替换为 `rj` 的低 `(msbw-lsbw+1)` 位，得到的 32 位结果符号扩展至 GRLEN 位后写入 `rd`。`msbw`、`lsbw` 的范围为 0～31，且 `msbw ≥ lsbw`。

编码



编码值

```
0x00600000 | ((msbw & 0x1f) << 16) | ((lsbw & 0x1f) << 10) | (rj << 5) | rd
```

汇编示例

```
bstrins.w $a0, $a1, 15, 8      # $a0[15:8] = $a1[7:0]
```

语义伪代码

```
bstr32[31:msbw+1] = GR[rd][31:msbw+1]
bstr32[msbw:lsbw] = GR[rj][msbw-lsbw:0]
bstr32[lsbw-1:0] = GR[rd][lsbw-1:0]
GR[rd] = SignExtend(bstr32[31:0], GRLEN)
```

---

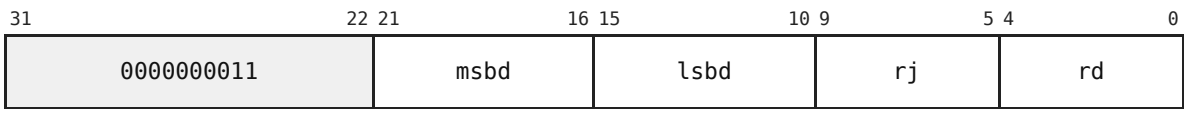
bstrpick.d rd, rj, msbd, lsbd

GR[rd] = ZeroExtend(GR[rj][msbd:lsbd], 64)

位串提取（双字）（Bit String Pick Doubleword） · 2RI6 型 · LA64

提取 `rj` 的 `[msbd:lsbd]` 位，零扩展至 64 位后写入 `rd`。`msbd`、`lsbd` 的范围为 0～63，且 `msbd ≥ lsbd`。

编码



编码值

```
0x00c00000 | ((msbd & 0x3f) << 16) | ((lsbd & 0x3f) << 10) | (rj << 5) | rd
```

汇编示例

```
bstrpick.d $a0, $a1, 39, 8      # $a0 = $a1[39:8]
```

语义伪代码

```
GR[rd] = ZeroExtend(GR[rj][msbd:lsbd], 64)
```

---

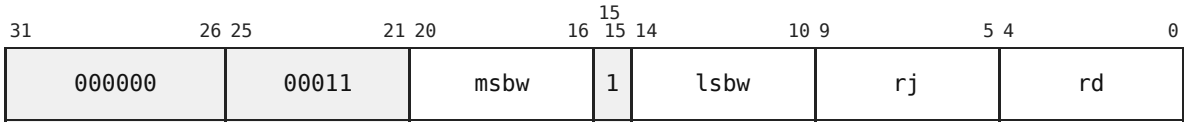
bstrpick.w `rd, rj, msbw, lsbw`

```
GR[rd] = SignExtend(ZeroExtend(GR[rj][msbw:lsbw], 32), GRLEN)
```

位串提取（字）（Bit String Pick Word） · 2RI5 型 · LA32 / LA64

提取 `rj` 低 32 位的 `[msbw:lsbw]` 位，零扩展至 32 位后再符号扩展至 GRLEN 位写入 `rd`。`msbw`、`lsbw` 的范围为 0～31，且 `msbw ≥ lsbw`。

编码



编码值

```
0x00608000 | ((msbw & 0x1f) << 16) | ((lsbw & 0x1f) << 10) | (rj << 5) | rd
```

汇编示例

```
bstrpick.w $a0, $a1, 15, 8      # $a0 = $a1[15:8]
```

语义伪代码

```
bstr32[31:0] = ZeroExtend(GR[rj][msbw:lsbw], 32)
GR[rd] = SignExtend(bstr32[31:0], GRLEN)
```

---

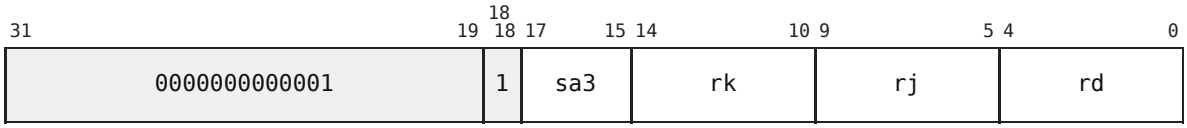
bytepick.d `rd, rj, rk, sa3`

```
GR[rd] = {GR[rk][8*(8-sa3):0], GR[rj][63:8*(8-sa3)]}
```

字节拼接（双字）（Byte Pick Doubleword） · 3R 型 · LA64

将 `rk` 与 `rj` 拼接为 128 位，从拼接结果左端（`rk` 的最高字节）起第 `sa3` 个字节开始，截取 8 个连续字节写入 `rd`。`sa3` 的范围为 0～7。

编码



编码值

```
0x000c0000 | ((sa3 & 0x7) << 15) | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
bytepick.d $a0, $a1, $a2, 5      # 从 {rk,rj} 第 5 字节起取 8 字节
```

语义伪代码

```
GR[rd] = {GR[rk][8*(8-sa3):0], GR[rj][63:8*(8-sa3)]}
```

---

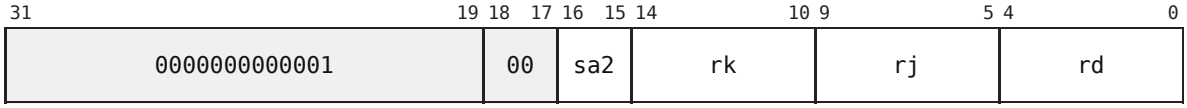
bytepick.w `rd, rj, rk, sa2`

```
GR[rd] = SignExtend({GR[rk][8*(4-sa2):0], GR[rj][31:8*(4-sa2)]}, GRLEN)
```

字节拼接（字）（Byte Pick Word） · 3R 型 · LA32 / LA64

将 `rk` 与 `rj` 的低 32 位拼接为 64 位，从拼接结果左端（`rk` 的最高字节）起第 `sa2` 个字节开始，截取 4 个连续字节，结果符号扩展至 GRLEN 位后写入 `rd`。`sa2` 的范围为 0～3。

编码



编码值

```
0x00080000 | ((sa2 & 0x3) << 15) | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
bytepick.w $a0, $a1, $a2, 2      # 从 {rk,rj} 第 2 字节起取 4 字节
```

语义伪代码

```
tmp = {GR[rk][8*(4-sa2):0], GR[rj][31:8*(4-sa2)]}
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

C · 计数、CRC、CSR 与缓存指令

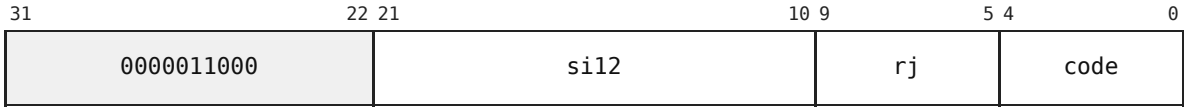
cacop `code, rj, si12`

```
CacheOp(code, GR[rj] + si12)
```

缓存维护操作（Cache Operation） · 2RI12 型 · LA32 / LA64

按 `code` 指定的操作类型，对地址 `rj + si12` 所在的缓存行执行维护操作。

编码



编码值

```
0x06000000 | ((si12 & 0xfff) << 10) | (rj << 5) | ((code & 0x1f) << 0)
```

汇编示例

```
cacop 0, $a0, 0      # 对 [$a0] 执行 code=0 的缓存操作
```

语义伪代码

```
CacheOp(code, GR[rj] + SignExtend(si12, GRLEN))
```

---

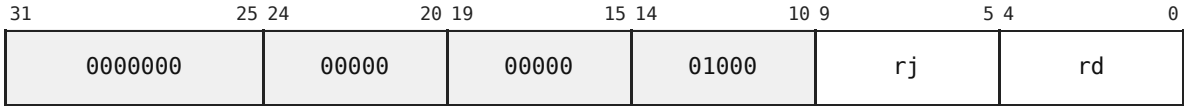
clo.d `rd, rj`

```
GR[rd] = CLO64(GR[rj])
```

双字前导一计数（Count Leading Ones Doubleword） · 2R 型 · LA64

统计 `rj` 低 64 位中从最高位起连续 1的个数，结果写入 `rd`。

编码



编码值

```
0x00002000 | (rj << 5) | rd
```

汇编示例

```
clo.d $a0, $a1      # $a0 = clo(64-bit $a1)
```

语义伪代码

```
GR[rd] = CLO64(GR[rj])
```

---

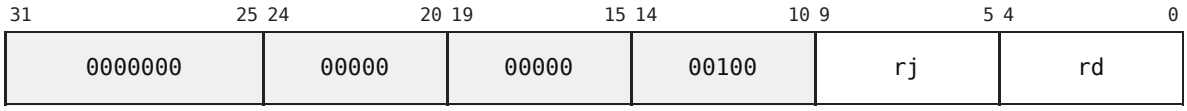
clo.w rd, rj

GR[rd] = CLO32(GR[rj])

字前导一计数（Count Leading Ones Word） · 2R 型 · LA32 / LA64

统计 `rj` 低 32 位中从最高位起连续 1的个数，结果写入 `rd`。

编码



编码值 `0x00001000 | (rj << 5) | rd`

汇编示例

```
clo.w $a0, $a1      # $a0 = clo(32-bit $a1)
```

语义伪代码

```
GR[rd] = CLO32(GR[rj])
```

---

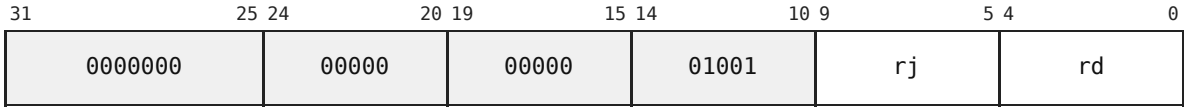
clz.d rd, rj

GR[rd] = CLZ64(GR[rj])

双字前导零计数（Count Leading Zeros Doubleword） · 2R 型 · LA64

统计 `rj` 低 64 位中从最高位起连续 0的个数，结果写入 `rd`。

编码



编码值 `0x00002400 | (rj << 5) | rd`

汇编示例

```
clz.d $a0, $a1      # $a0 = clz(64-bit $a1)
```

语义伪代码

```
GR[rd] = CLZ64(GR[rj])
```

---

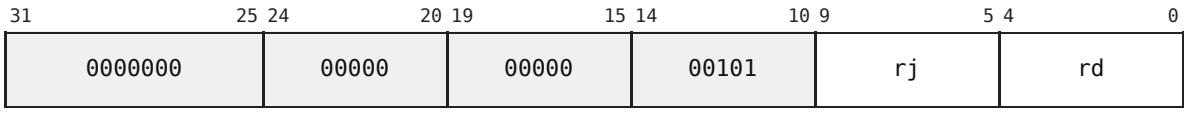
clz.w rd, rj

GR[rd] = CLZ32(GR[rj])

字前导零计数（Count Leading Zeros Word） · 2R 型 · LA32 / LA64

统计 `rj` 低 32 位中从最高位起连续 0的个数，结果写入 `rd`。

编码



编码值 `0x00001400 | (rj << 5) | rd`

汇编示例

```
clz.w $a0, $a1      # $a0 = clz(32-bit $a1)
```

语义伪代码

```
GR[rd] = CLZ32(GR[rj])
```

---

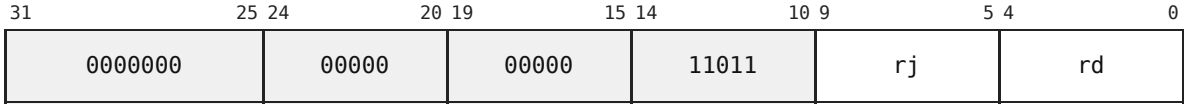
cpucfg `rd, rj`

GR[rd] = CPUCFG(GR[rj])

读取 CPU 配置字（CPU Configuration） · 2R 型 · LA32 / LA64

以 `rj` 为配置字编号，读取对应的 CPU 配置字并写入 `rd`。

编码



编码值 `0x00006c00 | (rj << 5) | rd`

汇编示例

```
cpucfg $a0, $a1      # $a0 = CPUCFG[$a1]
```

语义伪代码

```
GR[rd] = CPUCFG(GR[rj])
```

---

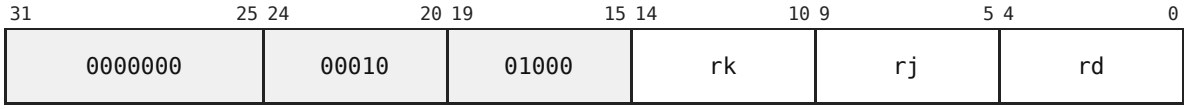
crc.w.b.w `rd, rj, rk`

GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][7:0], 8, 0xEDB88320), GRLEN)

CRC-32 校验（字节，IEEE 802.3）（CRC-32 IEEE 802.3 Byte） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 8 位数据计算 CRC-32（IEEE 802.3 多项式），结果符号扩展后写入 `rd`。

编码



编码值 `0x00240000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
crc.w.b.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][7:0], 8, 0xEDB88320)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

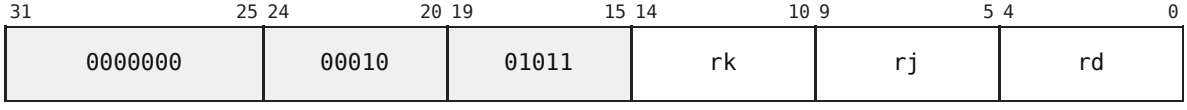
crc.w.d.w `rd, rj, rk`

GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][63:0], 64, 0xEDB88320), GRLEN)

CRC-32 校验（双字，IEEE 802.3）（CRC-32 IEEE 802.3 Doubleword） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 64 位数据计算 CRC-32（IEEE 802.3 多项式），结果符号扩展后写入 `rd`。

编码



编码值 `0x00258000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
crc.w.d.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][63:0], 64, 0xEDB88320)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

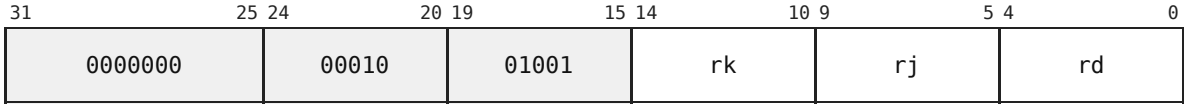
crc.w.h.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][15:0], 16, 0xEDB88320), GRLEN)
```

CRC-32 校验（半字，IEEE 802.3）（CRC-32 IEEE 802.3 Halfword） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 16 位数据计算 CRC-32（IEEE 802.3 多项式），结果符号扩展后写入 `rd`。

编码



编码值

```
0x00248000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
crc.w.h.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][15:0], 16, 0xEDB88320)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

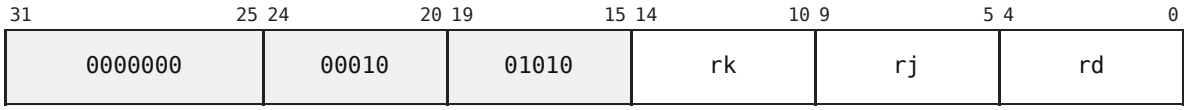
crc.w.w.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][31:0], 32, 0xEDB88320), GRLEN)
```

CRC-32 校验（字，IEEE 802.3）（CRC-32 IEEE 802.3 Word） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 32 位数据计算 CRC-32（IEEE 802.3 多项式），结果符号扩展后写入 `rd`。

编码



编码值

```
0x00250000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
crc.w.w.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][31:0], 32, 0xEDB88320)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

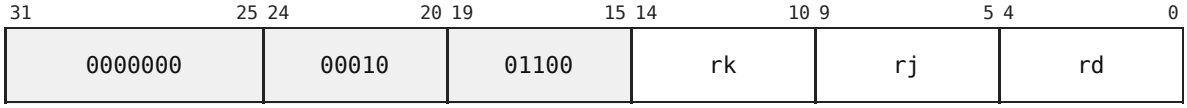
crcc.w.b.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][7:0], 8, 0x82F63B78), GRLEN)
```

CRC-32 校验（字节，Castagnoli）（CRC-32 Castagnoli Byte） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 8 位数据计算 CRC-32（Castagnoli 多项式），结果符号扩展后写入 `rd`。

编码



编码值 `0x00260000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
crcc.w.b.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][7:0], 8, 0x82F63B78)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

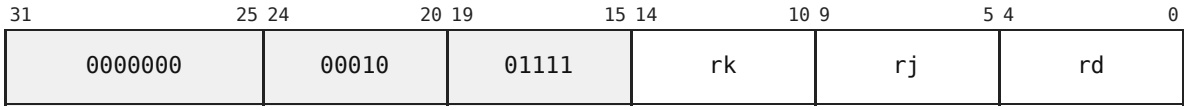
crcc.w.d.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][63:0], 64, 0x82F63B78), GRLEN)
```

CRC-32 校验（双字，Castagnoli）（CRC-32 Castagnoli Doubleword） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 64 位数据计算 CRC-32（Castagnoli 多项式），结果符号扩展后写入 `rd`。

编码



编码值 `0x00278000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
crcc.w.d.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][63:0], 64, 0x82F63B78)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

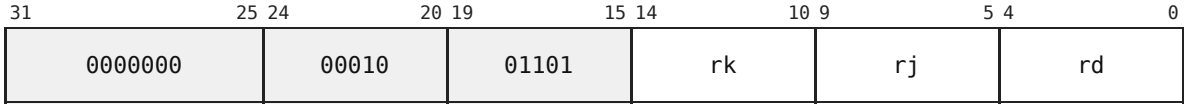
crcc.w.h.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][15:0], 16, 0x82F63B78), GRLEN)
```

CRC-32 校验（半字，Castagnoli）（CRC-32 Castagnoli Halfword） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 16 位数据计算 CRC-32（Castagnoli 多项式），结果符号扩展后写入 `rd`。

编码



编码值

```
0x00268000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
crcc.w.h.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][15:0], 16, 0x82F63B78)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

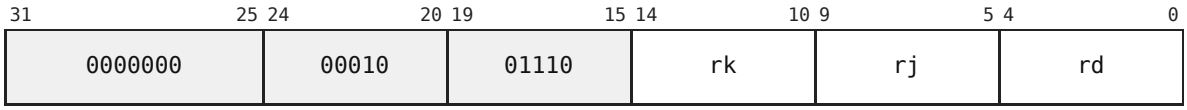
crcc.w.w.w `rd, rj, rk`

```
GR[rd] = SignExtend(CRC32(GR[rk], GR[rj][31:0], 32, 0x82F63B78), GRLEN)
```

CRC-32 校验（字，Castagnoli）（CRC-32 Castagnoli Word） · 3R 型 · LA64

以 `rk` 中的 32 位 CRC 累积值为初值，对 `rj` 的低 32 位数据计算 CRC-32（Castagnoli 多项式），结果符号扩展后写入 `rd`。

编码



编码值

```
0x00270000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
crcc.w.w.w $a0, $a1, $a2      # $a0 = crc32($a1, $a2)
```

语义伪代码

```
chksum = CRC32(GR[rk][31:0], GR[rj][31:0], 32, 0x82F63B78)
GR[rd] = SignExtend(chksum, GRLEN)
```

---

csrrd `rd, csr`

```
GR[rd] = CSR[csr]
```

读 CSR（Read CSR） · 2R 型 · LA32 / LA64

读取编号为 `csr` 的控制状态寄存器，结果写入 `rd`。

编码



编码值

```
0x04000000 | ((csr & 0x3fff) << 10) | rd
```

汇编示例

```
csrrd $a0, 0x0      # $a0 = CSR[0x0]
```

语义伪代码

```
GR[rd] = CSR[csr]
```

---

csrwr rd, csr

`tmp = CSR[csr]; CSR[csr] = GR[rd]; GR[rd] = tmp`

写 CSR（Write CSR） · 2R 型 · LA32 / LA64

将 `rd` 写入编号为 `csr` 的控制状态寄存器，并把旧值写回 `rd`。

编码



编码值 `0x04000020 | ((csr & 0x3fff) << 10) | rd`

汇编示例

```
csrwr $a0, 0x0      # CSR[0x0] = $a0; $a0 = old
```

语义伪代码

```
tmp = CSR[csr]
CSR[csr] = GR[rd]
GR[rd] = tmp
```

---

csrxchg rd, rj, csr

`old = CSR[csr]; CSR[csr] = (old & ~rj) | (rd & rj); rd = old`

交换 CSR 位域（Exchange CSR） · 3R 型 · LA32 / LA64

以 `rj` 为掩码，把 `rd` 中对应的位写入 CSR，其余位保持；CSR 旧值写回 `rd`。

编码



编码值 `0x04000000 | ((csr & 0x3fff) << 10) | (rj << 5) | rd`

汇编示例

```
csrxchg $a0, $a1, 0x0      # CSR[0x0] = ($a0 & $a1) | (old & ~$a1); $a0 = old
```

语义伪代码

```
tmp = CSR[csr]
CSR[csr] = (CSR[csr] & ~GR[rj]) | (GR[rd] & GR[rj])
GR[rd] = tmp
```

---

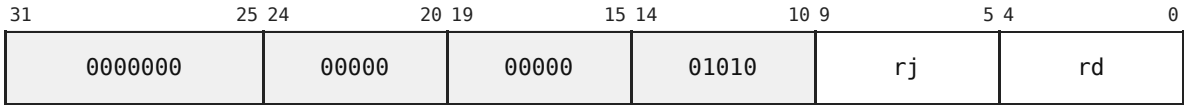
cto.d rd, rj

`GR[rd] = CTO64(GR[rj])`

双字尾随一计数（Count Trailing Ones Doubleword） · 2R 型 · LA64

统计 `rj` 低 64 位中从最低位起连续 1的个数，结果写入 `rd`。

编码



编码值 `0x00002800 | (rj << 5) | rd`

汇编示例

```
cto.d $a0, $a1      # $a0 = cto(64-bit $a1)
```

语义伪代码

```
GR[rd] = CTO64(GR[rj])
```

---

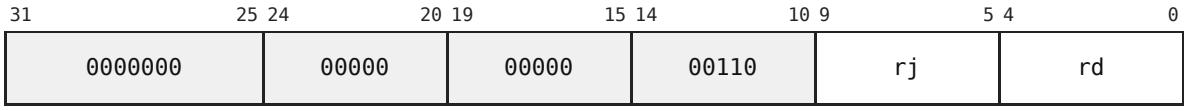
cto.w rd, rj

GR[rd] = CTO32(GR[rj])

字尾随一计数（Count Trailing Ones Word） · 2R 型 · LA32 / LA64

统计 `rj` 低 32 位中从最低位起连续 1的个数，结果写入 `rd`。

编码



编码值 `0x00001800 | (rj << 5) | rd`

汇编示例

```
cto.w $a0, $a1      # $a0 = cto(32-bit $a1)
```

语义伪代码

```
GR[rd] = CTO32(GR[rj])
```

---

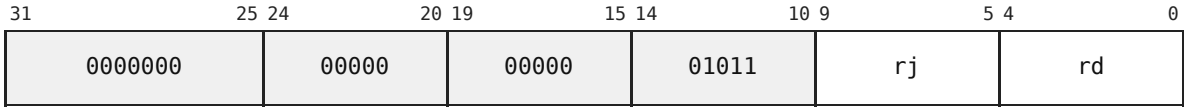
ctz.d rd, rj

GR[rd] = CTZ64(GR[rj])

双字尾随零计数（Count Trailing Zeros Doubleword） · 2R 型 · LA64

统计 `rj` 低 64 位中从最低位起连续 0的个数，结果写入 `rd`。

编码



编码值 `0x00002c00 | (rj << 5) | rd`

汇编示例

```
ctz.d $a0, $a1      # $a0 = ctz(64-bit $a1)
```

语义伪代码

```
GR[rd] = CTZ64(GR[rj])
```

---

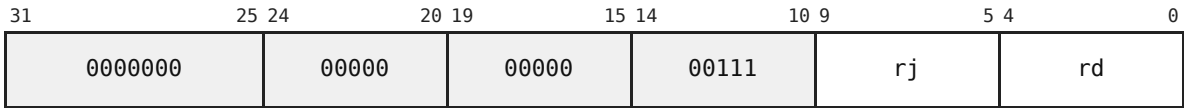
ctz.w rd, rj

GR[rd] = CTZ32(GR[rj])

字尾随零计数（Count Trailing Zeros Word） · 2R 型 · LA32 / LA64

统计 `rj` 低 32 位中从最低位起连续 0的个数，结果写入 `rd`。

编码



编码值 `0x00001c00 | (rj << 5) | rd`

汇编示例

```
ctz.w $a0, $a1      # $a0 = ctz(32-bit $a1)
```

语义伪代码

```
GR[rd] = CTZ32(GR[rj])
```

D · 除法、屏障与调试指令

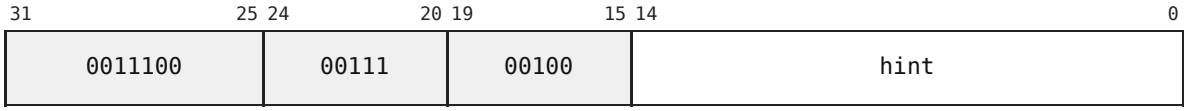
dbar hint

DataBarrier(hint)

数据屏障 （Data Barrier） · I15 型 · LA32 / LA64

完成前后加载/存储访存操作之间的屏障；`hint=0` 表示完全同步屏障。

编码



编码值 `0x38720000 | ((hint & 0x7fff) << 0)`

汇编示例

```
dbar 0      # 完全数据屏障
```

语义伪代码

```
DataBarrier(hint)
```

---

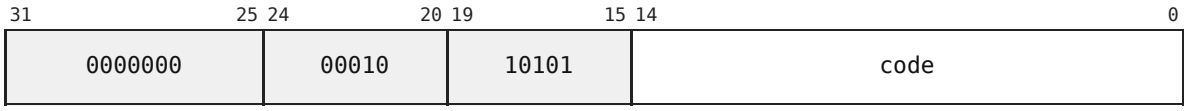
dbcl code

EnterDebugMode(code)

进入调试模式 （Debug Clear） · I15 型 · LA32 / LA64

立即进入调试模式，`code` 作为调试处理程序可读取的参数。

编码



编码值 `0x002a8000 | ((code & 0x7fff) << 0)`

汇编示例

```
dbcl 1      # 进入调试模式，code = 1
```

语义伪代码

```
EnterDebugMode(code)
```

---

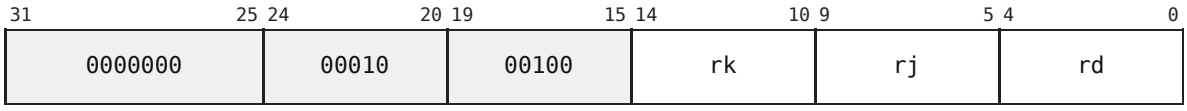
div.d rd, rj, rk

GR[rd] = signed(GR[rj]) / signed(GR[rk])

双字有符号除法 （Divide Doubleword） · 3R 型 · LA64

将 `rj` 与 `rk` 作为有符号数相除，商写入 `rd`。除零与溢出结果按架构规定处理。

编码



编码值 `0x00220000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
div.d $a0, $a1, $a2      # $a0 = $a1 / $a2
```

语义伪代码

```
GR[rd] = signed(GR[rj]) / signed(GR[rk])
```

---

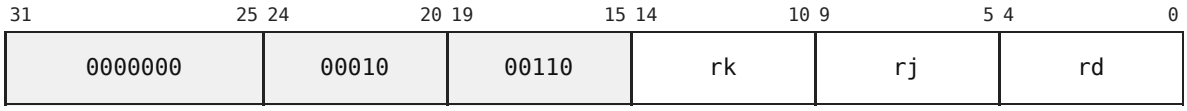
div.du rd, rj, rk

GR[rd] = unsigned(GR[rj]) / unsigned(GR[rk])

双字无符号除法 （Divide Doubleword Unsigned） · 3R 型 · LA64

将 `rj` 与 `rk` 作为无符号数相除，商写入 `rd`。除零与溢出结果按架构规定处理。

编码



编码值

```
0x00230000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
div.du $a0, $a1, $a2      # $a0 = $a1 / $a2
```

语义伪代码

```
GR[rd] = unsigned(GR[rj]) / unsigned(GR[rk])
```

---

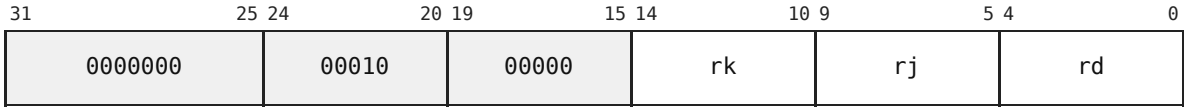
div.w rd, rj, rk

GR[rd] = signed(GR[rj]) / signed(GR[rk])

字有符号除法 （Divide Word） · 3R 型 · LA32 / LA64

将 `rj` 与 `rk` 作为有符号数相除，商写入 `rd`。除零与溢出结果按架构规定处理。

编码



编码值

```
0x00200000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
div.w $a0, $a1, $a2      # $a0 = $a1 / $a2
```

语义伪代码

```
GR[rd] = signed(GR[rj]) / signed(GR[rk])
```

---

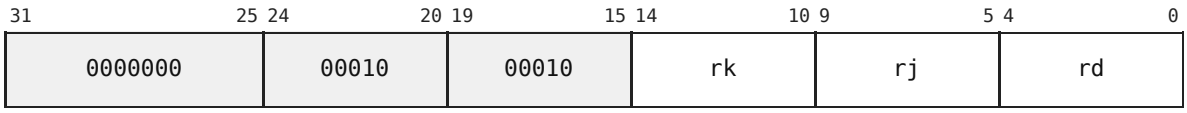
div.wu rd, rj, rk

GR[rd] = unsigned(GR[rj]) / unsigned(GR[rk])

字无符号除法 （Divide Word Unsigned） · 3R 型 · LA32 / LA64

将 `rj` 与 `rk` 作为无符号数相除，商写入 `rd`。除零与溢出结果按架构规定处理。

编码



编码值

```
0x00210000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
div.wu $a0, $a1, $a2      # $a0 = $a1 / $a2
```

语义伪代码

```
GR[rd] = unsigned(GR[rj]) / unsigned(GR[rk])
```

E · 扩展与异常返回指令

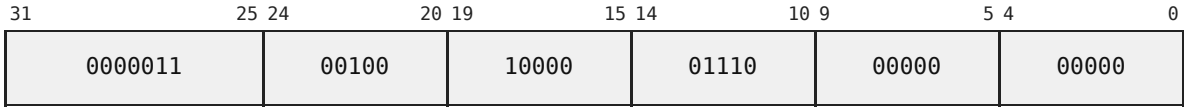
ertn

PC = CSR.ERA; restore CSR.PRMD/CSR.CRMD

异常返回 （Exception Return） · 专用格式 · LA32 / LA64

从异常返回：恢复特权级与状态，并跳转到 `CSR.ERA` 保存的返回地址。

编码



编码值 `0x06483800`

汇编示例

```
ertn       # 从异常返回
```

语义伪代码

```
PC = CSR.ERA
RestorePrivilegeAndState()
```

---

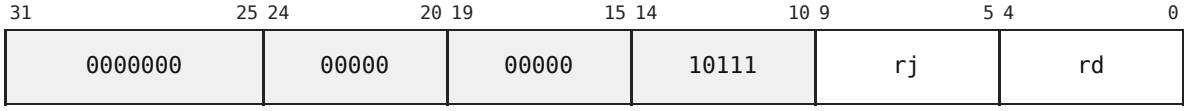
ext.w.b rd, rj

GR[rd] = SignExtend(GR[rj][7:0], GRLEN)

字节符号扩展 （Extend Byte） · 2R 型 · LA32 / LA64

将 `rj` 的最低字节符号扩展至 GRLEN 位后写入 `rd`。

编码



编码值 `0x00005c00 | (rj << 5) | rd`

汇编示例

```
ext.w.b $a0, $a1      # $a0 = (signed)$a1[7:0]
```

语义伪代码

```
GR[rd] = SignExtend(GR[rj][7:0], GRLEN)
```

---

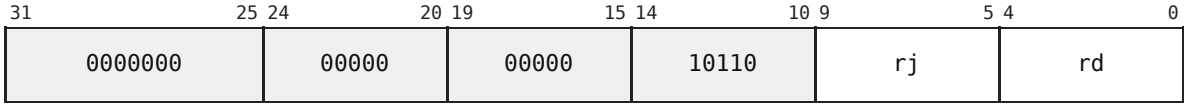
ext.w.h rd, rj

GR[rd] = SignExtend(GR[rj][15:0], GRLEN)

半字符号扩展 （Extend Halfword） · 2R 型 · LA32 / LA64

将 `rj` 的最低半字符号扩展至 GRLEN 位后写入 `rd`。

编码



编码值 `0x00005800 | (rj << 5) | rd`

汇编示例

```
ext.w.h $a0, $a1      # $a0 = (signed)$a1[15:0]
```

语义伪代码

```
GR[rd] = SignExtend(GR[rj][15:0], GRLEN)
```

F · 浮点指令

本章收录 95 条浮点指令。FR[x] 表示浮点寄存器，FCSR 为浮点控制与状态寄存器；单精度指令作用于低 32 位，双精度指令作用于 64 位。浮点运算按 IEEE 754-2008 与 FCSR 的舍入/异常设置执行。

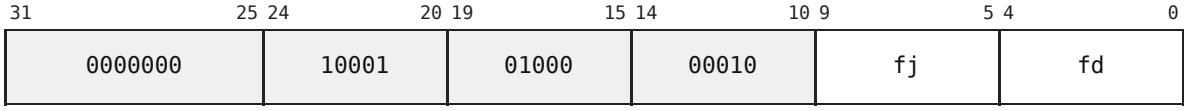
fabs.d fd, fj

FR[fd] = abs(FR[fj])

浮点绝对值（双精度）（Floating-point Absolute Double） · 2R 型 · LA64

取 `fj` 的绝对值（清符号位）写入 `fd`。

编码



编码值

```
0x01140800 | (fj << 5) | fd
```

汇编示例

```
fabs.d $f0, $f1      # $f0 = abs($f1)
```

语义伪代码

```
FR[fd] = abs(FR[fj])
```

---

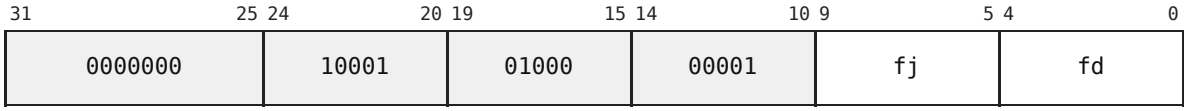
fabs.s fd, fj

FR[fd] = abs(FR[fj])

浮点绝对值（单精度）（Floating-point Absolute Single） · 2R 型 · LA32 / LA64

取 `fj` 的绝对值（清符号位）写入 `fd`。

编码



编码值

```
0x01140400 | (fj << 5) | fd
```

汇编示例

```
fabs.s $f0, $f1      # $f0 = abs($f1)
```

语义伪代码

```
FR[fd] = abs(FR[fj])
```

---

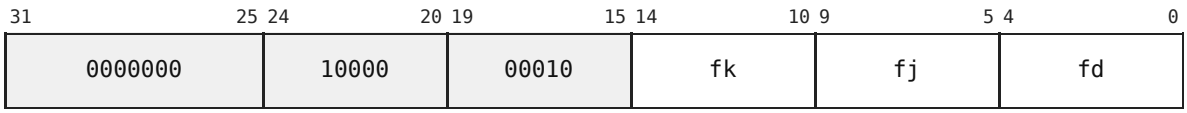
fadd.d fd, fj, fk

FR[fd] = FR[fj] + FR[fk]

浮点加法（双精度）（Floating-point Add Double） · 3R 型 · LA64

对 `fj`、`fk` 执行双精度浮点加法，结果写入 `fd`。

编码



编码值

```
0x01010000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fadd.d $f0, $f1, $f2      # $f0 = fadd($f1, $f2)
```

语义伪代码

```
FR[fd] = FR[fj] + FR[fk]
```

---

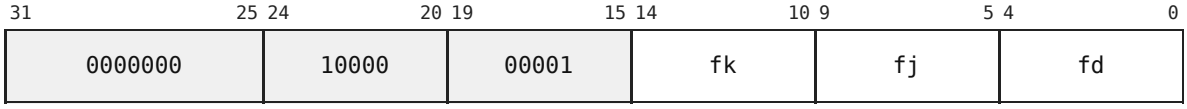
fadd.s `fd, fj, fk`

`FR[fd][31:0] = FR[fj][31:0] + FR[fk][31:0]`

浮点加法（单精度）（Floating-point Add Single） · 3R 型 · LA32 / LA64

对 `fj`、`fk` 执行单精度浮点加法，结果写入 `fd`。

编码



编码值 `0x01008000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fadd.s $f0, $f1, $f2      # $f0 = fadd($f1, $f2)
```

语义伪代码

```
FR[fd][31:0] = FR[fj][31:0] + FR[fk][31:0]
```

---

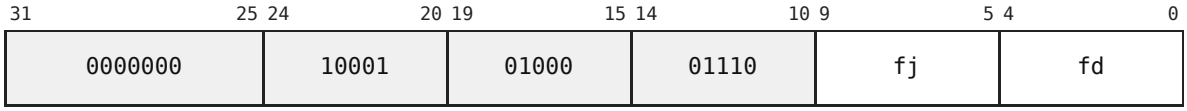
fclass.d `fd, fj`

`FR[fd] = class(FR[fj])`

浮点分类（双精度）（Floating-point Classify Double） · 2R 型 · LA64

按 IEEE 754 class(x) 对 `fj` 分类，10 位结果位图写入 `fd`。

编码



编码值 `0x01143800 | (fj << 5) | fd`

汇编示例

```
fclass.d $f0, $f1      # 分类结果写入 $f0
```

语义伪代码

```
FR[fd] = class(FR[fj])
```

---

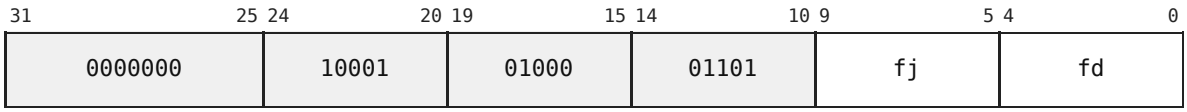
fclass.s `fd, fj`

`FR[fd] = class(FR[fj])`

浮点分类（单精度）（Floating-point Classify Single） · 2R 型 · LA32 / LA64

按 IEEE 754 class(x) 对 `fj` 分类，10 位结果位图写入 `fd`。

编码



编码值 `0x01143400 | (fj << 5) | fd`

汇编示例

```
fclass.s $f0, $f1      # 分类结果写入 $f0
```

语义伪代码

```
FR[fd] = class(FR[fj])
```

---

fcmp.cond.d `cd, fj, fk`

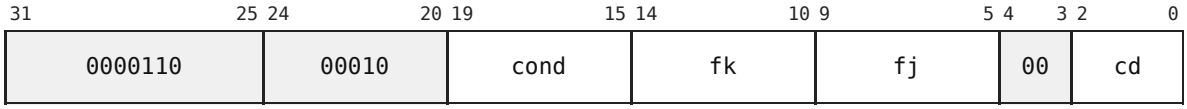
`CFR[cd] = Compare(FR[fj], FR[fk], cond)`

浮点比较（双精度）（Floating-point Compare Double） · 3R 型 · LA64

按 `cond` 指定的条件比较 `fj`、`fk`，结果写入条件标志寄存器 `cd`。

> `cond` 可取 22 种比较条件：CAF、CUN、CEQ、CUEQ、CLT、CULT、CLE、CULE、CNE、COR、CUNE、SAF、SUN、SEQ、SUEQ、SLT、SULT、SLE、SULE、SNE、SOR、SUNE。结果写入条件标志寄存器 `cd`（0～7，汇编名 `$fcc0`～`$fcc7`）。

编码



编码值

```
0x0c200000 | ((cond & 0x1f) << 15) | (fk << 10) | (fj << 5) | cd
```

汇编示例

```
fcmp.ceq.d $fcc0, $f1, $f2      # $fcc0 = compare($f1, $f2)
```

语义伪代码

```
CFR[cd] = Compare(FR[fj], FR[fk], cond)
```

---

fcmp.cond.s `cd, fj, fk`

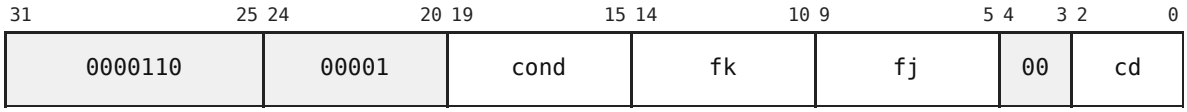
`CFR[cd] = Compare(FR[fj], FR[fk], cond)`

浮点比较（单精度）（Floating-point Compare Single） · 3R 型 · LA32 / LA64

按 `cond` 指定的条件比较 `fj`、`fk`，结果写入条件标志寄存器 `cd`。

> `cond` 可取 22 种比较条件：CAF、CUN、CEQ、CUEQ、CLT、CULT、CLE、CULE、CNE、COR、CUNE、SAF、SUN、SEQ、SUEQ、SLT、SULT、SLE、SULE、SNE、SOR、SUNE。结果写入条件标志寄存器 `cd`（0～7，汇编名 `$fcc0`～`$fcc7`）。

编码



编码值

```
0x0c100000 | ((cond & 0x1f) << 15) | (fk << 10) | (fj << 5) | cd
```

汇编示例

```
fcmp.ceq.s $fcc0, $f1, $f2      # $fcc0 = compare($f1, $f2)
```

语义伪代码

```
CFR[cd] = Compare(FR[fj], FR[fk], cond)
```

---

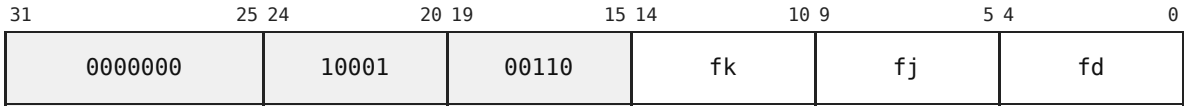
fcopysign.d `fd, fj, fk`

`FR[fd] = copySign(FR[fj], FR[fk])`

浮点复制符号（双精度）（Floating-point Copy Sign Double） · 3R 型 · LA64

取 `fj` 的数值部分与 `fk` 的符号位组合，结果写入 `fd`。

编码



编码值 `0x01130000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fcopysign.d $f0, $f1, $f2      # $f0 = copySign($f1, $f2)
```

语义伪代码

```
FR[fd] = copySign(FR[fj], FR[fk])
```

---

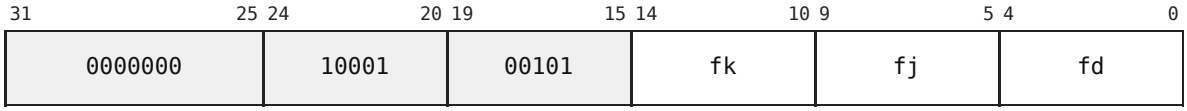
fcopysign.s `fd, fj, fk`

`FR[fd] = copySign(FR[fj], FR[fk])`

浮点复制符号（单精度）（Floating-point Copy Sign Single） · 3R 型 · LA32 / LA64

取 `fj` 的数值部分与 `fk` 的符号位组合，结果写入 `fd`。

编码



编码值 `0x01128000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fcopysign.s $f0, $f1, $f2      # $f0 = copySign($f1, $f2)
```

语义伪代码

```
FR[fd] = copySign(FR[fj], FR[fk])
```

---

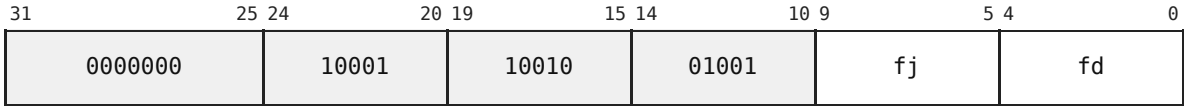
fcvt.d.s `fd, fj`

`FR[fd] = ConvertSingleToDouble(FR[fj][31:0])`

单精度转双精度（Convert Single to Double） · 2R 型 · LA64

将单精度浮点数转换为双精度，结果写入 `fd`。

编码



编码值 `0x01192400 | (fj << 5) | fd`

汇编示例

```
fcvt.d.s $f0, $f1      # 单精度转双精度
```

语义伪代码

```
FR[fd] = ConvertSingleToDouble(FR[fj][31:0])
```

---

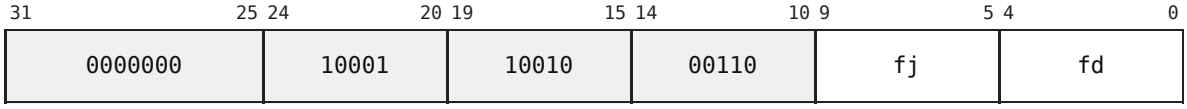
fcvt.s.d `fd, fj`

`FR[fd][31:0] = ConvertDoubleToSingle(FR[fj])`

双精度转单精度 （Convert Double to Single） · 2R 型 · LA64

将双精度浮点数转换为单精度，结果写入 `fd` 低 32 位。

编码



编码值 `0x01191800 | (fj << 5) | fd`

汇编示例

```
fcvt.s.d $f0, $f1       # 双精度转单精度
```

语义伪代码

```
FR[fd][31:0] = ConvertDoubleToSingle(FR[fj])
```

---

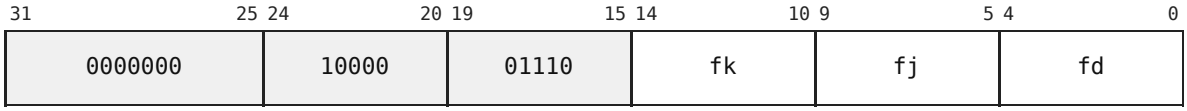
fdiv.d `fd, fj, fk`

`FR[fd] = FR[fj] / FR[fk]`

浮点除法（双精度） （Floating-point Divide Double） · 3R 型 · LA64

对 `fj`、`fk` 执行双精度浮点除法，结果写入 `fd`。

编码



编码值 `0x01070000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fdiv.d $f0, $f1, $f2      # $f0 = fdiv($f1, $f2)
```

语义伪代码

```
FR[fd] = FR[fj] / FR[fk]
```

---

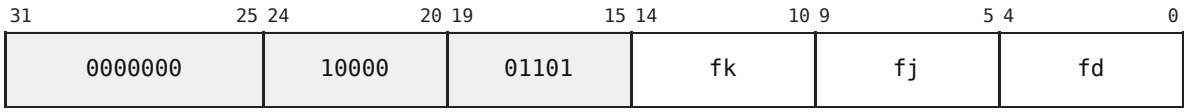
fdiv.s `fd, fj, fk`

`FR[fd][31:0] = FR[fj][31:0] / FR[fk][31:0]`

浮点除法（单精度） （Floating-point Divide Single） · 3R 型 · LA32 / LA64

对 `fj`、`fk` 执行单精度浮点除法，结果写入 `fd`。

编码



编码值 `0x01068000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fdiv.s $f0, $f1, $f2      # $f0 = fdiv($f1, $f2)
```

语义伪代码

```
FR[fd][31:0] = FR[fj][31:0] / FR[fk][31:0]
```

---

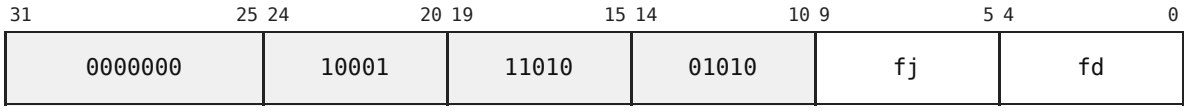
ffint.d.l `fd, fj`

`FR[fd] = ConvertInt64ToFloat(GR[rj])`

整数转双精度浮点（双字源）（Integer to Double from Doubleword） · 2R 型 · LA64

将 `rj` 的 64 位整数转换为双精度浮点数，结果写入 `fd`。

编码



编码值 `0x011d2800 | (fj << 5) | fd`

汇编示例

```
ffint.d.l $f0, $a1      # $f0 = (双精度)$a1
```

语义伪代码

```
FR[fd] = ConvertInt64ToFloat(GR[rj])
```

---

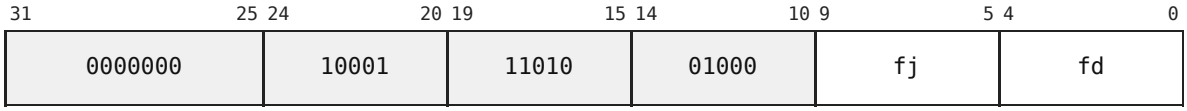
ffint.d.w `fd, fj`

`FR[fd] = ConvertInt32ToFloat(GR[rj])`

整数转双精度浮点（字源）（Integer to Double from Word） · 2R 型 · LA64

将 `rj` 的 32 位整数转换为双精度浮点数，结果写入 `fd`。

编码



编码值 `0x011d2000 | (fj << 5) | fd`

汇编示例

```
ffint.d.w $f0, $a1      # $f0 = (双精度)$a1
```

语义伪代码

```
FR[fd] = ConvertInt32ToFloat(GR[rj])
```

---

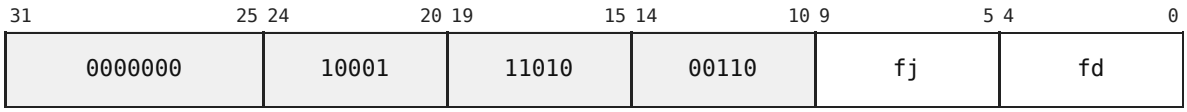
ffint.s.l `fd, fj`

`FR[fd] = ConvertInt64ToFloat(GR[rj])`

整数转单精度浮点（双字源）（Integer to Single from Doubleword） · 2R 型 · LA64

将 `rj` 的 64 位整数转换为单精度浮点数，结果写入 `fd`。

编码



编码值 `0x011d1800 | (fj << 5) | fd`

汇编示例

```
ffint.s.l $f0, $a1      # $f0 = (单精度)$a1
```

语义伪代码

```
FR[fd] = ConvertInt64ToFloat(GR[rj])
```

---

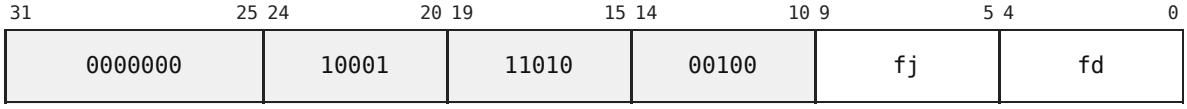
ffint.s.w `fd, fj`

`FR[fd] = ConvertInt32ToFloat(GR[rj])`

整数转单精度浮点（字源） （Integer to Single from Word） · 2R 型 · LA32 / LA64

将 `rj` 的 32 位整数转换为单精度浮点数，结果写入 `fd`。

编码



编码值

```
0x011d1000 | (fj << 5) | fd
```

汇编示例

```
ffint.s.w $f0, $a1      # $f0 = (单精度)$a1
```

语义伪代码

```
FR[fd] = ConvertInt32ToFloat(GR[rj])
```

---

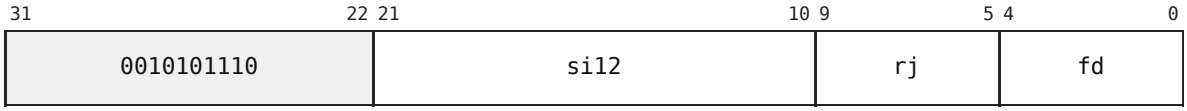
fld.d `fd, rj, si12`

`FR[fd] = Memory[GR[rj] + si12]`

浮点加载（双精度） （Floating-point Load Double） · 2RI12 型 · LA64

地址 `rj + si12` 处加载双精度浮点数。

编码



编码值

```
0x2b800000 | ((si12 & 0xfff) << 10) | (rj << 5) | fd
```

汇编示例

```
fld.d $f0, $a0, 0      # $f0 = [$a0]
```

语义伪代码

```
FR[fd] = Memory[GR[rj] + SignExtend(si12, GRLEN)]
```

---

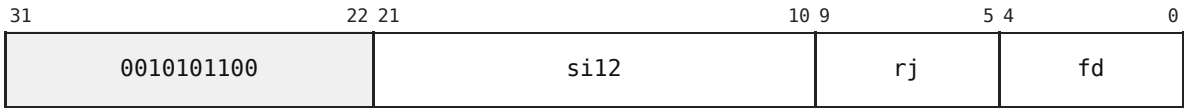
fld.s `fd, rj, si12`

`FR[fd] = Memory[GR[rj] + si12]`

浮点加载（单精度） （Floating-point Load Single） · 2RI12 型 · LA32 / LA64

地址 `rj + si12` 处加载单精度浮点数。

编码



编码值

```
0x2b000000 | ((si12 & 0xfff) << 10) | (rj << 5) | fd
```

汇编示例

```
fld.s $f0, $a0, 0      # $f0 = [$a0]
```

语义伪代码

```
FR[fd] = Memory[GR[rj] + SignExtend(si12, GRLEN)]
```

---

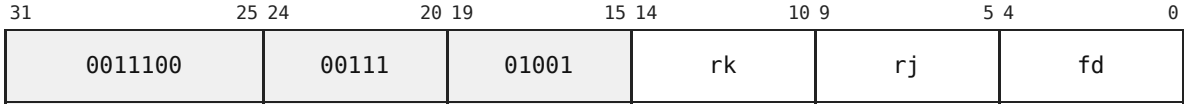
fldgt.d `fd, rj, rk`

`if (GR[rj] > GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE`

浮点加载（大于检查，双精度）（Floating-point Load Greater Than Double） · 3R 型 · LA64

检查 `rj > rk`，满足时加载浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38748000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldgt.d $f0, $a0, $a1      # if ($a0 > $a1) load
```

语义伪代码

```
if (GR[rj] > GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE
```

---

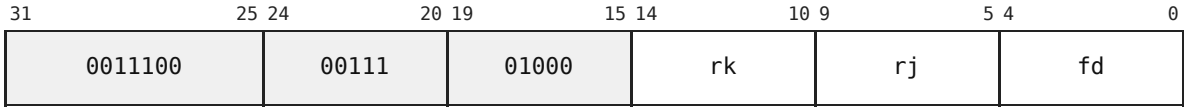
fldgt.s `fd, rj, rk`

`if (GR[rj] > GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE`

浮点加载（大于检查，单精度）（Floating-point Load Greater Than Single） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时加载浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38740000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldgt.s $f0, $a0, $a1      # if ($a0 > $a1) load
```

语义伪代码

```
if (GR[rj] > GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE
```

---

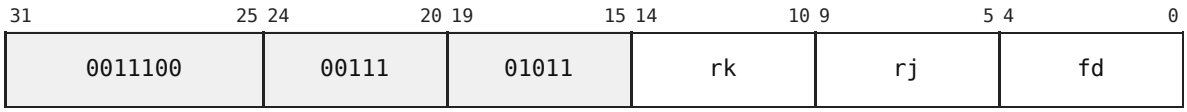
fldle.d `fd, rj, rk`

`if (GR[rj] <= GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE`

浮点加载（小于等于检查，双精度）（Floating-point Load Less Equal Double） · 3R 型 · LA64

检查 `rj <= rk`，满足时加载浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38758000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldle.d $f0, $a0, $a1      # if ($a0 <= $a1) load
```

语义伪代码

```
if (GR[rj] <= GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE
```

---

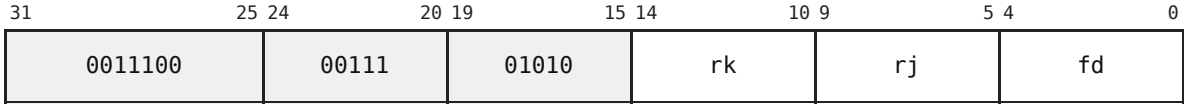
fldle.s `fd, rj, rk`

`if (GR[rj] <= GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE`

浮点加载（小于等于检查，单精度）（Floating-point Load Less Equal Single） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时加载浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38750000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldle.s $f0, $a0, $a1      # if ($a0 <= $a1) load
```

语义伪代码

```
if (GR[rj] <= GR[rk]) FR[fd] = Memory[GR[rj]] else raise BCE
```

---

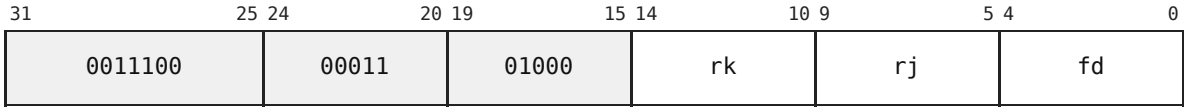
fldx.d `fd, rj, rk`

`FR[fd] = Memory[GR[rj] + GR[rk]]`

浮点索引加载（双精度）（Floating-point Indexed Load Double） · 3R 型 · LA64

地址 `rj + rk` 处加载双精度浮点数。

编码



编码值 `0x38340000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldx.d $f0, $a0, $a1      # $f0 = [$a0+$a1]
```

语义伪代码

```
FR[fd] = Memory[GR[rj] + GR[rk]]
```

---

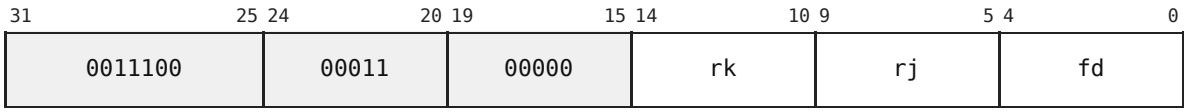
fldx.s `fd, rj, rk`

`FR[fd] = Memory[GR[rj] + GR[rk]]`

浮点索引加载（单精度）（Floating-point Indexed Load Single） · 3R 型 · LA32 / LA64

地址 `rj + rk` 处加载单精度浮点数。

编码



编码值 `0x38300000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fldx.s $f0, $a0, $a1      # $f0 = [$a0+$a1]
```

语义伪代码

```
FR[fd] = Memory[GR[rj] + GR[rk]]
```

---

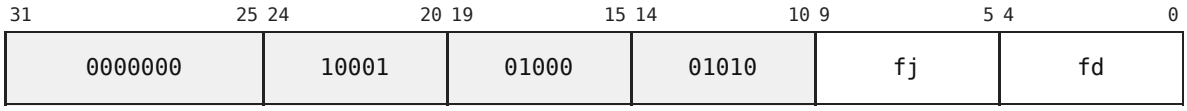
flogb.d `fd, fj`

`FR[fd] = logB(FR[fj])`

浮点以 2 为底对数（双精度）（Floating-point Log Base 2 Double） · 2R 型 · LA64

计算 `fj` 的以 2 为底对数写入 `fd`。

编码



编码值 `0x01142800 | (fj << 5) | fd`

汇编示例

```
flogb.d $f0, $f1      # $f0 = logB($f1)
```

语义伪代码

```
FR[fd] = logB(FR[fj])
```

---

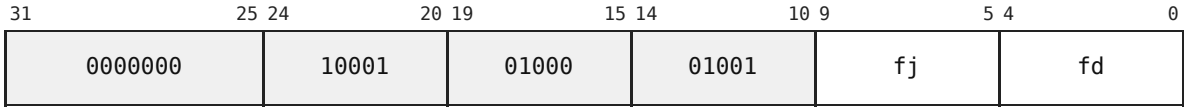
flogb.s `fd, fj`

`FR[fd] = logB(FR[fj])`

浮点以 2 为底对数（单精度）（Floating-point Log Base 2 Single） · 2R 型 · LA32 / LA64

计算 `fj` 的以 2 为底对数写入 `fd`。

编码



编码值 `0x01142400 | (fj << 5) | fd`

汇编示例

```
flogb.s $f0, $f1      # $f0 = logB($f1)
```

语义伪代码

```
FR[fd] = logB(FR[fj])
```

---

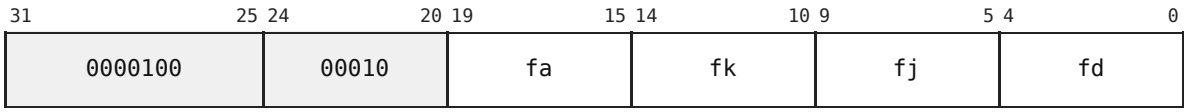
fmadd.d `fd, fj, fk, fa`

`FR[fd] = fma(FR[fj], FR[fk], + FR[fa])`

浮点融合乘加（双精度）（Floating-point Fused Multiply-Add Double） · 4R 型 · LA64

执行 `FR[fd] = (FR[fj] * FR[fk] + FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值 `0x08200000 | (fa << 15) | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fmadd.d $f0, $f1, $f2, $f3      # $f0 = ($f1 * $f2 + $f3)
```

语义伪代码

```
FR[fd] = fusedMultiplyAdd(FR[fj], FR[fk], + FR[fa])
```

---

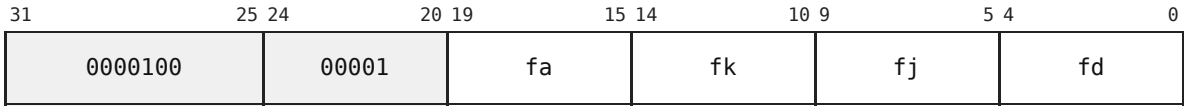
fmadd.s `fd, fj, fk, fa`

`FR[fd] = fma(FR[fj], FR[fk], + FR[fa])`

浮点融合乘加（单精度） （Floating-point Fused Multiply-Add Single） · 4R 型 · LA32 / LA64

执行 `FR[fd] = (FR[fj] * FR[fk] + FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值

```
0x08100000 | (fa << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmadd.s $f0, $f1, $f2, $f3      # $f0 = ($f1 * $f2 + $f3)
```

语义伪代码

```
FR[fd] = fusedMultiplyAdd(FR[fj], FR[fk], + FR[fa])
```

---

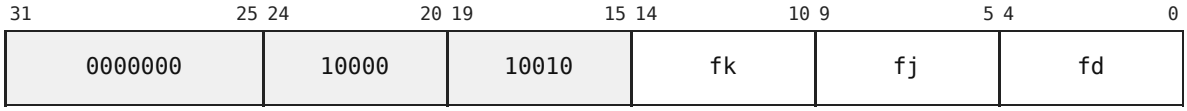
fmax.d `fd, fj, fk`

`FR[fd] = maxNum(FR[fj], FR[fk])`

浮点最大值（双精度） （Floating-point Maximum Double） · 3R 型 · LA64

按 IEEE 754 maxNum 语义返回 `fj`、`fk` 中较大者。

编码



编码值

```
0x01090000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmax.d $f0, $f1, $f2      # $f0 = maxNum($f1, $f2)
```

语义伪代码

```
FR[fd] = maxNum(FR[fj], FR[fk])
```

---

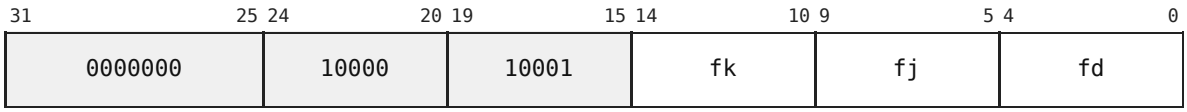
fmax.s `fd, fj, fk`

`FR[fd] = maxNum(FR[fj], FR[fk])`

浮点最大值（单精度） （Floating-point Maximum Single） · 3R 型 · LA32 / LA64

按 IEEE 754 maxNum 语义返回 `fj`、`fk` 中较大者。

编码



编码值

```
0x01088000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmax.s $f0, $f1, $f2      # $f0 = maxNum($f1, $f2)
```

语义伪代码

```
FR[fd] = maxNum(FR[fj], FR[fk])
```

---

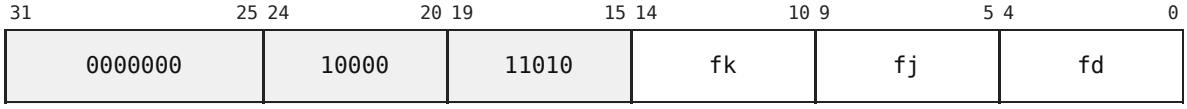
fmaxa.d `fd, fj, fk`

`FR[fd] = maxNumMag(FR[fj], FR[fk])`

浮点绝对值最大值（双精度） （Floating-point Maximum Magnitude Double） · 3R 型 · LA64

按绝对值比较 `fj`、`fk`，返回绝对值较大者。

编码



编码值 `0x010d0000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fmaxa.d $f0, $f1, $f2      # $f0 = maxNumMag($f1, $f2)
```

语义伪代码

```
FR[fd] = maxNumMag(FR[fj], FR[fk])
```

---

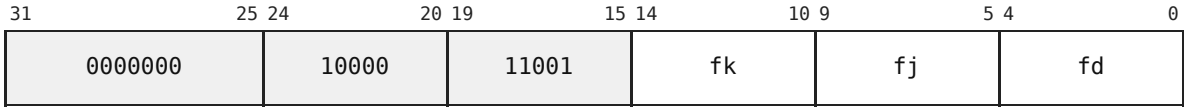
fmaxa.s `fd, fj, fk`

`FR[fd] = maxNumMag(FR[fj], FR[fk])`

浮点绝对值最大值（单精度） （Floating-point Maximum Magnitude Single） · 3R 型 · LA32 / LA64

按绝对值比较 `fj`、`fk`，返回绝对值较大者。

编码



编码值 `0x010c8000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fmaxa.s $f0, $f1, $f2      # $f0 = maxNumMag($f1, $f2)
```

语义伪代码

```
FR[fd] = maxNumMag(FR[fj], FR[fk])
```

---

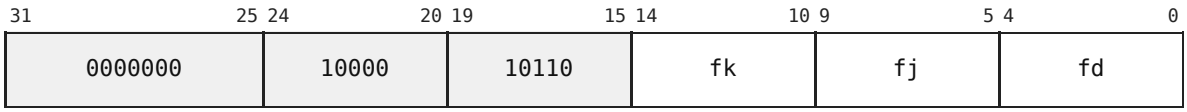
fmin.d `fd, fj, fk`

`FR[fd] = minNum(FR[fj], FR[fk])`

浮点最小值（双精度） （Floating-point Minimum Double） · 3R 型 · LA64

按 IEEE 754 minNum 语义返回 `fj`、`fk` 中较小者。

编码



编码值 `0x010b0000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fmin.d $f0, $f1, $f2      # $f0 = minNum($f1, $f2)
```

语义伪代码

```
FR[fd] = minNum(FR[fj], FR[fk])
```

---

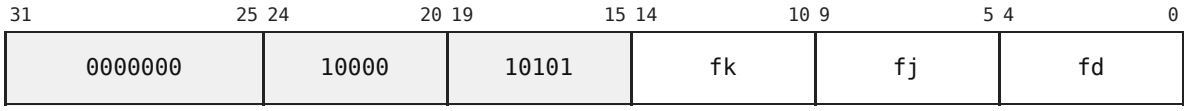
fmin.s fd, fj, fk

FR[fd] = minNum(FR[fj], FR[fk])

浮点最小值（单精度） （Floating-point Minimum Single） · 3R 型 · LA32 / LA64

按 IEEE 754 minNum 语义返回 `fj`、`fk` 中较小者。

编码



编码值

```
0x010a8000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmin.s $f0, $f1, $f2      # $f0 = minNum($f1, $f2)
```

语义伪代码

```
FR[fd] = minNum(FR[fj], FR[fk])
```

---

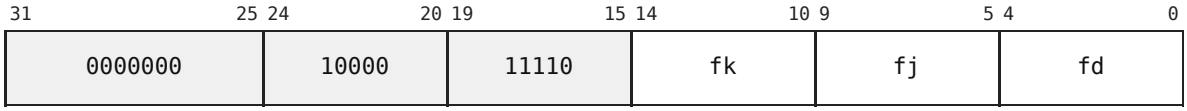
fmina.d fd, fj, fk

FR[fd] = minNumMag(FR[fj], FR[fk])

浮点绝对值最小值（双精度） （Floating-point Minimum Magnitude Double） · 3R 型 · LA64

按绝对值比较 `fj`、`fk`，返回绝对值较小者。

编码



编码值

```
0x010f0000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmina.d $f0, $f1, $f2      # $f0 = minNumMag($f1, $f2)
```

语义伪代码

```
FR[fd] = minNumMag(FR[fj], FR[fk])
```

---

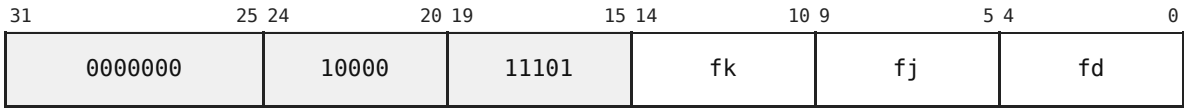
fmina.s fd, fj, fk

FR[fd] = minNumMag(FR[fj], FR[fk])

浮点绝对值最小值（单精度） （Floating-point Minimum Magnitude Single） · 3R 型 · LA32 / LA64

按绝对值比较 `fj`、`fk`，返回绝对值较小者。

编码



编码值

```
0x010e8000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmina.s $f0, $f1, $f2      # $f0 = minNumMag($f1, $f2)
```

语义伪代码

```
FR[fd] = minNumMag(FR[fj], FR[fk])
```

---

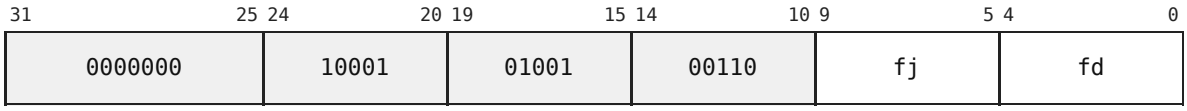
fmov.d `fd, fj`

`FR[fd] = FR[fj]`

浮点搬移（双精度）（Floating-point Move Double） · 2R 型 · LA64

将 `fj` 原样写入 `fd`，不触发浮点异常。

编码



编码值 `0x01149800 | (fj << 5) | fd`

汇编示例

```
fmov.d $f0, $f1      # $f0 = $f1
```

语义伪代码

```
FR[fd] = FR[fj]
```

---

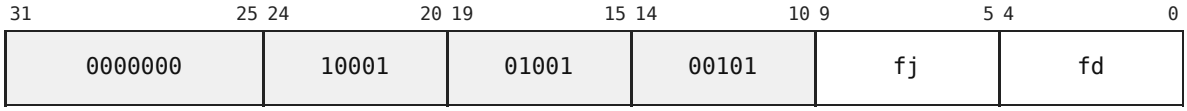
fmov.s `fd, fj`

`FR[fd] = FR[fj]`

浮点搬移（单精度）（Floating-point Move Single） · 2R 型 · LA32 / LA64

将 `fj` 原样写入 `fd`，不触发浮点异常。

编码



编码值 `0x01149400 | (fj << 5) | fd`

汇编示例

```
fmov.s $f0, $f1      # $f0 = $f1
```

语义伪代码

```
FR[fd] = FR[fj]
```

---

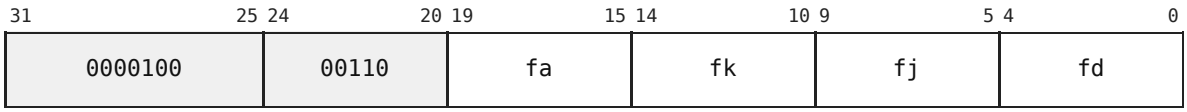
fmsub.d `fd, fj, fk, fa`

`FR[fd] = fma(FR[fj], FR[fk], - FR[fa])`

浮点融合乘减（双精度）（Floating-point Fused Multiply-Subtract Double） · 4R 型 · LA64

执行 `FR[fd] = (FR[fj] * FR[fk] - FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值 `0x08600000 | (fa << 15) | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fmsub.d $f0, $f1, $f2, $f3      # $f0 = ($f1 * $f2 - $f3)
```

语义伪代码

```
FR[fd] = fusedMultiplyAdd(FR[fj], FR[fk], - FR[fa])
```

---

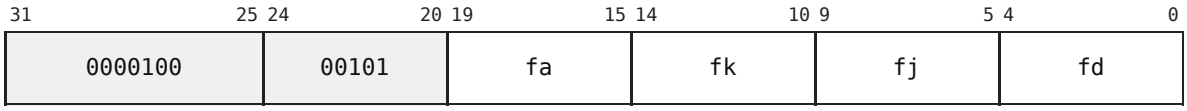
fmsub.s `fd, fj, fk, fa`

`FR[fd] = fma(FR[fj], FR[fk], - FR[fa])`

浮点融合乘减（单精度）（Floating-point Fused Multiply-Subtract Single） · 4R 型 · LA32 / LA64

执行 `FR[fd] = (FR[fj] * FR[fk] - FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值

```
0x08500000 | (fa << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmsub.s $f0, $f1, $f2, $f3      # $f0 = ($f1 * $f2 - $f3)
```

语义伪代码

```
FR[fd] = fusedMultiplyAdd(FR[fj], FR[fk], - FR[fa])
```

---

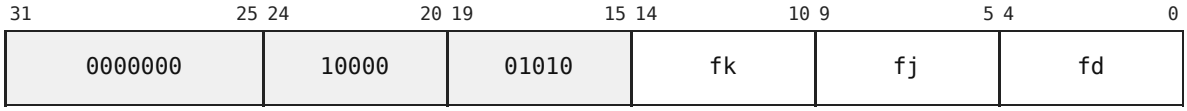
fmul.d `fd, fj, fk`

`FR[fd] = FR[fj] * FR[fk]`

浮点乘法（双精度）（Floating-point Multiply Double） · 3R 型 · LA64

对 `fj`、`fk` 执行双精度浮点乘法，结果写入 `fd`。

编码



编码值

```
0x01050000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmul.d $f0, $f1, $f2      # $f0 = fmul($f1, $f2)
```

语义伪代码

```
FR[fd] = FR[fj] * FR[fk]
```

---

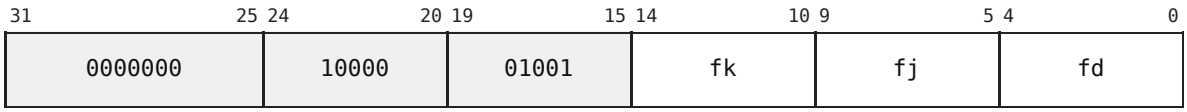
fmul.s `fd, fj, fk`

`FR[fd][31:0] = FR[fj][31:0] * FR[fk][31:0]`

浮点乘法（单精度）（Floating-point Multiply Single） · 3R 型 · LA32 / LA64

对 `fj`、`fk` 执行单精度浮点乘法，结果写入 `fd`。

编码



编码值

```
0x01048000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fmul.s $f0, $f1, $f2      # $f0 = fmul($f1, $f2)
```

语义伪代码

```
FR[fd][31:0] = FR[fj][31:0] * FR[fk][31:0]
```

---

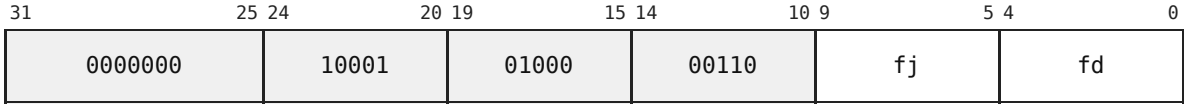
fneg.d `fd, fj`

`FR[fd] = negate(FR[fj])`

浮点取负（双精度）（Floating-point Negate Double） · 2R 型 · LA64

翻转 `fj` 的符号位后写入 `fd`。

编码



编码值 `0x01141800 | (fj << 5) | fd`

汇编示例

```
fneg.d $f0, $f1      # $f0 = negate($f1)
```

语义伪代码

```
FR[fd] = negate(FR[fj])
```

---

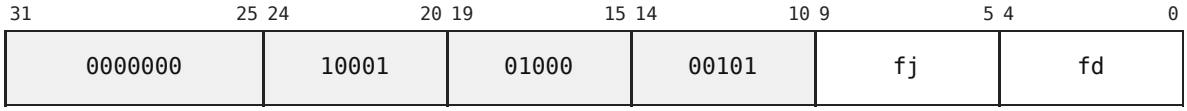
fneg.s `fd, fj`

`FR[fd] = negate(FR[fj])`

浮点取负（单精度）（Floating-point Negate Single） · 2R 型 · LA32 / LA64

翻转 `fj` 的符号位后写入 `fd`。

编码



编码值 `0x01141400 | (fj << 5) | fd`

汇编示例

```
fneg.s $f0, $f1      # $f0 = negate($f1)
```

语义伪代码

```
FR[fd] = negate(FR[fj])
```

---

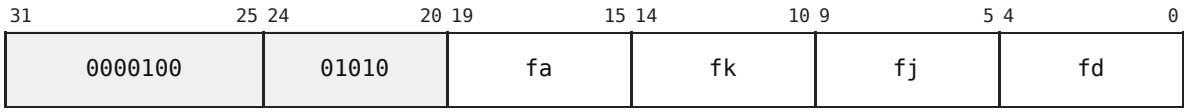
fnmadd.d `fd, fj, fk, fa`

`FR[fd] = -fma(FR[fj], FR[fk], + FR[fa])`

浮点负融合乘加（双精度）（Floating-point Negated Fused Multiply-Add Double） · 4R 型 · LA64

执行 `FR[fd] = -(FR[fj] * FR[fk] + FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值 `0x08a00000 | (fa << 15) | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fnmadd.d $f0, $f1, $f2, $f3      # $f0 = -($f1 * $f2 + $f3)
```

语义伪代码

```
FR[fd] = -fusedMultiplyAdd(FR[fj], FR[fk], + FR[fa])
```

---

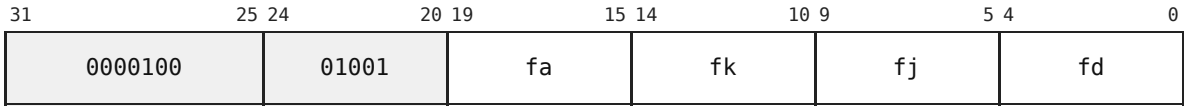
fnmadd.s `fd, fj, fk, fa`

`FR[fd] = -fma(FR[fj], FR[fk], + FR[fa])`

浮点负融合乘加（单精度） （Floating-point Negated Fused Multiply-Add Single） · 4R 型 · LA32 / LA64

执行 `FR[fd] = -(FR[fj] * FR[fk] + FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值

```
0x08900000 | (fa << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fnmadd.s $f0, $f1, $f2, $f3      # $f0 = -($f1 * $f2 + $f3)
```

语义伪代码

```
FR[fd] = -fusedMultiplyAdd(FR[fj], FR[fk], + FR[fa])
```

---

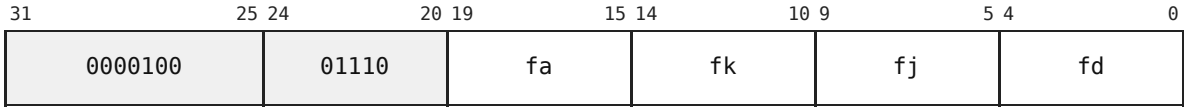
fnmsub.d `fd, fj, fk, fa`

`FR[fd] = -fma(FR[fj], FR[fk], - FR[fa])`

浮点负融合乘减（双精度） （Floating-point Negated Fused Multiply-Subtract Double） · 4R 型 · LA64

执行 `FR[fd] = -(FR[fj] * FR[fk] - FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值

```
0x08e00000 | (fa << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fnmsub.d $f0, $f1, $f2, $f3      # $f0 = -($f1 * $f2 - $f3)
```

语义伪代码

```
FR[fd] = -fusedMultiplyAdd(FR[fj], FR[fk], - FR[fa])
```

---

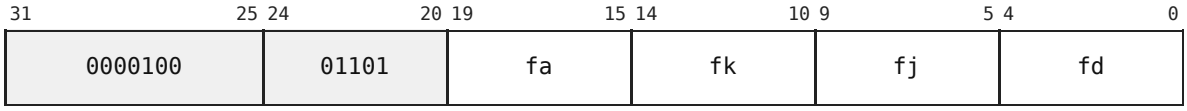
fnmsub.s `fd, fj, fk, fa`

`FR[fd] = -fma(FR[fj], FR[fk], - FR[fa])`

浮点负融合乘减（单精度） （Floating-point Negated Fused Multiply-Subtract Single） · 4R 型 · LA32 / LA64

执行 `FR[fd] = -(FR[fj] * FR[fk] - FR[fa])` 的融合乘加，按 IEEE 754 fusedMultiplyAdd 语义计算。

编码



编码值

```
0x08d00000 | (fa << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fnmsub.s $f0, $f1, $f2, $f3      # $f0 = -($f1 * $f2 - $f3)
```

语义伪代码

```
FR[fd] = -fusedMultiplyAdd(FR[fj], FR[fk], - FR[fa])
```

---

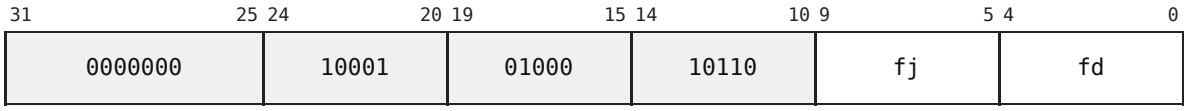
frecip.d `fd, fj`

FR[fd] = reciprocal(FR[fj])

浮点倒数（双精度）（Floating-point Reciprocal Double） · 2R 型 · LA64

计算 `1.0 / fj` 写入 `fd`。

编码



编码值 `0x01145800 | (fj << 5) | fd`

汇编示例

```
frecip.d $f0, $f1      # $f0 = reciprocal($f1)
```

语义伪代码

```
FR[fd] = reciprocal(FR[fj])
```

---

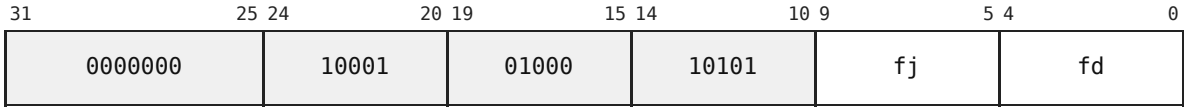
frecip.s `fd, fj`

FR[fd] = reciprocal(FR[fj])

浮点倒数（单精度）（Floating-point Reciprocal Single） · 2R 型 · LA32 / LA64

计算 `1.0 / fj` 写入 `fd`。

编码



编码值 `0x01145400 | (fj << 5) | fd`

汇编示例

```
frecip.s $f0, $f1      # $f0 = reciprocal($f1)
```

语义伪代码

```
FR[fd] = reciprocal(FR[fj])
```

---

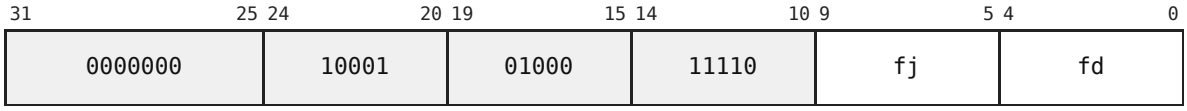
frecipe.d `fd, fj`

FR[fd] = reciprocalEstimate(FR[fj])

浮点倒数近似（双精度）（Floating-point Reciprocal Estimate Double） · 2R 型 · LA64

计算 `1.0 / fj` 的近似值，相对误差小于 2^-14。

编码



编码值 `0x01147800 | (fj << 5) | fd`

汇编示例

```
frecipe.d $f0, $f1      # $f0 = reciprocalEstimate($f1)
```

语义伪代码

```
FR[fd] = reciprocalEstimate(FR[fj])
```

---

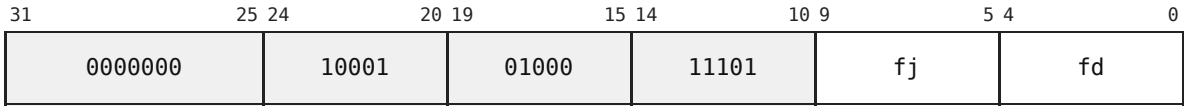
frecipe.s `fd, fj`

FR[fd] = reciprocalEstimate(FR[fj])

浮点倒数近似（单精度） （Floating-point Reciprocal Estimate Single） · 2R 型 · LA32 / LA64

计算 `1.0 / fj` 的近似值，相对误差小于 2^-14。

编码



编码值 `0x01147400 | (fj << 5) | fd`

汇编示例

```
frecipe.s $f0, $f1      # $f0 = reciprocalEstimate($f1)
```

语义伪代码

```
FR[fd] = reciprocalEstimate(FR[fj])
```

---

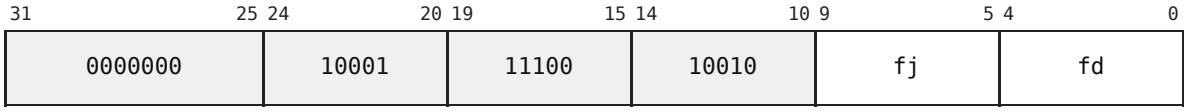
frint.d `fd, fj`

FR[fd] = roundToInteger(FR[fj])

浮点取整（双精度） （Floating-point Round to Integer Double） · 2R 型 · LA64

按 FCSR.RM 将 `fj` 取整为浮点整数值写入 `fd`。

编码



编码值 `0x011e4800 | (fj << 5) | fd`

汇编示例

```
frint.d $f0, $f1      # $f0 = roundToInteger($f1)
```

语义伪代码

```
FR[fd] = roundToInteger(FR[fj])
```

---

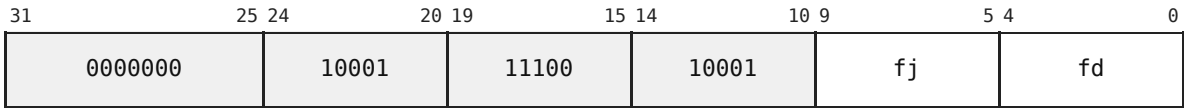
frint.s `fd, fj`

FR[fd] = roundToInteger(FR[fj])

浮点取整（单精度） （Floating-point Round to Integer Single） · 2R 型 · LA32 / LA64

按 FCSR.RM 将 `fj` 取整为浮点整数值写入 `fd`。

编码



编码值 `0x011e4400 | (fj << 5) | fd`

汇编示例

```
frint.s $f0, $f1      # $f0 = roundToInteger($f1)
```

语义伪代码

```
FR[fd] = roundToInteger(FR[fj])
```

---

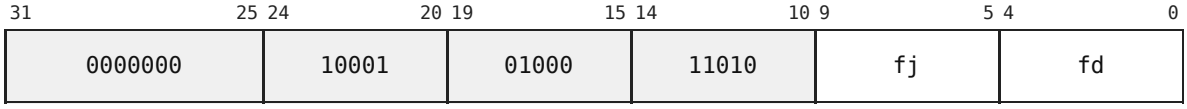
frsqrt.d `fd, fj`

`FR[fd] = reciprocalSquareRoot(FR[fj])`

浮点平方根倒数（双精度）（Floating-point Reciprocal Square Root Double） · 2R 型 · LA64

计算 `1.0 / sqrt(fj)` 写入 `fd`。

编码



编码值 `0x01146800 | (fj << 5) | fd`

汇编示例

```
frsqrt.d $f0, $f1      # $f0 = reciprocalSquareRoot($f1)
```

语义伪代码

```
FR[fd] = reciprocalSquareRoot(FR[fj])
```

---

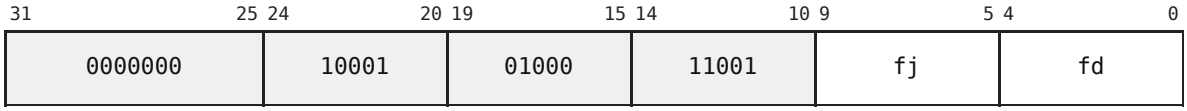
frsqrt.s `fd, fj`

`FR[fd] = reciprocalSquareRoot(FR[fj])`

浮点平方根倒数（单精度）（Floating-point Reciprocal Square Root Single） · 2R 型 · LA32 / LA64

计算 `1.0 / sqrt(fj)` 写入 `fd`。

编码



编码值 `0x01146400 | (fj << 5) | fd`

汇编示例

```
frsqrt.s $f0, $f1      # $f0 = reciprocalSquareRoot($f1)
```

语义伪代码

```
FR[fd] = reciprocalSquareRoot(FR[fj])
```

---

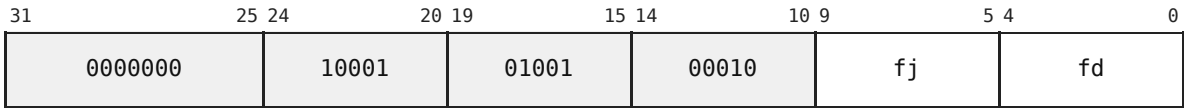
frsqrte.d `fd, fj`

`FR[fd] = reciprocalSquareRootEstimate(FR[fj])`

浮点平方根倒数近似（双精度）（Floating-point Reciprocal Square Root Estimate Double） · 2R 型 · LA64

计算 `1.0 / sqrt(fj)` 的近似值，相对误差小于 2^-14。

编码



编码值 `0x01148800 | (fj << 5) | fd`

汇编示例

```
frsqrte.d $f0, $f1      # $f0 = reciprocalSquareRootEstimate($f1)
```

语义伪代码

```
FR[fd] = reciprocalSquareRootEstimate(FR[fj])
```

---

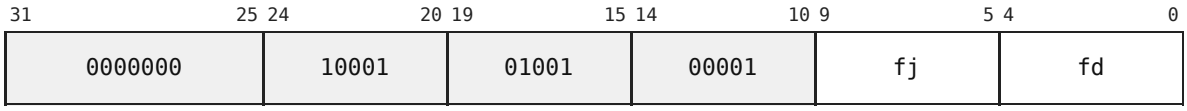
frsqrte.s `fd, fj`

`FR[fd] = reciprocalSquareRootEstimate(FR[fj])`

浮点平方根倒数近似（单精度）（Floating-point Reciprocal Square Root Estimate Single） · 2R 型 · LA32 / LA64

计算 `1.0 / sqrt(fj)` 的近似值，相对误差小于 2^-14。

编码



编码值 `0x01148400 | (fj << 5) | fd`

汇编示例

```
frsqrte.s $f0, $f1      # $f0 = reciprocalSquareRootEstimate($f1)
```

语义伪代码

```
FR[fd] = reciprocalSquareRootEstimate(FR[fj])
```

---

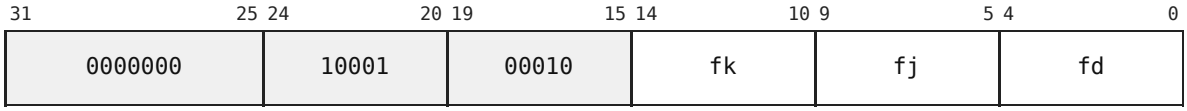
fscaleb.d `fd, fj, fk`

`FR[fd] = scaleB(FR[fj], FR[fk])`

浮点指数缩放（双精度）（Floating-point Scale by Power of 2 Double） · 3R 型 · LA64

计算 `fj × 2^N`，其中 N 为 `fk` 表示的整数（IEEE 754 scaleB），结果写入 `fd`。

编码



编码值 `0x01110000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fscaleb.d $f0, $f1, $f2      # $f0 = scaleB($f1, $f2)
```

语义伪代码

```
FR[fd] = scaleB(FR[fj], FR[fk])
```

---

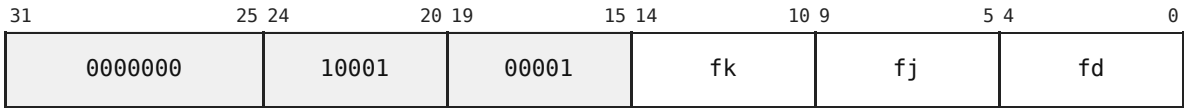
fscaleb.s `fd, fj, fk`

`FR[fd] = scaleB(FR[fj], FR[fk])`

浮点指数缩放（单精度）（Floating-point Scale by Power of 2 Single） · 3R 型 · LA32 / LA64

计算 `fj × 2^N`，其中 N 为 `fk` 表示的整数（IEEE 754 scaleB），结果写入 `fd`。

编码



编码值 `0x01108000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fscaleb.s $f0, $f1, $f2      # $f0 = scaleB($f1, $f2)
```

语义伪代码

```
FR[fd] = scaleB(FR[fj], FR[fk])
```

---

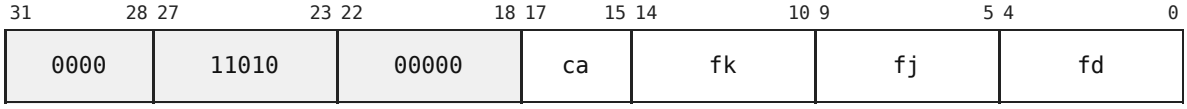
fsel `fd, fj, fk, ca`

`FR[fd] = CFR[ca] ? FR[fk] : FR[fj]`

浮点条件选择 （Floating-point Select） · 4R 型 · LA32 / LA64

条件标志 `ca` 为 0 时选择 `fj`，否则选择 `fk`，结果写入 `fd`。

编码



编码值

```
0x0d000000 | (ca << 15) | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fsel $f0, $f1, $f2, $fcc0      # $f0 = $fcc0 ? $f2 : $f1
```

语义伪代码

```
FR[fd] = CFR[ca] ? FR[fk] : FR[fj]
```

---

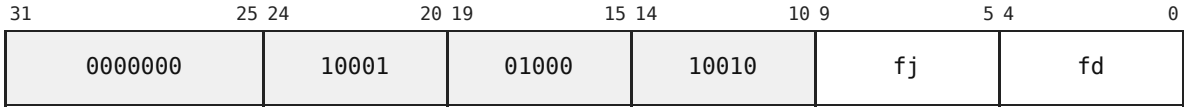
fsqrt.d `fd, fj`

`FR[fd] = squareRoot(FR[fj])`

浮点平方根（双精度） （Floating-point Square Root Double） · 2R 型 · LA64

计算 `fj` 的平方根写入 `fd`。

编码



编码值

```
0x01144800 | (fj << 5) | fd
```

汇编示例

```
fsqrt.d $f0, $f1      # $f0 = squareRoot($f1)
```

语义伪代码

```
FR[fd] = squareRoot(FR[fj])
```

---

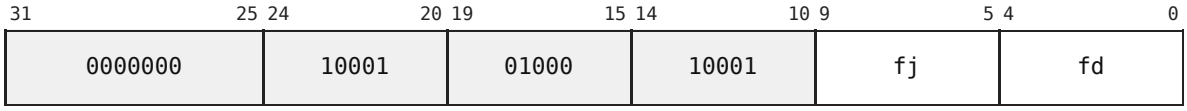
fsqrt.s `fd, fj`

`FR[fd] = squareRoot(FR[fj])`

浮点平方根（单精度） （Floating-point Square Root Single） · 2R 型 · LA32 / LA64

计算 `fj` 的平方根写入 `fd`。

编码



编码值

```
0x01144400 | (fj << 5) | fd
```

汇编示例

```
fsqrt.s $f0, $f1      # $f0 = squareRoot($f1)
```

语义伪代码

```
FR[fd] = squareRoot(FR[fj])
```

---

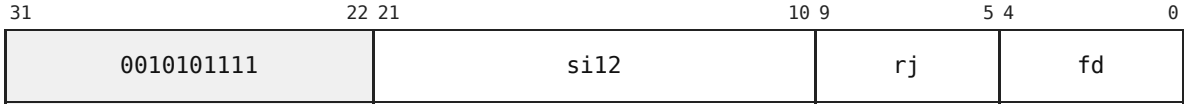
fst.d `fd, rj, si12`

`Memory[GR[rj] + si12] = FR[fd][63:0]`

浮点存储（双精度）（Floating-point Store Double） · 2RI12 型 · LA64

地址 `rj + si12` 处存储双精度浮点数。

编码



编码值

```
0x2bc00000 | ((si12 & 0xfff) << 10) | (rj << 5) | fd
```

汇编示例

```
fst.d $f0, $a0, 0      # [$a0] = $f0
```

语义伪代码

```
Memory[GR[rj] + SignExtend(si12, GRLEN)] = FR[fd][63:0]
```

---

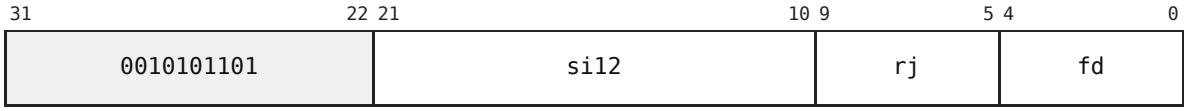
fst.s `fd, rj, si12`

`Memory[GR[rj] + si12] = FR[fd][31:0]`

浮点存储（单精度）（Floating-point Store Single） · 2RI12 型 · LA32 / LA64

地址 `rj + si12` 处存储单精度浮点数。

编码



编码值

```
0x2b400000 | ((si12 & 0xfff) << 10) | (rj << 5) | fd
```

汇编示例

```
fst.s $f0, $a0, 0      # [$a0] = $f0
```

语义伪代码

```
Memory[GR[rj] + SignExtend(si12, GRLEN)] = FR[fd][31:0]
```

---

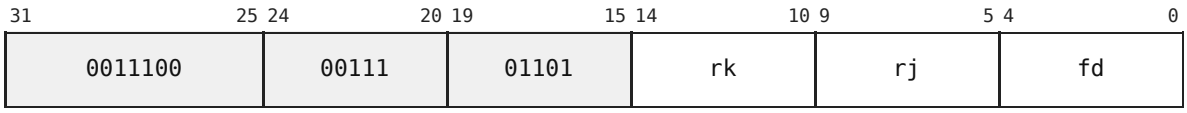
fstgt.d `fd, rj, rk`

`if (GR[rj] > GR[rk]) Memory[GR[rj]] = FR[fd][63:0] else raise BCE`

浮点存储（大于检查，双精度）（Floating-point Store Greater Than Double） · 3R 型 · LA64

检查 `rj > rk`，满足时存储浮点数，否则触发边界检查异常（BCE）。

编码



编码值

```
0x38768000 | (rk << 10) | (rj << 5) | fd
```

汇编示例

```
fstgt.d $f0, $a0, $a1      # if ($a0 > $a1) store
```

语义伪代码

```
if (GR[rj] > GR[rk]) Memory[GR[rj]] = FR[fd][63:0] else raise BCE
```

---

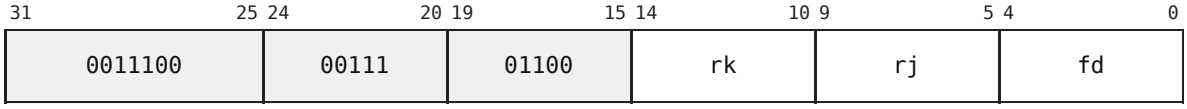
fstgt.s `fd, rj, rk`

`if (GR[rj] > GR[rk]) Memory[GR[rj]] = FR[fd][31:0] else raise BCE`

浮点存储（大于检查，单精度）（Floating-point Store Greater Than Single） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时存储浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38760000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fstgt.s $f0, $a0, $a1      # if ($a0 > $a1) store
```

语义伪代码

```
if (GR[rj] > GR[rk]) Memory[GR[rj]] = FR[fd][31:0] else raise BCE
```

---

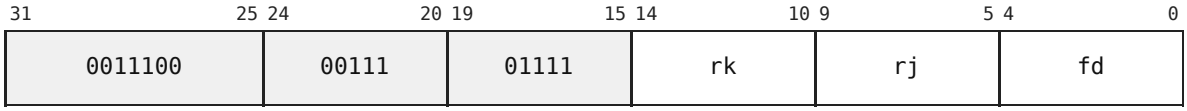
fstle.d `fd, rj, rk`

`if (GR[rj] <= GR[rk]) Memory[GR[rj]] = FR[fd][63:0] else raise BCE`

浮点存储（小于等于检查，双精度）（Floating-point Store Less Equal Double） · 3R 型 · LA64

检查 `rj <= rk`，满足时存储浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38778000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fstle.d $f0, $a0, $a1      # if ($a0 <= $a1) store
```

语义伪代码

```
if (GR[rj] <= GR[rk]) Memory[GR[rj]] = FR[fd][63:0] else raise BCE
```

---

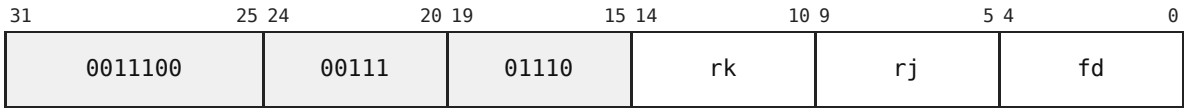
fstle.s `fd, rj, rk`

`if (GR[rj] <= GR[rk]) Memory[GR[rj]] = FR[fd][31:0] else raise BCE`

浮点存储（小于等于检查，单精度）（Floating-point Store Less Equal Single） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时存储浮点数，否则触发边界检查异常（BCE）。

编码



编码值 `0x38770000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fstle.s $f0, $a0, $a1      # if ($a0 <= $a1) store
```

语义伪代码

```
if (GR[rj] <= GR[rk]) Memory[GR[rj]] = FR[fd][31:0] else raise BCE
```

---

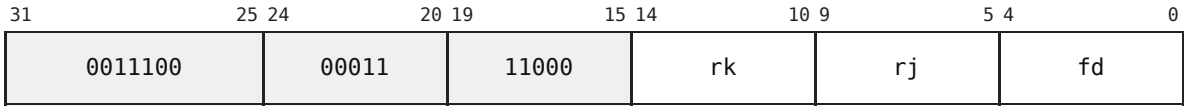
fstx.d `fd, rj, rk`

`Memory[GR[rj] + GR[rk]] = FR[fd][63:0]`

浮点索引存储（双精度）（Floating-point Indexed Store Double） · 3R 型 · LA64

地址 `rj + rk` 处存储双精度浮点数。

编码



编码值 `0x383c0000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fstx.d $f0, $a0, $a1      # [$a0+$a1] = $f0
```

语义伪代码

```
Memory[GR[rj] + GR[rk]] = FR[fd][63:0]
```

---

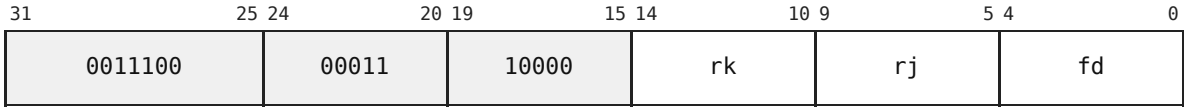
fstx.s `fd, rj, rk`

`Memory[GR[rj] + GR[rk]] = FR[fd][31:0]`

浮点索引存储（单精度）（Floating-point Indexed Store Single） · 3R 型 · LA32 / LA64

地址 `rj + rk` 处存储单精度浮点数。

编码



编码值 `0x38380000 | (rk << 10) | (rj << 5) | fd`

汇编示例

```
fstx.s $f0, $a0, $a1      # [$a0+$a1] = $f0
```

语义伪代码

```
Memory[GR[rj] + GR[rk]] = FR[fd][31:0]
```

---

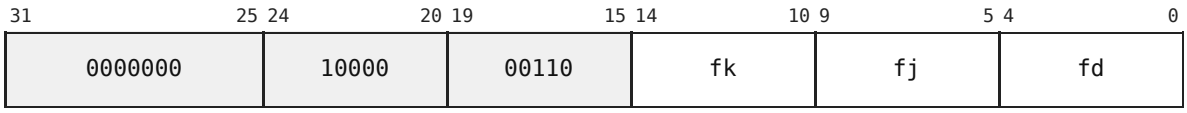
fsub.d `fd, fj, fk`

`FR[fd] = FR[fj] - FR[fk]`

浮点减法（双精度）（Floating-point Subtract Double） · 3R 型 · LA64

对 `fj`、`fk` 执行双精度浮点减法，结果写入 `fd`。

编码



编码值 `0x01030000 | (fk << 10) | (fj << 5) | fd`

汇编示例

```
fsub.d $f0, $f1, $f2      # $f0 = fsub($f1, $f2)
```

语义伪代码

```
FR[fd] = FR[fj] - FR[fk]
```

---

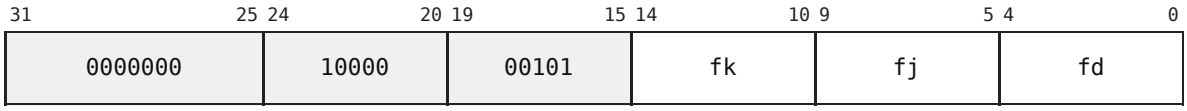
fsub.s `fd, fj, fk`

`FR[fd][31:0] = FR[fj][31:0] - FR[fk][31:0]`

浮点减法（单精度）（Floating-point Subtract Single） · 3R 型 · LA32 / LA64

对 `fj`、`fk` 执行单精度浮点减法，结果写入 `fd`。

编码



编码值

```
0x01028000 | (fk << 10) | (fj << 5) | fd
```

汇编示例

```
fsub.s $f0, $f1, $f2      # $f0 = fsub($f1, $f2)
```

语义伪代码

```
FR[fd][31:0] = FR[fj][31:0] - FR[fk][31:0]
```

---

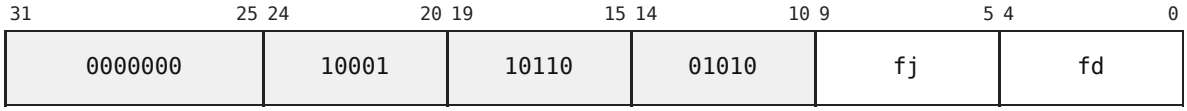
ftint.l.d `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], FCSR.RM)`

浮点转整数（按 FCSR 舍入，双字）（Float to Integer Doubleword from Double） · 2R 型 · LA64

按 FCSR.RM 舍入，将双精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值

```
0x011b2800 | (fj << 5) | fd
```

汇编示例

```
ftint.l.d $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], FCSR.RM)
```

---

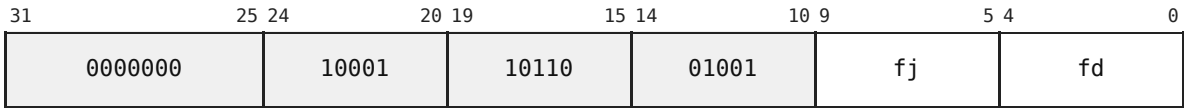
ftint.l.s `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], FCSR.RM)`

浮点转整数（按 FCSR 舍入，双字）（Float to Integer Doubleword from Single） · 2R 型 · LA64

按 FCSR.RM 舍入，将单精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值

```
0x011b2400 | (fj << 5) | fd
```

汇编示例

```
ftint.l.s $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], FCSR.RM)
```

---

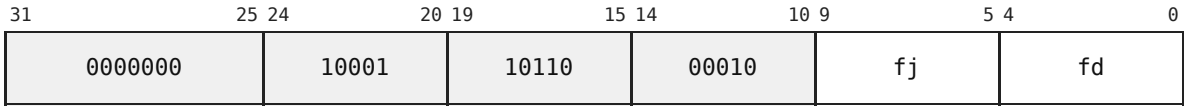
ftint.w.d fd, fj

GR[rd] = ConvertToInt32(FR[fj], FCSR.RM)

浮点转整数（按 FCSR 舍入，字） （Float to Integer Word from Double） · 2R 型 · LA64

按 FCSR.RM 舍入，将双精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011b0800 | (fj << 5) | fd`

汇编示例

```
ftint.w.d $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], FCSR.RM)
```

---

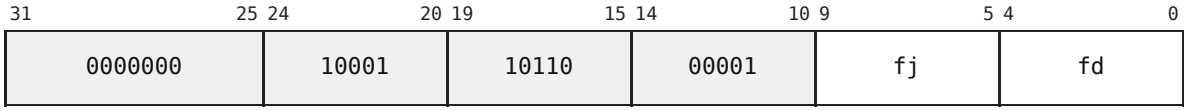
ftint.w.s fd, fj

GR[rd] = ConvertToInt32(FR[fj], FCSR.RM)

浮点转整数（按 FCSR 舍入，字） （Float to Integer Word from Single） · 2R 型 · LA32 / LA64

按 FCSR.RM 舍入，将单精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011b0400 | (fj << 5) | fd`

汇编示例

```
ftint.w.s $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], FCSR.RM)
```

---

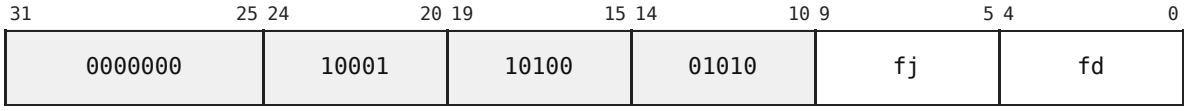
ftintrm.l.d fd, fj

GR[rd] = ConvertToInt64(FR[fj], roundMinusInfinity)

浮点转整数（向 −∞，双字） （Float to Integer Round Minus Infinity Doubleword from Double） · 2R 型 · LA64

按向 −∞舍入，将双精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011a2800 | (fj << 5) | fd`

汇编示例

```
ftintrm.l.d $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundMinusInfinity)
```

---

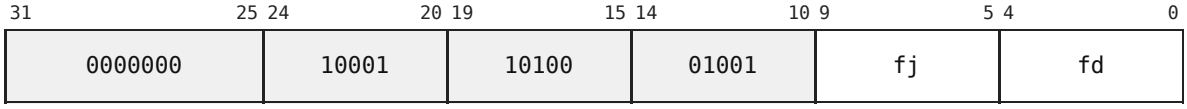
ftintrm.l.s `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundMinusInfinity)`

浮点转整数（向 −∞，双字）（Float to Integer Round Minus Infinity Doubleword from Single） · 2R 型 · LA64

按向 −∞舍入，将单精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011a2400 | (fj << 5) | fd`

汇编示例

```
ftintrm.l.s $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundMinusInfinity)
```

---

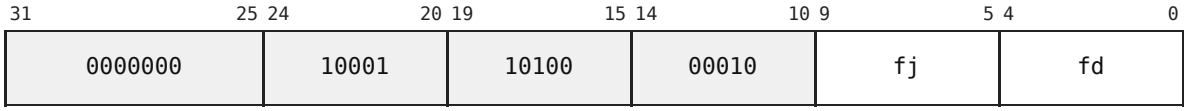
ftintrm.w.d `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundMinusInfinity)`

浮点转整数（向 −∞，字）（Float to Integer Round Minus Infinity Word from Double） · 2R 型 · LA64

按向 −∞舍入，将双精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a0800 | (fj << 5) | fd`

汇编示例

```
ftintrm.w.d $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundMinusInfinity)
```

---

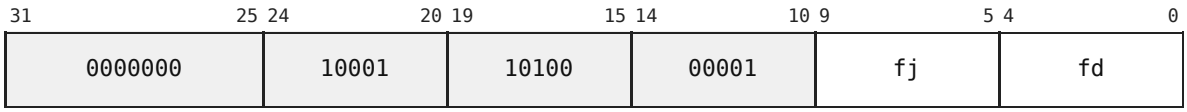
ftintrm.w.s `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundMinusInfinity)`

浮点转整数（向 −∞，字）（Float to Integer Round Minus Infinity Word from Single） · 2R 型 · LA32 / LA64

按向 −∞舍入，将单精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a0400 | (fj << 5) | fd`

汇编示例

```
ftintrm.w.s $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundMinusInfinity)
```

---

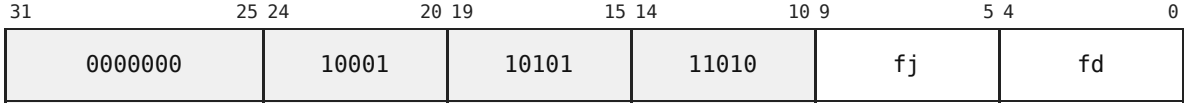
ftintrne.l.d `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundToNearestEven)`

浮点转整数（就近偶数，双字）（Float to Integer Round to Nearest Even Doubleword from Double） · 2R 型 · LA64

按就近偶数舍入，将双精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011ae800 | (fj << 5) | fd`

汇编示例

```
ftintrne.l.d $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundToNearestEven)
```

---

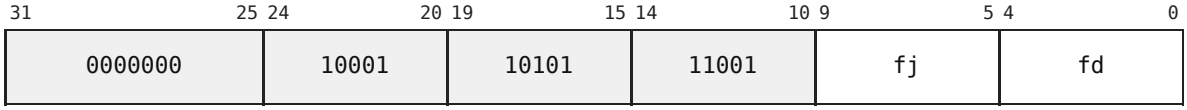
ftintrne.l.s `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundToNearestEven)`

浮点转整数（就近偶数，双字）（Float to Integer Round to Nearest Even Doubleword from Single） · 2R 型 · LA64

按就近偶数舍入，将单精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011ae400 | (fj << 5) | fd`

汇编示例

```
ftintrne.l.s $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundToNearestEven)
```

---

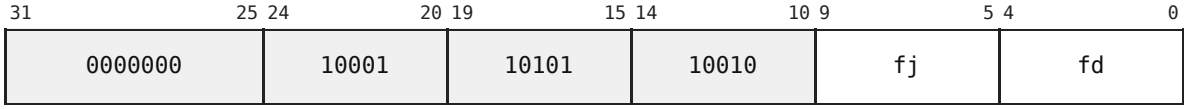
ftintrne.w.d `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundToNearestEven)`

浮点转整数（就近偶数，字）（Float to Integer Round to Nearest Even Word from Double） · 2R 型 · LA64

按就近偶数舍入，将双精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011ac800 | (fj << 5) | fd`

汇编示例

```
ftintrne.w.d $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundToNearestEven)
```

---

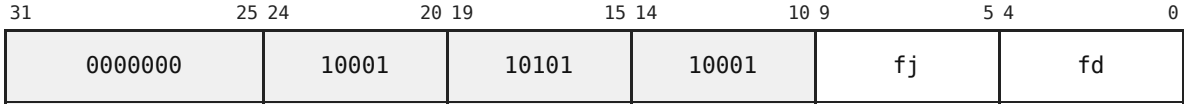
ftintrne.w.s `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundToNearestEven)`

浮点转整数（就近偶数，字） （Float to Integer Round to Nearest Even Word from Single） · 2R 型 · LA32 / LA64

按就近偶数舍入，将单精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011ac400 | (fj << 5) | fd`

汇编示例

```
ftintrne.w.s $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundToNearestEven)
```

---

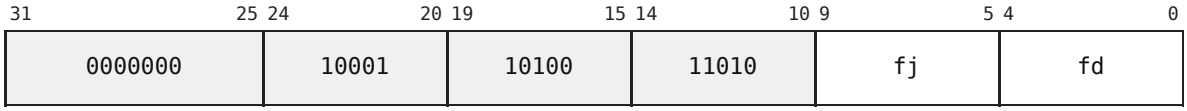
ftintrp.l.d `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundPlusInfinity)`

浮点转整数（向 +∞，双字） （Float to Integer Round Plus Infinity Doubleword from Double） · 2R 型 · LA64

按向 +∞舍入，将双精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011a6800 | (fj << 5) | fd`

汇编示例

```
ftintrp.l.d $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundPlusInfinity)
```

---

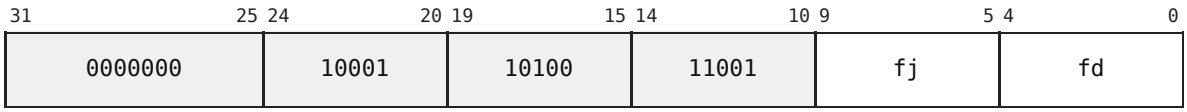
ftintrp.l.s `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundPlusInfinity)`

浮点转整数（向 +∞，双字） （Float to Integer Round Plus Infinity Doubleword from Single） · 2R 型 · LA64

按向 +∞舍入，将单精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011a6400 | (fj << 5) | fd`

汇编示例

```
ftintrp.l.s $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundPlusInfinity)
```

---

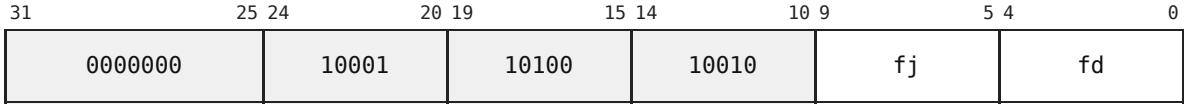
ftintrp.w.d `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundPlusInfinity)`

浮点转整数（向 +∞，字） （Float to Integer Round Plus Infinity Word from Double） · 2R 型 · LA64

按向 +∞舍入，将双精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a4800 | (fj << 5) | fd`

汇编示例

```
ftintrp.w.d $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundPlusInfinity)
```

---

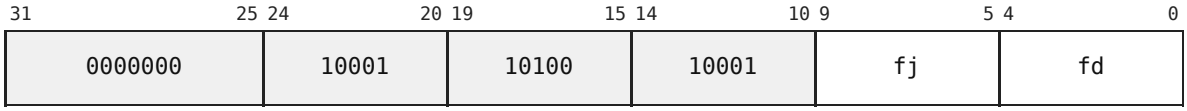
ftintrp.w.s `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundPlusInfinity)`

浮点转整数（向 +∞，字） （Float to Integer Round Plus Infinity Word from Single） · 2R 型 · LA32 / LA64

按向 +∞舍入，将单精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a4400 | (fj << 5) | fd`

汇编示例

```
ftintrp.w.s $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundPlusInfinity)
```

---

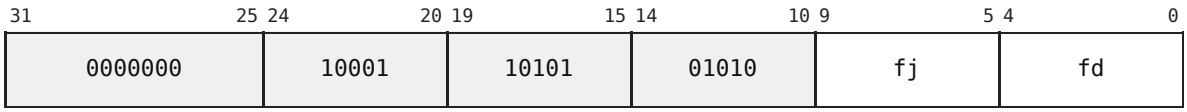
ftintrz.l.d `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundTowardZero)`

浮点转整数（向零，双字） （Float to Integer Round Toward Zero Doubleword from Double） · 2R 型 · LA64

按向零舍入，将双精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011aa800 | (fj << 5) | fd`

汇编示例

```
ftintrz.l.d $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundTowardZero)
```

---

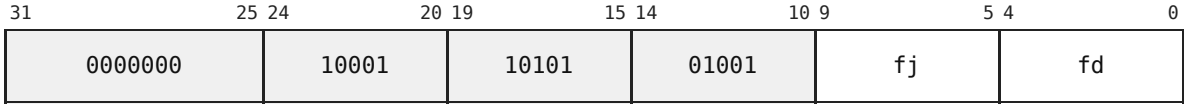
ftintrz.l.s `fd, fj`

`GR[rd] = ConvertToInt64(FR[fj], roundTowardZero)`

浮点转整数（向零，双字） （Float to Integer Round Toward Zero Doubleword from Single） · 2R 型 · LA64

按向零舍入，将单精度浮点数转换为 64 位整数并写入 `rd`。

编码



编码值 `0x011aa400 | (fj << 5) | fd`

汇编示例

```
ftintrz.l.s $a0, $f1      # $a0 = int64($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt64(FR[fj], roundTowardZero)
```

---

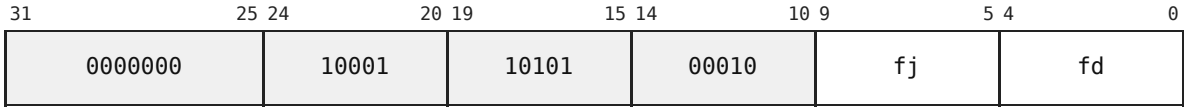
ftintrz.w.d `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundTowardZero)`

浮点转整数（向零，字） （Float to Integer Round Toward Zero Word from Double） · 2R 型 · LA64

按向零舍入，将双精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a8800 | (fj << 5) | fd`

汇编示例

```
ftintrz.w.d $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundTowardZero)
```

---

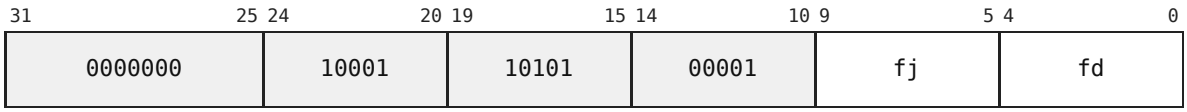
ftintrz.w.s `fd, fj`

`GR[rd] = ConvertToInt32(FR[fj], roundTowardZero)`

浮点转整数（向零，字） （Float to Integer Round Toward Zero Word from Single） · 2R 型 · LA32 / LA64

按向零舍入，将单精度浮点数转换为 32 位整数并写入 `rd`。

编码



编码值 `0x011a8400 | (fj << 5) | fd`

汇编示例

```
ftintrz.w.s $a0, $f1      # $a0 = int32($f1)
```

语义伪代码

```
GR[rd] = ConvertToInt32(FR[fj], roundTowardZero)
```

I · IOCSR、空闲、TLB 无效与指令屏障

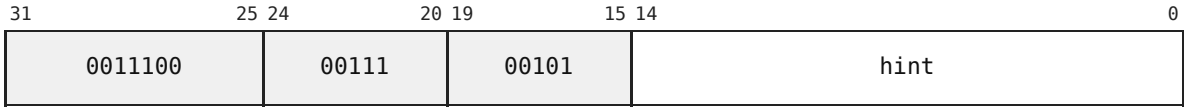
ibar `hint`

`InstructionBarrier(hint)`

指令屏障 （Instruction Barrier） · I15 型 · LA32 / LA64

保证 `IBAR` 之后的取指能看到之前所有存储操作的效果；`hint=0` 为默认完全屏障。

编码



编码值 `0x38728000 | ((hint & 0x7fff) << 0)`

汇编示例

```
ibar 0      # 完全指令屏障
```

语义伪代码

```
InstructionBarrier(hint)
```

---

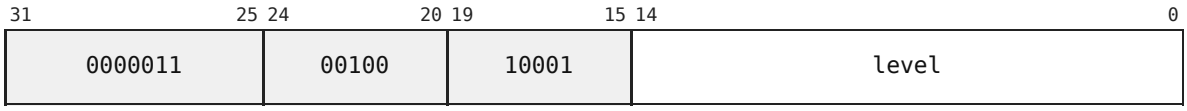
idle `level`

`WaitForInterrupt(level)`

空闲等待 （Idle） · I15 型 · LA32 / LA64

停止取指并进入等待状态，直到中断或复位唤醒；`level` 为等待级别。

编码



编码值 `0x06488000 | ((level & 0x7fff) << 0)`

汇编示例

```
idle 0      # 进入空闲等待
```

语义伪代码

```
WaitForInterrupt(level)
```

---

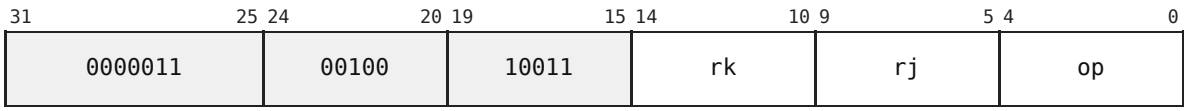
invtlb `op, rj, rk`

`InvalidateTLB(op, GR[rj], GR[rk])`

无效 TLB （Invalidate TLB） · 3R 型 · LA32 / LA64

按 `op` 指定的方式无效 TLB 表项；`rj`、`rk` 提供地址/ASID 等参数。

编码



编码值 `0x06498000 | (rk << 10) | (rj << 5) | ((op & 0x1f) << 0)`

汇编示例

```
invtlb 0, $a0, $a1      # 按 op=0 无效 TLB
```

语义伪代码

```
InvalidateTLB(op, GR[rj], GR[rk])
```

---

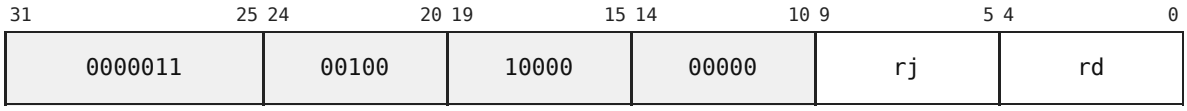
iocsrrd.b rd, rj

GR[rd] = IOCSR[GR[rj]]

IOCSR 读（字节）（IOCSR Read Byte） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 读取字节数据。

编码



编码值 `0x06480000 | (rj << 5) | rd`

汇编示例

```
iocsrrd.b $a0, $a1      # IOCSR read
```

语义伪代码

```
GR[rd] = IOCSR[GR[rj]]
```

---

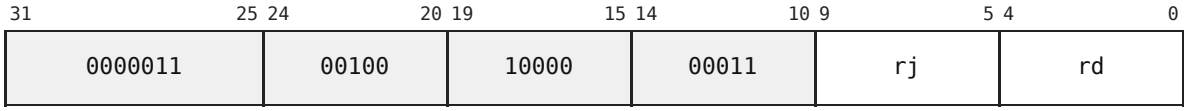
iocsrrd.d rd, rj

GR[rd] = IOCSR[GR[rj]]

IOCSR 读（双字）（IOCSR Read Doubleword） · 2R 型 · LA64

从 IOCSR 地址 `rj` 读取双字数据。

编码



编码值 `0x06480c00 | (rj << 5) | rd`

汇编示例

```
iocsrrd.d $a0, $a1      # IOCSR read
```

语义伪代码

```
GR[rd] = IOCSR[GR[rj]]
```

---

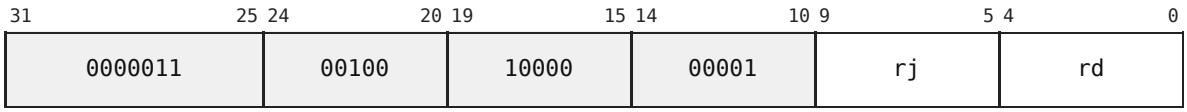
iocsrrd.h rd, rj

GR[rd] = IOCSR[GR[rj]]

IOCSR 读（半字）（IOCSR Read Halfword） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 读取半字数据。

编码



编码值 `0x06480400 | (rj << 5) | rd`

汇编示例

```
iocsrrd.h $a0, $a1      # IOCSR read
```

语义伪代码

```
GR[rd] = IOCSR[GR[rj]]
```

---

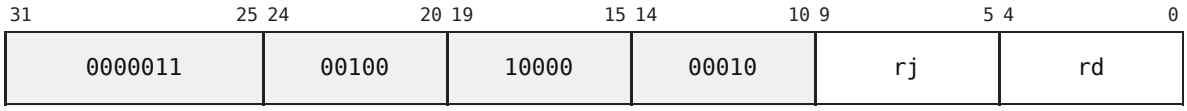
iocsrrd.w rd, rj

GR[rd] = IOCSR[GR[rj]]

IOCSR 读（字）（IOCSR Read Word） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 读取字数据。

编码



编码值 `0x06480800 | (rj << 5) | rd`

汇编示例

```
iocsrrd.w $a0, $a1      # IOCSR read
```

语义伪代码

```
GR[rd] = IOCSR[GR[rj]]
```

---

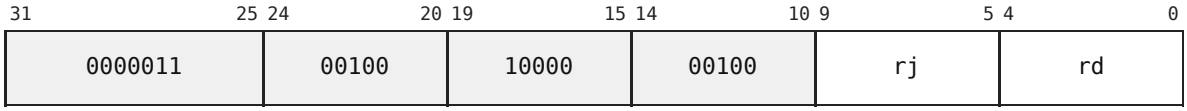
iocsrwr.b rd, rj

IOCSR[GR[rj]] = GR[rd][7:0]

IOCSR 写（字节）（IOCSR Write Byte） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 写入字节数据。

编码



编码值 `0x06481000 | (rj << 5) | rd`

汇编示例

```
iocsrwr.b $a0, $a1      # IOCSR write
```

语义伪代码

```
IOCSR[GR[rj]] = GR[rd][7:0]
```

---

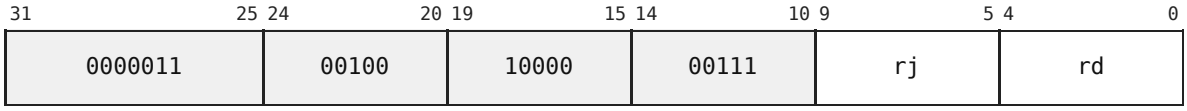
iocsrwr.d rd, rj

IOCSR[GR[rj]] = GR[rd][63:0]

IOCSR 写（双字）（IOCSR Write Doubleword） · 2R 型 · LA64

从 IOCSR 地址 `rj` 写入双字数据。

编码



编码值 `0x06481c00 | (rj << 5) | rd`

汇编示例

```
iocsrwr.d $a0, $a1      # IOCSR write
```

语义伪代码

```
IOCSR[GR[rj]] = GR[rd][63:0]
```

---

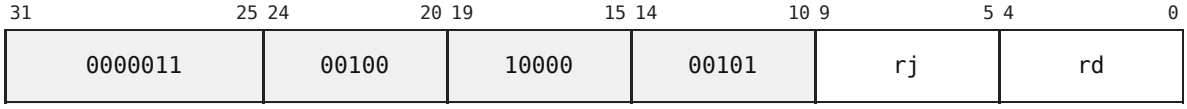
iocsrwr.h rd, rj

IOCSR[GR[rj]] = GR[rd][15:0]

IOCSR 写（半字）（IOCSR Write Halfword） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 写入半字数据。

编码



编码值 `0x06481400 | (rj << 5) | rd`

汇编示例

```
iocsrwr.h $a0, $a1      # IOCSR write
```

语义伪代码

```
IOCSR[GR[rj]] = GR[rd][15:0]
```

---

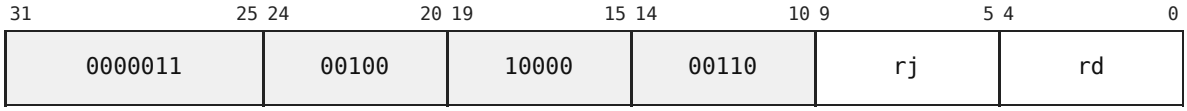
iocsrwr.w rd, rj

IOCSR[GR[rj]] = GR[rd][31:0]

IOCSR 写（字）（IOCSR Write Word） · 2R 型 · LA32 / LA64

从 IOCSR 地址 `rj` 写入字数据。

编码



编码值 `0x06481800 | (rj << 5) | rd`

汇编示例

```
iocsrwr.w $a0, $a1      # IOCSR write
```

语义伪代码

```
IOCSR[GR[rj]] = GR[rd][31:0]
```

J · 间接跳转指令

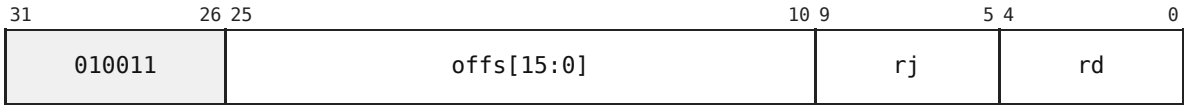
jirl rd, rj, offs

GR[rd] = PC + 4; PC = GR[rj] + SignExtend(offs, GRLEN)

间接跳转并链接（Jump and Link Register） · 2RI16 型 · LA32 / LA64

将 `PC + 4` 写入 `rd`，并跳转到 `rj + offs`。`offs` 为汇编中的字节偏移，编码字段为 `offs >> 2`。

编码



编码值 `0x4c000000 | (((offs >> 2) & 0xffff) << 10) | (rj << 5) | rd`

汇编示例

```
jirl $a0, $a1, 8      # $a0 = PC + 4; PC = $a1 + 8
```

语义伪代码

```
GR[rd] = PC + 4
PC = GR[rj] + SignExtend(offs, GRLEN)
```

L · 加载、链接加载、页表与立即数装载指令

本章中的加载/存储地址为 `GR[rj] + 立即数` 或 `GR[rj] + GR[rk]`。`si12`/`si14` 为字节偏移，直接参与地址计算。

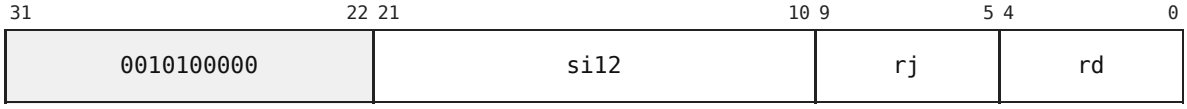
ld.b rd, rj, si12

GR[rd] = SignExtend(Memory[GR[rj]+si12, 8), GRLEN)

加载（字节符号扩展）（Load Byte Signed） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 8 位数据，符号扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x28000000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.b $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+si12, 8), GRLEN)
```

---

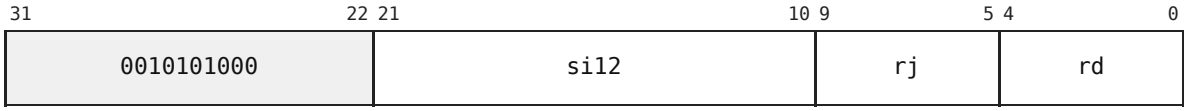
ld.bu rd, rj, si12

GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 8), GRLEN)

加载（字节零扩展）（Load Byte Unsigned） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 8 位数据，零扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x2a000000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.bu $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 8), GRLEN)
```

---

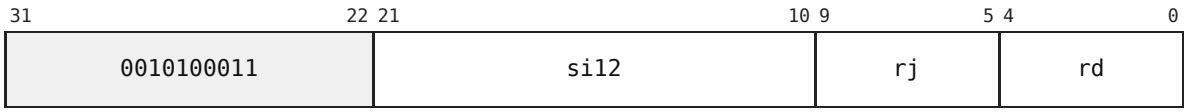
ld.d rd, rj, si12

GR[rd] = SignExtend(Memory[GR[rj]+si12, 64), GRLEN)

加载（双字符号扩展）（Load Doubleword Signed） · 2RI12 型 · LA64

从 `rj + si12` 加载 64 位数据，符号扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x28c00000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.d $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+si12, 64), GRLEN)
```

---

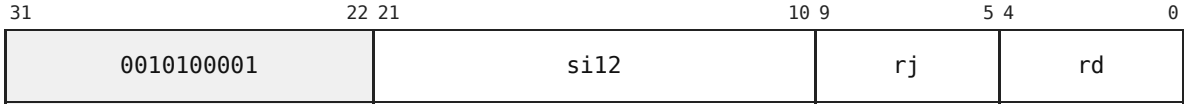
ld.h rd, rj, si12

GR[rd] = SignExtend(Memory[GR[rj]+si12, 16), GRLEN)

加载（半字符号扩展）（Load Halfword Signed） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 16 位数据，符号扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x28400000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.h $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+si12, 16), GRLEN)
```

---

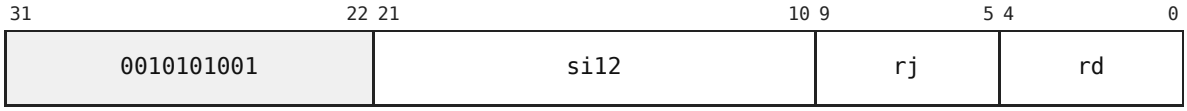
ld.hu rd, rj, si12

GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 16), GRLEN)

加载（半字零扩展）（Load Halfword Unsigned） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 16 位数据，零扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x2a400000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.hu $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 16), GRLEN)
```

---

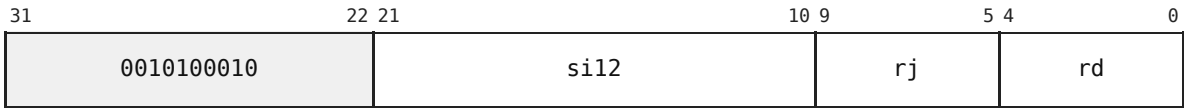
ld.w rd, rj, si12

GR[rd] = SignExtend(Memory[GR[rj]+si12, 32), GRLEN)

加载（字符号扩展）（Load Word Signed） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 32 位数据，符号扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x28800000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.w $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+si12, 32), GRLEN)
```

---

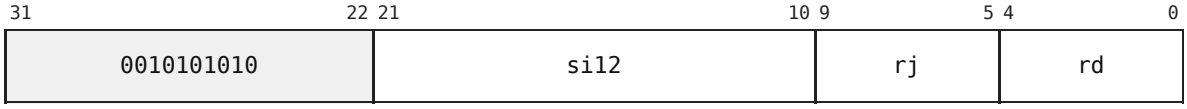
ld.wu `rd, rj, si12`

`GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 32), GRLEN)`

加载（字零扩展）（Load Word Unsigned） · 2RI12 型 · LA32 / LA64

从 `rj + si12` 加载 32 位数据，零扩展至 GRLEN 后写入 `rd`。

编码



编码值

```
0x2a800000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ld.wu $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+si12, 32), GRLEN)
```

---

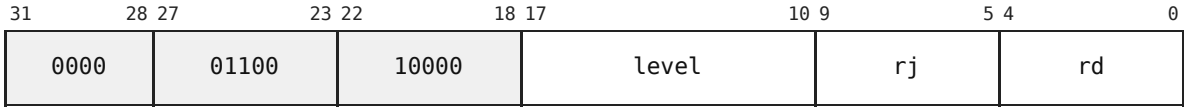
lddir `rd, rj, level`

`GR[rd] = PageWalkLoadDir(GR[rj], level)`

目录表项加载（Load Directory） · 2RI8 型 · LA32 / LA64

按 `level` 指定的页表层级，从地址 `rj` 读取目录表项并写入 `rd`。

编码



编码值

```
0x06400000 | ((level & 0xff) << 10) | (rj << 5) | rd
```

汇编示例

```
lddir $a0, $a1, 1      # 读取 level=1 的目录表项
```

语义伪代码

```
GR[rd] = PageWalkLoadDir(GR[rj], level)
```

---

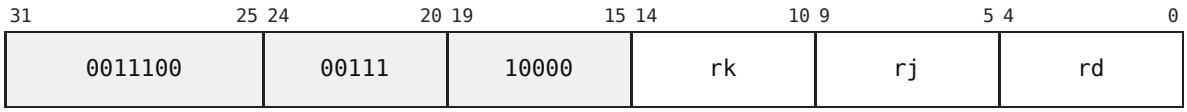
ldgt.b `rd, rj, rk`

`if (GR[rj] > GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 8) else raise BCE`

加载（大于检查，字节）（Load Greater Than Byte） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时从 `rj` 加载字节并符号扩展，否则触发边界检查异常。

编码



编码值

```
0x38780000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldgt.b $a0, $a1, $a2      # if ($a1 > $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] > GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 8)
else
    RaiseException(BCE)
```

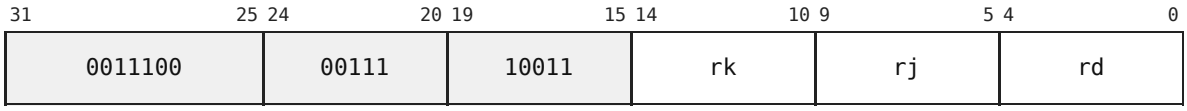
---

ldgt.d rd, rj, rk `if (GR[rj] > GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 64) else raise BCE`

加载（大于检查，双字）（Load Greater Than Doubleword） · 3R 型 · LA64

检查 `rj > rk`，满足时从 `rj` 加载双字并符号扩展，否则触发边界检查异常。

编码



编码值 `0x38798000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldgt.d $a0, $a1, $a2      # if ($a1 > $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] > GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 64)
else
    RaiseException(BCE)
```

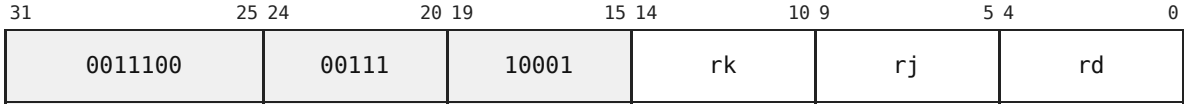
---

ldgt.h rd, rj, rk `if (GR[rj] > GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 16) else raise BCE`

加载（大于检查，半字）（Load Greater Than Halfword） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时从 `rj` 加载半字并符号扩展，否则触发边界检查异常。

编码



编码值 `0x38788000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldgt.h $a0, $a1, $a2      # if ($a1 > $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] > GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 16)
else
    RaiseException(BCE)
```

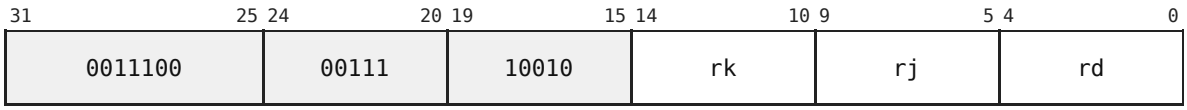
---

ldgt.w rd, rj, rk `if (GR[rj] > GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 32) else raise BCE`

加载（大于检查，字）（Load Greater Than Word） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时从 `rj` 加载字并符号扩展，否则触发边界检查异常。

编码



编码值 `0x38790000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldgt.w $a0, $a1, $a2      # if ($a1 > $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] > GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 32)
else
    RaiseException(BCE)
```

---

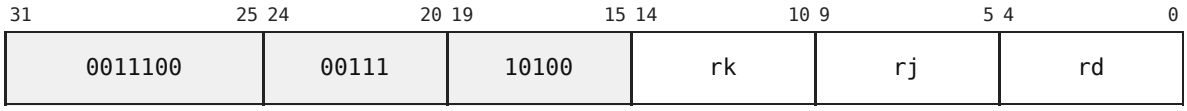
ldle.b rd, rj, rk

`if (GR[rj] <= GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 8) else raise BCE`

加载（小于等于检查，字节）（Load Less Equal Byte） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时从 `rj` 加载字节并符号扩展，否则触发边界检查异常。

编码



编码值 `0x387a0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldle.b $a0, $a1, $a2      # if ($a1 <= $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 8)
else
    RaiseException(BCE)
```

---

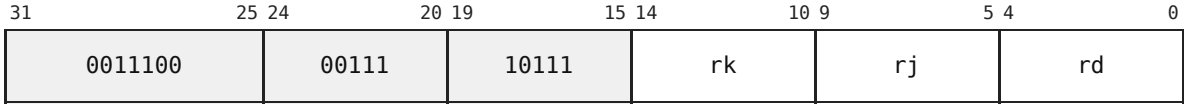
ldle.d rd, rj, rk

`if (GR[rj] <= GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 64) else raise BCE`

加载（小于等于检查，双字）（Load Less Equal Doubleword） · 3R 型 · LA64

检查 `rj <= rk`，满足时从 `rj` 加载双字并符号扩展，否则触发边界检查异常。

编码



编码值 `0x387b8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldle.d $a0, $a1, $a2      # if ($a1 <= $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 64)
else
    RaiseException(BCE)
```

---

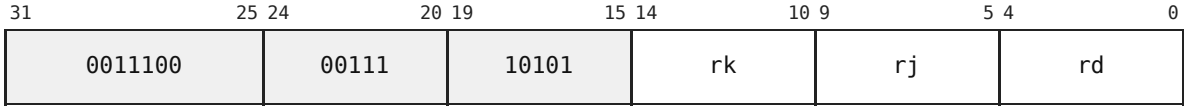
ldle.h rd, rj, rk

`if (GR[rj] <= GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 16) else raise BCE`

加载（小于等于检查，半字）（Load Less Equal Halfword） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时从 `rj` 加载半字并符号扩展，否则触发边界检查异常。

编码



编码值

```
0x387a8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldle.h $a0, $a1, $a2      # if ($a1 <= $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 16)
else
    RaiseException(BCE)
```

---

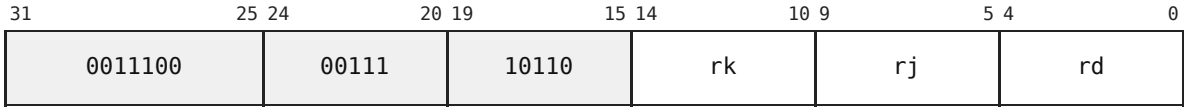
ldle.w rd, rj, rk

`if (GR[rj] <= GR[rk]) GR[rd] = SignExtend(Memory[GR[rj]], 32) else raise BCE`

加载（小于等于检查，字）（Load Less Equal Word） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时从 `rj` 加载字并符号扩展，否则触发边界检查异常。

编码



编码值

```
0x387b0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldle.w $a0, $a1, $a2      # if ($a1 <= $a2) $a0 = [$a1]
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    GR[rd] = SignExtend(Memory[GR[rj]], 32)
else
    RaiseException(BCE)
```

---

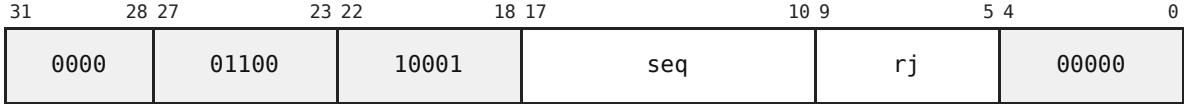
ldpte rj, seq

`PageWalkLoadPTE(GR[rj], seq)`

页表项加载（Load Page Table Entry） · 2RI8 型 · LA32 / LA64

按 `seq` 读取页表项到内部页表项寄存器，供地址翻译使用。

编码



编码值

```
0x06440000 | ((seq & 0xff) << 10) | (rj << 5)
```

汇编示例

```
ldpte $a0, 0      # 读取第 0 个页表项
```

语义伪代码

```
PageWalkLoadPTE(GR[rj], seq)
```

---

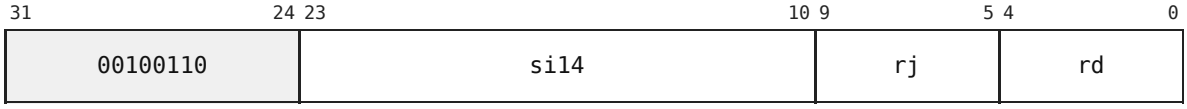
ldptr.d `rd, rj, si14`

`GR[rd] = Memory[GR[rj]+si14]`

指针加载（双字） （Load Pointer Doubleword） · 2RI14 型 · LA64

从 `rj + si14` 加载64 位值到 `rd`，常与 `addu16i.d` 配合访问 GOT。

编码



编码值

```
0x26000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
ldptr.d $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = Memory[GR[rj]+si14]
```

---

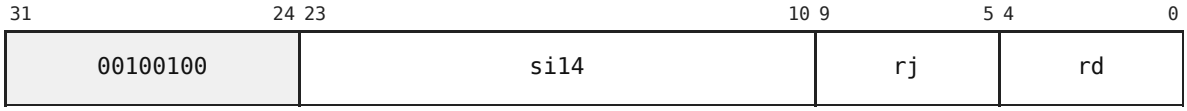
ldptr.w `rd, rj, si14`

`GR[rd] = SignExtend(Memory[GR[rj]+si14], GRLEN)`

指针加载（字） （Load Pointer Word） · 2RI14 型 · LA32 / LA64

从 `rj + si14` 加载32 位符号扩展值到 `rd`，常与 `addu16i.d` 配合访问 GOT。

编码



编码值

```
0x24000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
ldptr.w $a0, $a1, 0      # $a0 = [$a1]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+si14], GRLEN)
```

---

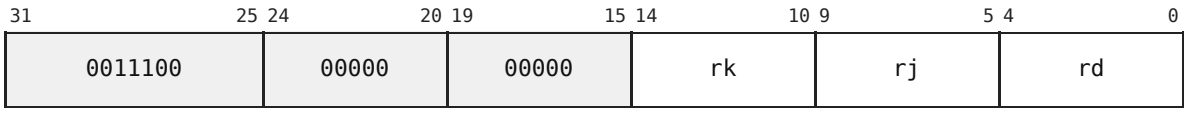
ldx.b `rd, rj, rk`

`GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 8), GRLEN)`

索引加载（字节符号扩展） （Indexed Load Byte Signed） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 8 位数据，符号扩展后写入 `rd`。

编码



编码值

```
0x38000000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldx.b $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 8), GRLEN)
```

---

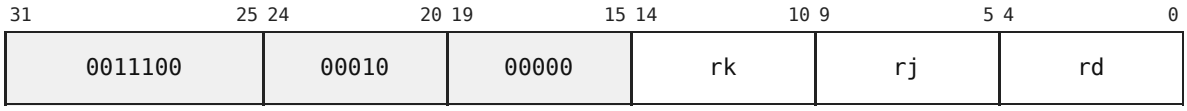
ldx.bu rd, rj, rk

GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 8), GRLEN)

索引加载（字节零扩展）（Indexed Load Byte Unsigned） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 8 位数据，零扩展后写入 `rd`。

编码



编码值 `0x38200000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldx.bu $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 8), GRLEN)
```

---

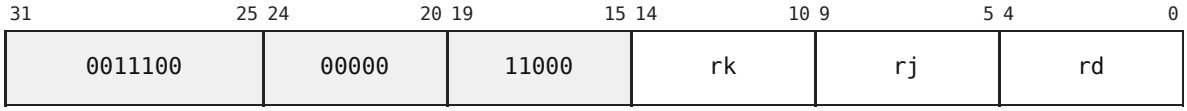
ldx.d rd, rj, rk

GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 64), GRLEN)

索引加载（双字符号扩展）（Indexed Load Doubleword Signed） · 3R 型 · LA64

从 `rj + rk` 加载 64 位数据，符号扩展后写入 `rd`。

编码



编码值 `0x380c0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldx.d $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 64), GRLEN)
```

---

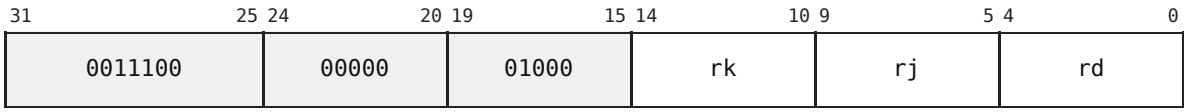
ldx.h rd, rj, rk

GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 16), GRLEN)

索引加载（半字符号扩展）（Indexed Load Halfword Signed） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 16 位数据，符号扩展后写入 `rd`。

编码



编码值 `0x38040000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
ldx.h $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 16), GRLEN)
```

---

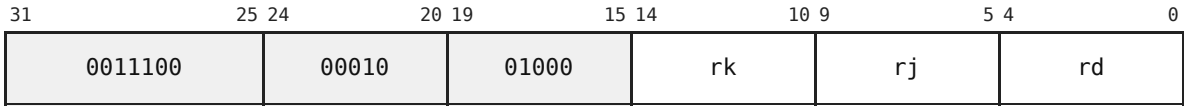
ldx.hu `rd, rj, rk`

`GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 16), GRLEN)`

索引加载（半字零扩展）（Indexed Load Halfword Unsigned） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 16 位数据，零扩展后写入 `rd`。

编码



编码值

```
0x38240000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldx.hu $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 16), GRLEN)
```

---

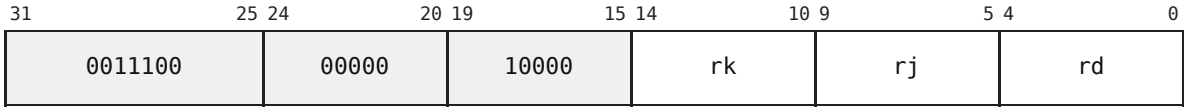
ldx.w `rd, rj, rk`

`GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 32), GRLEN)`

索引加载（字符号扩展）（Indexed Load Word Signed） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 32 位数据，符号扩展后写入 `rd`。

编码



编码值

```
0x38080000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldx.w $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]+GR[rk], 32), GRLEN)
```

---

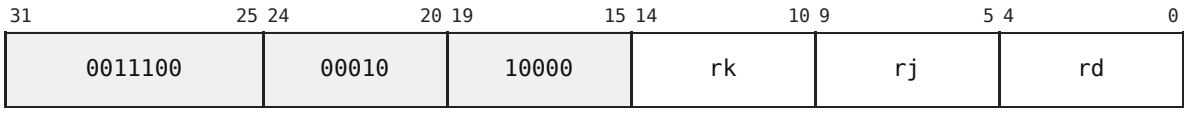
ldx.wu `rd, rj, rk`

`GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 32), GRLEN)`

索引加载（字零扩展）（Indexed Load Word Unsigned） · 3R 型 · LA32 / LA64

从 `rj + rk` 加载 32 位数据，零扩展后写入 `rd`。

编码



编码值

```
0x38280000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
ldx.wu $a0, $a1, $a2      # $a0 = [$a1+$a2]
```

语义伪代码

```
GR[rd] = ZeroExtend(Memory[GR[rj]+GR[rk], 32), GRLEN)
```

---

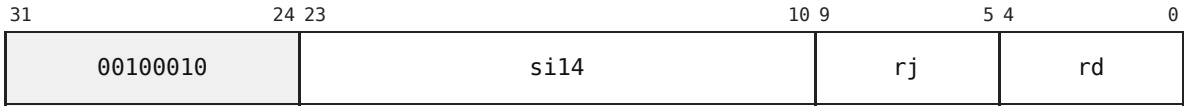
ll.d rd, rj, si14

GR[rd] = Memory[GR[rj] + si14]; LLbit = 1

链接加载（双字） （Load Linked Doubleword） · 2RI14 型 · LA64

从 `rj + si14` 加载 64 位数据到 `rd`，并置 LLbit。

编码



编码值

```
0x22000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
ll.d $a0, $a1, 0      # $a0 = [$a1]; LLbit = 1
```

语义伪代码

```
GR[rd] = Memory[GR[rj] + si14]
LLbit = 1
```

---

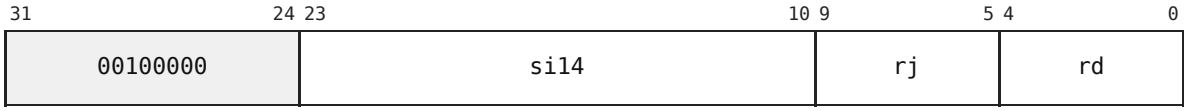
ll.w rd, rj, si14

GR[rd] = SignExtend(Memory[GR[rj] + si14], GRLEN); LLbit = 1

链接加载（字） （Load Linked Word） · 2RI14 型 · LA32 / LA64

从 `rj + si14` 加载 32 位数据，符号扩展后写入 `rd`，并置 LLbit。

编码



编码值

```
0x20000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
ll.w $a0, $a1, 0      # $a0 = [$a1]; LLbit = 1
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj] + si14], GRLEN)
LLbit = 1
```

---

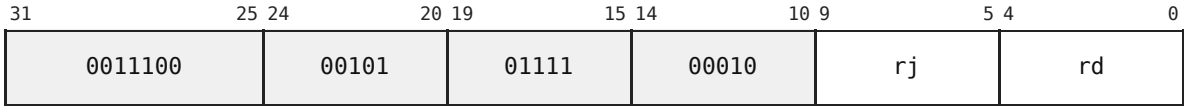
llacq.d rd, rj

GR[rd] = Memory[GR[rj]]; LLbit = 1; AcquireBarrier()

获取链接加载（双字） （Load Linked Acquire Doubleword） · 2R 型 · LA64

带获取语义的链接加载：读取 `rj` 地址的 64 位数据，置 LLbit，并执行获取屏障。

编码



编码值

```
0x38578800 | (rj << 5) | rd
```

汇编示例

```
llacq.d $a0, $a1      # $a0 = [$a1]; acquire
```

语义伪代码

```
GR[rd] = Memory[GR[rj]]
LLbit = 1
AcquireBarrier()
```

---

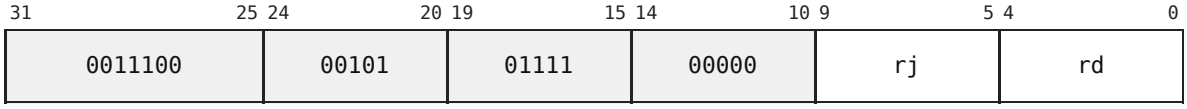
llacq.w rd, rj

GR[rd] = SignExtend(Memory[GR[rj]], GRLEN); LLbit = 1; AcquireBarrier()

获取链接加载（字）（Load Linked Acquire Word） · 2R 型 · LA32 / LA64

带获取语义的链接加载：读取 `rj` 地址的 32 位数据，置 LLbit，并执行获取屏障。

编码



编码值

```
0x38578000 | (rj << 5) | rd
```

汇编示例

```
llacq.w $a0, $a1      # $a0 = [$a1]; acquire
```

语义伪代码

```
GR[rd] = SignExtend(Memory[GR[rj]], GRLEN)
LLbit = 1
AcquireBarrier()
```

---

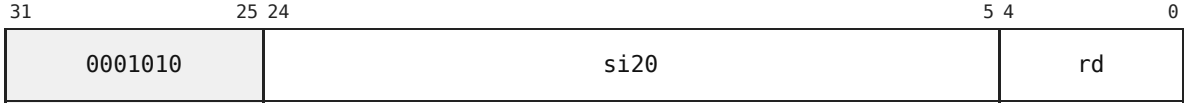
lu12i.w rd, si20

GR[rd] = SignExtend({si20, 12'b0}, GRLEN)

装载 20 位立即数到高 12 位（Load Upper 12-bit Immediate Word） · 1RI20 型 · LA32 / LA64

将 20 位立即数左移 12 位并符号扩展后写入 `rd`。

编码



编码值

```
0x14000000 | ((si20 & 0xfffff) << 5) | rd
```

汇编示例

```
lu12i.w $a0, 1      # $a0 = 0x1000
```

语义伪代码

```
GR[rd] = SignExtend({si20, 12'b0}, GRLEN)
```

---

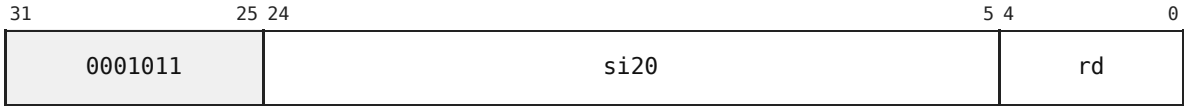
lu32i.d rd, si20

GR[rd] = {SignExtend(si20, 32), GR[rd][31:0]}

装载 20 位立即数到高 32 位（Load Upper 32-bit Immediate Doubleword） · 1RI20 型 · LA64

将 20 位立即数符号扩展为 32 位，替换 `rd` 的高 32 位，低 32 位保持不变。

编码



编码值

```
0x16000000 | ((si20 & 0xfffff) << 5) | rd
```

汇编示例

```
lu32i.d $a0, 1      # $a0[63:32] = 1
```

语义伪代码

```
GR[rd] = {SignExtend(si20, 32), GR[rd][31:0]}
```

---

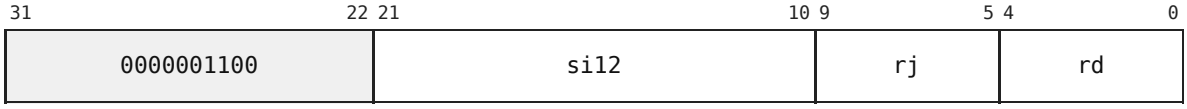
lu52i.d rd, rj, si12

GR[rd] = {si12, GR[rj][51:0]}

装载 12 位立即数到高 52 位（Load Upper 52-bit Immediate Doubleword） · 2RI12 型 · LA64

用 12 位立即数替换 `rj` 的高 12 位，结果写入 `rd`。

编码



编码值

```
0x03000000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
lu52i.d $a0, $a1, 1      # $a0[63:52] = 1
```

语义伪代码

```
GR[rd] = {si12, GR[rj][51:0]}
```

M · 乘法、取模与浮点搬移指令

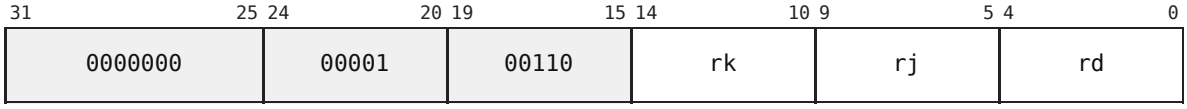
maskeqz rd, rj, rk

GR[rd] = (GR[rk] == 0) ? 0 : GR[rj]

条件置零（等于零）（Mask if Equal Zero） · 3R 型 · LA32 / LA64

`rk` 为 0 时将 `rd` 置 0，否则写入 `rj`。

编码



编码值

```
0x00130000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
maskeqz $a0, $a1, $a2      # $a0 = ($a2 == 0) ? 0 : $a1
```

语义伪代码

```
GR[rd] = (GR[rk] == 0) ? 0 : GR[rj]
```

---

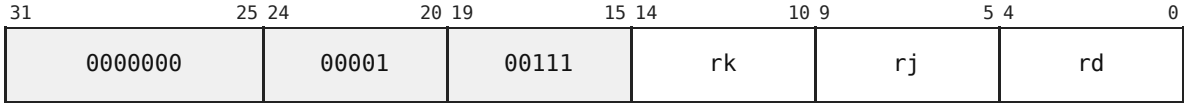
masknez rd, rj, rk

GR[rd] = (GR[rk] != 0) ? 0 : GR[rj]

条件置零（不等于零）（Mask if Not Equal Zero） · 3R 型 · LA32 / LA64

`rk` 不为 0 时将 `rd` 置 0，否则写入 `rj`。

编码



编码值

```
0x00138000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
masknez $a0, $a1, $a2      # $a0 = ($a2 != 0) ? 0 : $a1
```

语义伪代码

```
GR[rd] = (GR[rk] != 0) ? 0 : GR[rj]
```

---

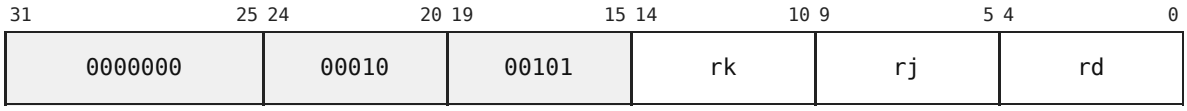
mod.d rd, rj, rk

GR[rd] = signed(GR[rj]) % signed(GR[rk])

取模（64 位有符号）（Modulo 64 Signed） · 3R 型 · LA64

将 rj 对 rk 取模，余数写入 rd。

编码



编码值

```
0x00228000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mod.d $a0, $a1, $a2      # $a0 = $a1 % $a2
```

语义伪代码

```
GR[rd] = signed(GR[rj]) % signed(GR[rk])
```

---

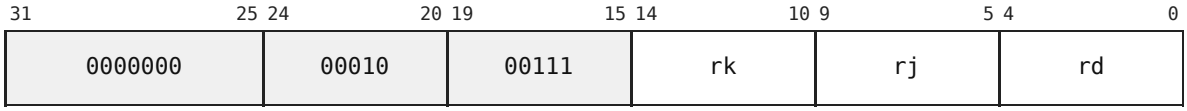
mod.du rd, rj, rk

GR[rd] = unsigned(GR[rj]) % unsigned(GR[rk])

取模（64 位无符号）（Modulo 64 Unsigned） · 3R 型 · LA64

将 rj 对 rk 取模，余数写入 rd。

编码



编码值

```
0x00238000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mod.du $a0, $a1, $a2      # $a0 = $a1 % $a2
```

语义伪代码

```
GR[rd] = unsigned(GR[rj]) % unsigned(GR[rk])
```

---

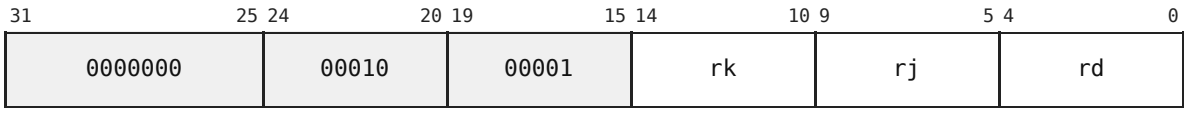
mod.w rd, rj, rk

GR[rd] = signed(GR[rj]) % signed(GR[rk])

取模（32 位有符号）（Modulo 32 Signed） · 3R 型 · LA32 / LA64

将 rj 对 rk 取模，余数写入 rd。

编码



编码值

```
0x00208000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mod.w $a0, $a1, $a2      # $a0 = $a1 % $a2
```

语义伪代码

```
GR[rd] = signed(GR[rj]) % signed(GR[rk])
```

---

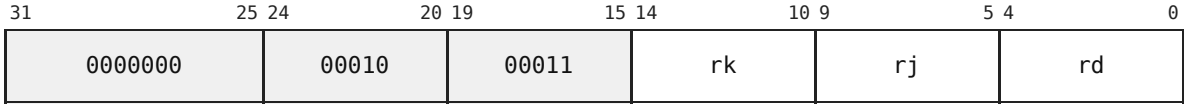
mod.wu rd, rj, rk

GR[rd] = unsigned(GR[rj]) % unsigned(GR[rk])

取模（32 位无符号）（Modulo 32 Unsigned） · 3R 型 · LA32 / LA64

将 `rj` 对 `rk` 取模，余数写入 `rd`。

编码



编码值 `0x00218000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
mod.wu $a0, $a1, $a2      # $a0 = $a1 % $a2
```

语义伪代码

```
GR[rd] = unsigned(GR[rj]) % unsigned(GR[rk])
```

---

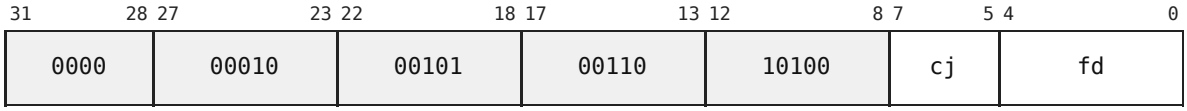
movcf2fr fd, cj

FR[fd][0] = CFR[cj]

条件标志到浮点寄存器（Move Condition Flag to FPR） · 2R 型 · LA32 / LA64

将条件标志寄存器 `cj` 的值写入 `fd` 的最低位。

编码



编码值 `0x0114d400 | (cj << 5) | fd`

汇编示例

```
movcf2fr $f0, $fcc0      # $f0[0] = $fcc0
```

语义伪代码

```
FR[fd][0] = CFR[cj]
```

---

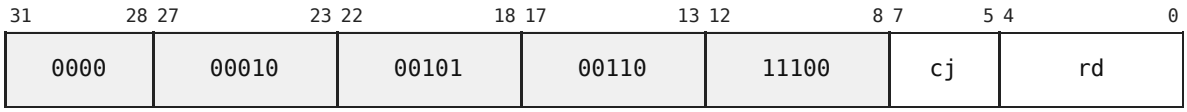
movcf2gr rd, cj

GR[rd] = ZeroExtend(CFR[cj], GRLEN)

条件标志到通用寄存器（Move Condition Flag to GPR） · 2R 型 · LA32 / LA64

将条件标志寄存器 `cj` 零扩展后写入 `rd`。

编码



编码值 `0x0114dc00 | (cj << 5) | rd`

汇编示例

```
movcf2gr $a0, $fcc0      # $a0 = $fcc0
```

语义伪代码

```
GR[rd] = ZeroExtend(CFR[cj], GRLEN)
```

---

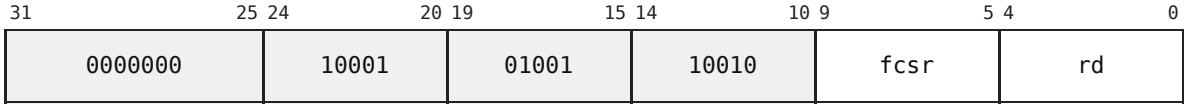
movfcsr2gr rd, fcsr

GR[rd] = FCSR[fcsr]

FCSR 到通用寄存器（Move FCSR to GPR） · 2R 型 · LA32 / LA64

读取编号为 `fcsr` 的浮点控制/状态寄存器，写入 `rd`。

编码



编码值 `0x0114c800 | (fcsr << 5) | rd`

汇编示例

```
movfcsr2gr $a0, $fcsr0      # $a0 = $fcsr0
```

语义伪代码

```
GR[rd] = FCSR[fcsr]
```

---

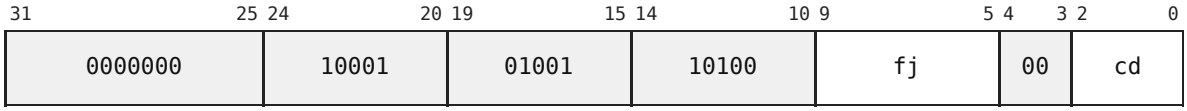
movfr2cf cd, fj

CFR[cd] = FR[fj][0]

浮点寄存器到条件标志（Move FPR to Condition Flag） · 2R 型 · LA32 / LA64

将 `fj` 的最低位写入条件标志寄存器 `cd`。

编码



编码值 `0x0114d000 | (fj << 5) | cd`

汇编示例

```
movfr2cf $fcc0, $f1      # $fcc0 = $f1[0]
```

语义伪代码

```
CFR[cd] = FR[fj][0]
```

---

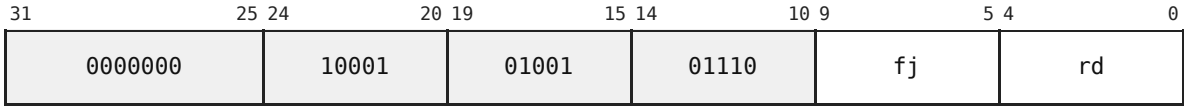
movfr2gr.d rd, fj

GR[rd] = FR[fj]

浮点寄存器到通用寄存器（双精度）（Move FPR to GPR Double） · 2R 型 · LA64

将 `fj` 的 64 位写入 `rd`。

编码



编码值 `0x0114b800 | (fj << 5) | rd`

汇编示例

```
movfr2gr.d $a0, $f1      # $a0 = $f1
```

语义伪代码

```
GR[rd] = FR[fj]
```

---

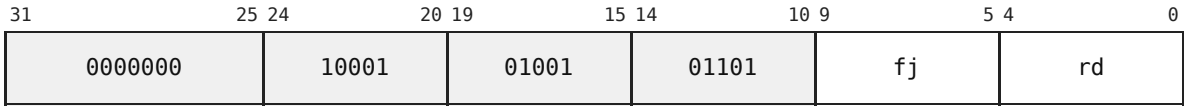
movfr2gr.s `rd, fj`

`GR[rd] = SignExtend(FR[fj][31:0], GRLEN)`

浮点寄存器到通用寄存器（单精度）（Move FPR to GPR Single） · 2R 型 · LA32 / LA64

将 `fj` 低 32 位符号扩展后写入 `rd`。

编码



编码值 `0x0114b400 | (fj << 5) | rd`

汇编示例

```
movfr2gr.s $a0, $f1      # $a0 = (signed)$f1[31:0]
```

语义伪代码

```
GR[rd] = SignExtend(FR[fj][31:0], GRLEN)
```

---

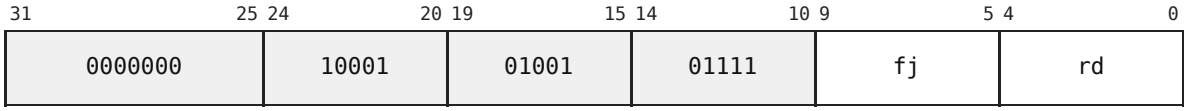
movfrh2gr.s `rd, fj`

`GR[rd] = SignExtend(FR[fj][63:32], GRLEN)`

浮点寄存器高 32 位到通用寄存器（Move FPR High Word to GPR） · 2R 型 · LA32 / LA64

将 `fj` 高 32 位符号扩展后写入 `rd`。

编码



编码值 `0x0114bc00 | (fj << 5) | rd`

汇编示例

```
movfrh2gr.s $a0, $f1      # $a0 = (signed)$f1[63:32]
```

语义伪代码

```
GR[rd] = SignExtend(FR[fj][63:32], GRLEN)
```

---

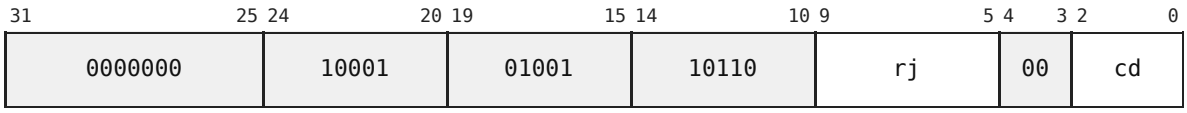
movgr2cf `cd, rj`

`CFR[cd] = GR[rj][0]`

通用寄存器到条件标志（Move GPR to Condition Flag） · 2R 型 · LA32 / LA64

将 `rj` 的最低位写入条件标志寄存器 `cd`。

编码



编码值 `0x0114d800 | (rj << 5) | cd`

汇编示例

```
movgr2cf $fcc0, $a1      # $fcc0 = $a1[0]
```

语义伪代码

```
CFR[cd] = GR[rj][0]
```

---

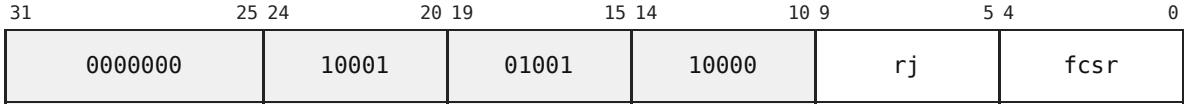
movgr2fcsr `fcsr, rj`

FCSR[fcsr] = GR[rj][31:0]

通用寄存器到 FCSR（Move GPR to FCSR） · 2R 型 · LA32 / LA64

将 `rj` 低 32 位写入编号为 `fcsr` 的浮点控制/状态寄存器。

编码



编码值 `0x0114c000 | (rj << 5) | fcsr`

汇编示例

```
movgr2fcsr $fcsr0, $a1      # $fcsr0 = $a1
```

语义伪代码

```
FCSR[fcsr] = GR[rj][31:0]
```

---

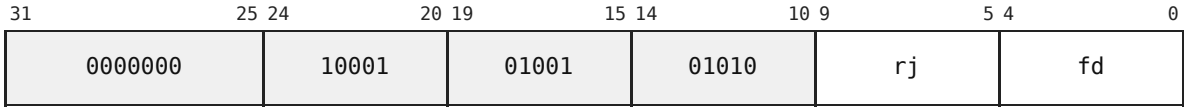
movgr2fr.d `fd, rj`

FR[fd] = GR[rj]

通用寄存器到浮点寄存器（64 位）（Move GPR to FPR Doubleword） · 2R 型 · LA64

将 `rj` 的 64 位写入 `fd`。

编码



编码值 `0x0114a800 | (rj << 5) | fd`

汇编示例

```
movgr2fr.d $f0, $a1      # $f0 = $a1
```

语义伪代码

```
FR[fd] = GR[rj]
```

---

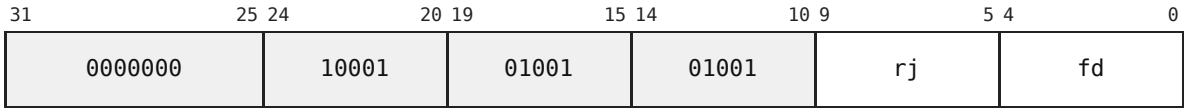
movgr2fr.w `fd, rj`

FR[fd][31:0] = GR[rj][31:0]

通用寄存器到浮点寄存器（低 32 位）（Move GPR to FPR Word） · 2R 型 · LA32 / LA64

将 `rj` 低 32 位写入 `fd` 低 32 位；高 32 位不确定。

编码



编码值 `0x0114a400 | (rj << 5) | fd`

汇编示例

```
movgr2fr.w $f0, $a1      # $f0[31:0] = $a1[31:0]
```

语义伪代码

```
FR[fd][31:0] = GR[rj][31:0]
```

---

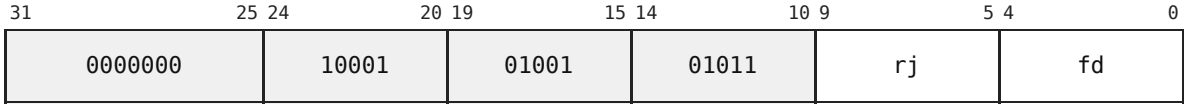
movgr2frh.w `fd, rj`

`FR[fd][63:32] = GR[rj][31:0]`

通用寄存器到浮点寄存器高 32 位 （Move GPR to FPR High Word） · 2R 型 · LA32 / LA64

将 `rj` 低 32 位写入 `fd` 高 32 位，低 32 位保持不变。

编码



编码值 `0x0114ac00 | (rj << 5) | fd`

汇编示例

```
movgr2frh.w $f0, $a1      # $f0[63:32] = $a1[31:0]
```

语义伪代码

```
FR[fd][63:32] = GR[rj][31:0]
```

---

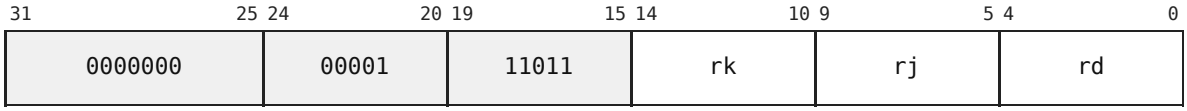
mul.d `rd, rj, rk`

`GR[rd] = (signed(GR[rj][63:0]) * signed(GR[rk][63:0]))[63:0]`

乘法（低半）（64 位） （Multiply Low 64） · 3R 型 · LA64

将 `rj`、`rk` 的低 64 位作为有符号数相乘，取积的低 64 位写入 `rd`。

编码



编码值 `0x001d8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
mul.d $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (signed(GR[rj][63:0]) * signed(GR[rk][63:0]))[63:0]
```

---

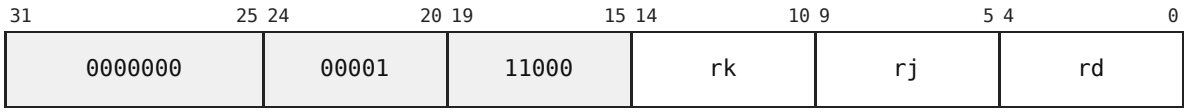
mul.w `rd, rj, rk`

`GR[rd] = (signed(GR[rj][31:0]) * signed(GR[rk][31:0]))[31:0]`

乘法（低半）（32 位） （Multiply Low 32） · 3R 型 · LA32 / LA64

将 `rj`、`rk` 的低 32 位作为有符号数相乘，取积的低 32 位写入 `rd`。

编码



编码值 `0x001c0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
mul.w $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (signed(GR[rj][31:0]) * signed(GR[rk][31:0]))[31:0]
```

---

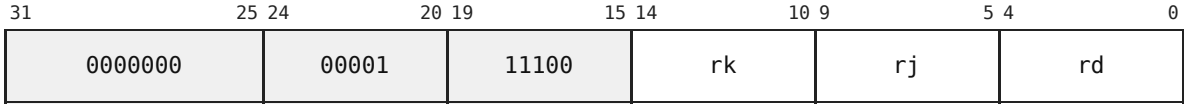
mulh.d rd, rj, rk

GR[rd] = (signed(GR[rj][63:0]) * signed(GR[rk][63:0]))[127:64]

乘法（高半）（64 位） （Multiply High 64） · 3R 型 · LA64

将 rj、rk 的低 64 位作为有符号数相乘，取积的高 64 位写入 rd。

编码



编码值

```
0x001e0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulh.d $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (signed(GR[rj][63:0]) * signed(GR[rk][63:0]))[127:64]
```

---

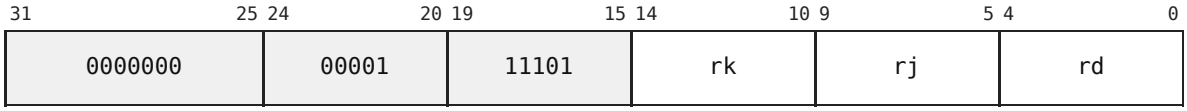
mulh.du rd, rj, rk

GR[rd] = (unsigned(GR[rj][63:0]) * unsigned(GR[rk][63:0]))[127:64]

乘法（高半）（64 位） （Multiply High 64） · 3R 型 · LA64

将 rj、rk 的低 64 位作为无符号数相乘，取积的高 64 位写入 rd。

编码



编码值

```
0x001e8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulh.du $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (unsigned(GR[rj][63:0]) * unsigned(GR[rk][63:0]))[127:64]
```

---

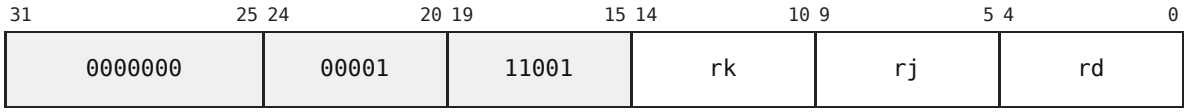
mulh.w rd, rj, rk

GR[rd] = (signed(GR[rj][31:0]) * signed(GR[rk][31:0]))[63:32]

乘法（高半）（32 位） （Multiply High 32） · 3R 型 · LA32 / LA64

将 rj、rk 的低 32 位作为有符号数相乘，取积的高 32 位写入 rd。

编码



编码值

```
0x001c8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulh.w $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (signed(GR[rj][31:0]) * signed(GR[rk][31:0]))[63:32]
```

---

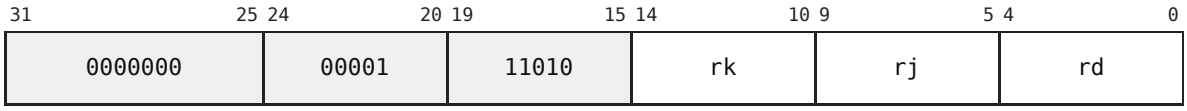
mulh.wu rd, rj, rk

GR[rd] = (unsigned(GR[rj][31:0]) * unsigned(GR[rk][31:0]))[63:32]

乘法（高半）(32 位) （Multiply High 32） · 3R 型 · LA32 / LA64

将 rj、rk 的低 32 位作为无符号数相乘，取积的高 32 位写入 rd。

编码



编码值

```
0x001d0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulh.wu $a0, $a1, $a2      # $a0 = product
```

语义伪代码

```
GR[rd] = (unsigned(GR[rj][31:0]) * unsigned(GR[rk][31:0]))[63:32]
```

---

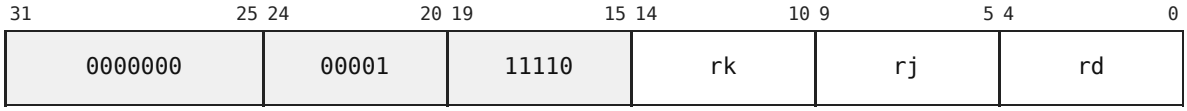
mulw.d.w rd, rj, rk

GR[rd] = signed(GR[rj][31:0]) * signed(GR[rk][31:0])

字乘双字（有符号） （Multiply Word to Doubleword Signed） · 3R 型 · LA64

将 rj、rk 的低 32 位作为有符号数相乘，64 位积写入 rd。

编码



编码值

```
0x001f0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulw.d.w $a0, $a1, $a2      # $a0 = $a1 * $a2
```

语义伪代码

```
GR[rd] = signed(GR[rj][31:0]) * signed(GR[rk][31:0])
```

---

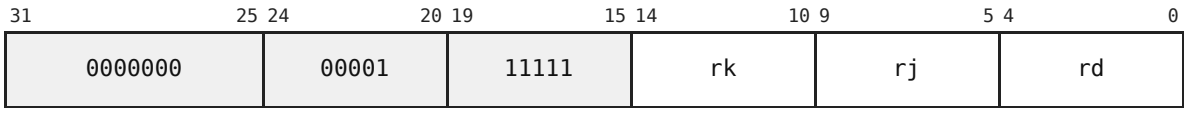
mulw.d.wu rd, rj, rk

GR[rd] = unsigned(GR[rj][31:0]) * unsigned(GR[rk][31:0])

字乘双字（无符号） （Multiply Word to Doubleword Unsigned） · 3R 型 · LA64

将 rj、rk 的低 32 位作为无符号数相乘，64 位积写入 rd。

编码



编码值

```
0x001f8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
mulw.d.wu $a0, $a1, $a2      # $a0 = $a1 * $a2
```

语义伪代码

```
GR[rd] = unsigned(GR[rj][31:0]) * unsigned(GR[rk][31:0])
```

N · 或非指令

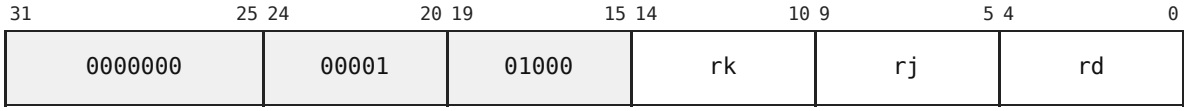
nor rd, rj, rk

GR[rd] = ~(GR[rj] | GR[rk])

按位或非（Bitwise NOR） · 3R 型 · LA32 / LA64

将 `rj`、`rk` 按位或后取反，结果写入 `rd`。

编码



编码值

```
0x00140000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
nor $a0, $a1, $a2      # $a0 = ~($a1 | $a2)
```

语义伪代码

```
GR[rd] = ~(GR[rj] | GR[rk])
```

O · 或运算指令

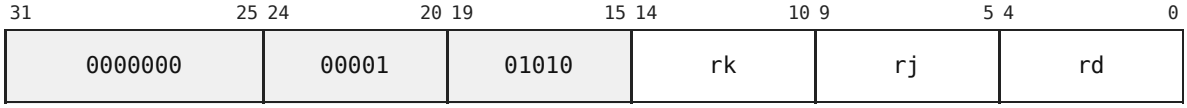
or rd, rj, rk

GR[rd] = GR[rj] | GR[rk]

按位或（Bitwise OR） · 3R 型 · LA32 / LA64

将 `rj`、`rk` 按位或，结果写入 `rd`。

编码



编码值

```
0x00150000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
or $a0, $a1, $a2      # $a0 = $a1 | $a2
```

语义伪代码

```
GR[rd] = GR[rj] | GR[rk]
```

---

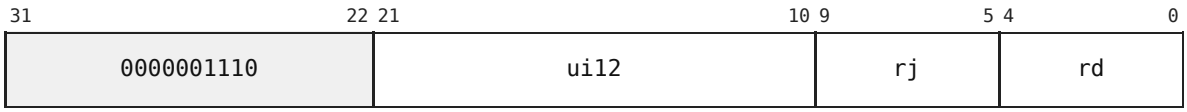
ori rd, rj, ui12

GR[rd] = GR[rj] | ZeroExtend(ui12, GRLEN)

立即数按位或（Bitwise OR Immediate） · 2RI12 型 · LA32 / LA64

将 `rj` 与零扩展的 12 位立即数按位或，结果写入 `rd`。

编码



编码值

```
0x03800000 | ((ui12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
ori $a0, $a1, 0xff      # $a0 = $a1 | 0xff
```

语义伪代码

```
GR[rd] = GR[rj] | ZeroExtend(ui12, GRLEN)
```

---

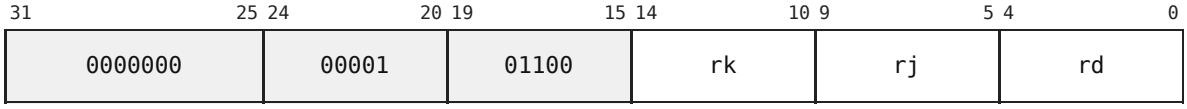
orn `rd, rj, rk`

`GR[rd] = GR[rj] | ~GR[rk]`

按位或非 （Bitwise OR NOT） · 3R 型 · LA32 / LA64

将 `rk` 取反后与 `rj` 按位或，结果写入 `rd`。

编码



编码值

```
0x00160000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
orn $a0, $a1, $a2      # $a0 = $a1 | ~$a2
```

语义伪代码

```
GR[rd] = GR[rj] | (~GR[rk])
```

P · PC 相对与预取指令

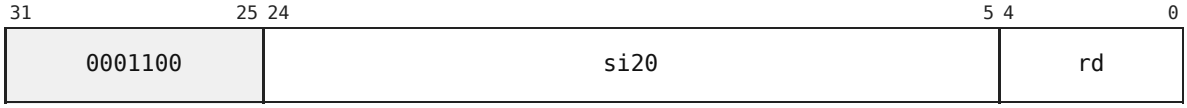
pcaddi `rd, si20`

`GR[rd] = PC + SignExtend({si20, 2'b0}, GRLEN)`

PC 相对加 4 字节偏移 （PC Add Immediate） · 1RI20 型 · LA32 / LA64

以 `PC` 为基准加上 20 位立即数 `si20` 左移 2 位后的值，结果写入 `rd`。

编码



编码值

```
0x18000000 | ((si20 & 0xfffff) << 5) | rd
```

汇编示例

```
pcaddi $a0, 1      # $a0 = PC + (1 << 2)
```

语义伪代码

```
GR[rd] = PC + SignExtend({si20, 2'b0}, GRLEN)
```

---

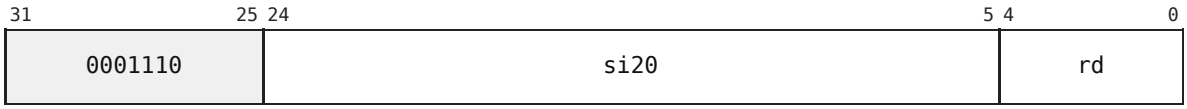
pcaddu12i `rd, si20`

`GR[rd] = PC + SignExtend({si20, 12'b0}, GRLEN)`

PC 相对加 12 位页偏移 （PC Add Upper 12-bit Immediate） · 1RI20 型 · LA32 / LA64

以 `PC` 为基准加上 20 位立即数 `si20` 左移 12 位后的值，结果写入 `rd`。

编码



编码值

```
0x1c000000 | ((si20 & 0xfffff) << 5) | rd
```

汇编示例

```
pcaddu12i $a0, 1      # $a0 = PC + (1 << 12)
```

语义伪代码

```
GR[rd] = PC + SignExtend({si20, 12'b0}, GRLEN)
```

---

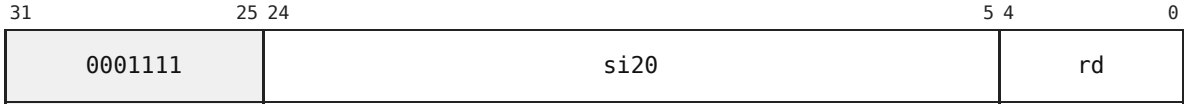
pcaddu18i `rd, si20`

`GR[rd] = PC + SignExtend({si20, 18'b0}, GRLEN)`

PC 相对加 18 位偏移 （PC Add Upper 18-bit Immediate） · 1RI20 型 · LA64

以 `PC` 为基准加上 20 位立即数 `si20` 左移 18 位后的值，结果写入 `rd`。

编码



编码值 `0x1e000000 | ((si20 & 0xfffff) << 5) | rd`

汇编示例

```
pcaddu18i $a0, 1      # $a0 = PC + (1 << 18)
```

语义伪代码

```
GR[rd] = PC + SignExtend({si20, 18'b0}, GRLEN)
```

---

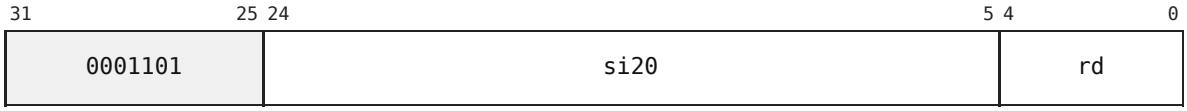
pcalau12i `rd, si20`

`tmp = PC + SignExtend({si20, 12'b0}, GRLEN); GR[rd] = {tmp[GRLEN-1:12], 12'b0}`

PC 页地址加 12 位偏移 （PC Align Add Upper 12-bit Immediate） · 1RI20 型 · LA32 / LA64

以 `PC` 为基准加上 20 位立即数 `si20` 左移 12 位后的值，结果写入 `rd`。

编码



编码值 `0x1a000000 | ((si20 & 0xfffff) << 5) | rd`

汇编示例

```
pcalau12i $a0, 1      # $a0 = PC + (1 << 12)
```

语义伪代码

```
tmp = PC + SignExtend({si20, 12'b0}, GRLEN)
GR[rd] = {tmp[GRLEN-1:12], 12'b0}
```

---

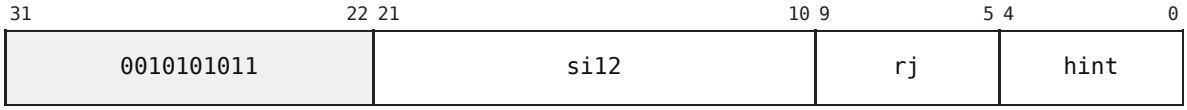
preld `hint, rj, si12`

`Prefetch(GR[rj] + si12, hint)`

预取（立即数偏移） （Prefetch with Immediate） · 2RI12 型 · LA32 / LA64

按 `hint` 对地址 `rj + si12` 发起预取。

编码



编码值 `0x2ac00000 | ((si12 & 0xfff) << 10) | (rj << 5) | ((hint & 0x1f) << 0)`

汇编示例

```
preld 0, $a0, 0      # 预取 [$a0]
```

语义伪代码

```
Prefetch(GR[rj] + SignExtend(si12, GRLEN), hint)
```

---

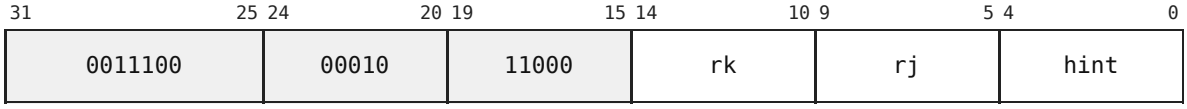
preldx `hint, rj, rk`

`Prefetch(GR[rj] + GR[rk], hint)`

预取（索引）（Prefetch Indexed） · 3R 型 · LA32 / LA64

按 `hint` 对地址 `rj + rk` 发起预取。

编码



编码值

```
0x382c0000 | (rk << 10) | (rj << 5) | ((hint & 0x1f) << 0)
```

汇编示例

```
preldx 0, $a0, $a1      # 预取 [$a0+$a1]
```

语义伪代码

```
Prefetch(GR[rj] + GR[rk], hint)
```

R · 字节反转、计时与循环移位指令

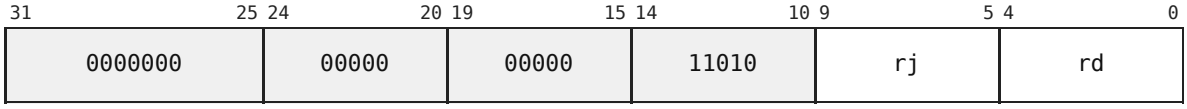
rdtime.d `rd, rj`

`GR[rd] = StableCounter; GR[rj] = CounterID`

读取计时器 64 位（Read Time Doubleword） · 2R 型 · LA64

读取稳定计数器 64 位到 `rd`，并将 Counter ID 写入 `rj`。

编码



编码值

```
0x00006800 | (rj << 5) | rd
```

汇编示例

```
rdtime.d $a0, $a1      # $a0 = counter
```

语义伪代码

```
GR[rd] = StableCounter
GR[rj] = CounterID
```

---

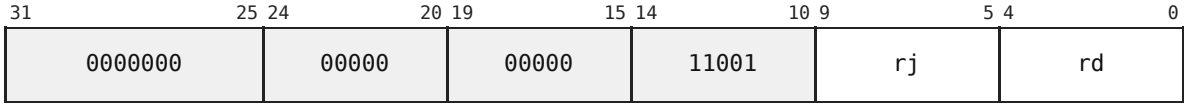
rdtimeh.w `rd, rj`

`GR[rd] = SignExtend(StableCounter[63:32], GRLEN); GR[rj] = CounterID`

读取计时器高 32 位（Read Time High Word） · 2R 型 · LA32 / LA64

读取稳定计数器高 32 位（符号扩展）到 `rd`，并将 Counter ID 写入 `rj`。

编码



编码值

```
0x00006400 | (rj << 5) | rd
```

汇编示例

```
rdtimeh.w $a0, $a1      # $a0 = counter[63:32]
```

语义伪代码

```
GR[rd] = SignExtend(StableCounter[63:32], GRLEN)
GR[rj] = CounterID
```

---

rdtimel.w `rd, rj`

`GR[rd] = SignExtend(StableCounter[31:0], GRLEN); GR[rj] = CounterID`

读取计时器低 32 位 （Read Time Low Word） · 2R 型 · LA32 / LA64

读取稳定计数器低 32 位（符号扩展）到 `rd`，并将 Counter ID 写入 `rj`。

编码

31–25	24–20	19–15	14–10	9–5	4–0
0000000	00000	00000	11000	rj	rd

编码值

```
0x00006000 | (rj << 5) | rd
```

汇编示例

```
rdtimel.w $a0, $a1      # $a0 = counter[31:0]
```

语义伪代码

```
GR[rd] = SignExtend(StableCounter[31:0], GRLEN)
GR[rj] = CounterID
```

---

revb.2h `rd, rj`

`GR[rd] = Reverse32(GR[rj])`

2 个半字内字节反转 （Reverse Bytes in 2 Halfwords） · 2R 型 · LA32 / LA64

按 32 位宽度对 `rj` 进行2 个半字内字节反转，结果写入 `rd`。

编码

31–25	24–20	19–15	14–10	9–5	4–0
0000000	00000	00000	01100	rj	rd

编码值

```
0x00003000 | (rj << 5) | rd
```

汇编示例

```
revb.2h $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse32(GR[rj])
```

---

revb.2w `rd, rj`

`GR[rd] = Reverse64(GR[rj])`

2 个字内字节反转 （Reverse Bytes in 2 Words） · 2R 型 · LA64

按 64 位宽度对 `rj` 进行2 个字内字节反转，结果写入 `rd`。

编码

31–25	24–20	19–15	14–10	9–5	4–0
0000000	00000	00000	01110	rj	rd

编码值

```
0x00003800 | (rj << 5) | rd
```

汇编示例

```
revb.2w $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse64(GR[rj])
```

---

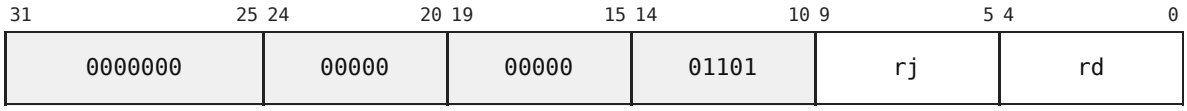
revb.4h rd, rj

GR[rd] = Reverse64(GR[rj])

4 个半字内字节反转（Reverse Bytes in 4 Halfwords） · 2R 型 · LA64

按 64 位宽度对 `rj` 进行4 个半字内字节反转，结果写入 `rd`。

编码



编码值

```
0x00003400 | (rj << 5) | rd
```

汇编示例

```
revb.4h $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse64(GR[rj])
```

---

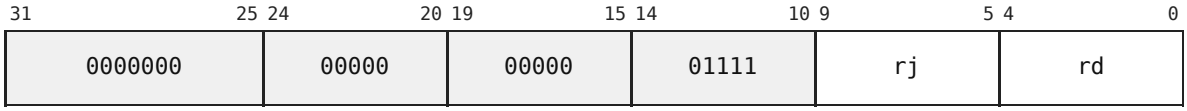
revb.d rd, rj

GR[rd] = Reverse64(GR[rj])

双字内字节反转（Reverse Bytes in Doubleword） · 2R 型 · LA64

按 64 位宽度对 `rj` 进行双字内字节反转，结果写入 `rd`。

编码



编码值

```
0x00003c00 | (rj << 5) | rd
```

汇编示例

```
revb.d $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse64(GR[rj])
```

---

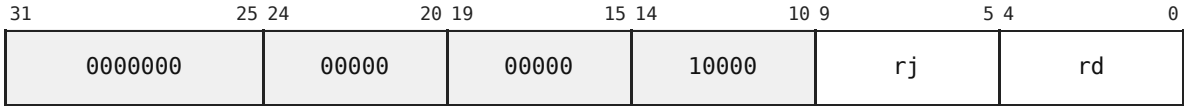
revh.2w rd, rj

GR[rd] = Reverse64(GR[rj])

2 个字内半字反转（Reverse Halfwords in 2 Words） · 2R 型 · LA64

按 64 位宽度对 `rj` 进行2 个字内半字反转，结果写入 `rd`。

编码



编码值

```
0x00004000 | (rj << 5) | rd
```

汇编示例

```
revh.2w $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse64(GR[rj])
```

---

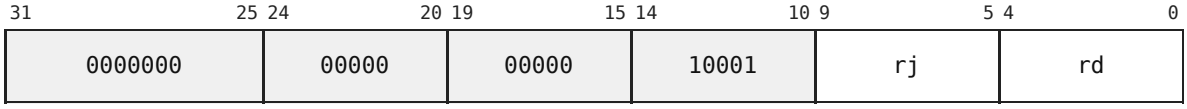
revh.d `rd, rj`

`GR[rd] = Reverse64(GR[rj])`

双字内半字反转 （Reverse Halfwords in Doubleword） · 2R 型 · LA64

按 64 位宽度对 `rj` 进行双字内半字反转，结果写入 `rd`。

编码



编码值

```
0x00004400 | (rj << 5) | rd
```

汇编示例

```
revh.d $a0, $a1      # 字节/半字反转
```

语义伪代码

```
GR[rd] = Reverse64(GR[rj])
```

---

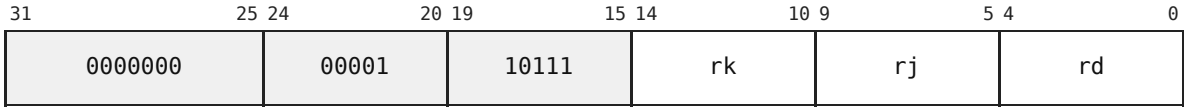
rotr.d `rd, rj, rk`

`GR[rd] = RotateRight(GR[rj][63:0], GR[rk][4:0])`

循环右移（双字） （Rotate Right Doubleword） · 3R 型 · LA64

将 `rj` 低 64 位按 `rk` 低 5/6 位循环右移，结果写入 `rd`。

编码



编码值

```
0x001b8000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
rotr.d $a0, $a1, $a2      # $a0 = rotr($a1, $a2)
```

语义伪代码

```
GR[rd] = RotateRight(GR[rj][63:0], GR[rk])
```

---

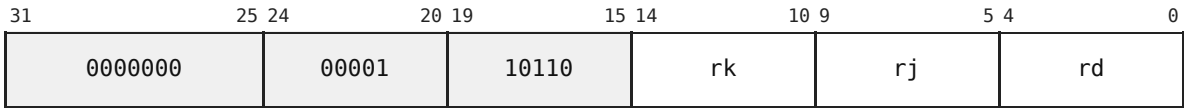
rotr.w `rd, rj, rk`

`GR[rd] = RotateRight(GR[rj][31:0], GR[rk][4:0])`

循环右移（字） （Rotate Right Word） · 3R 型 · LA32 / LA64

将 `rj` 低 32 位按 `rk` 低 5/6 位循环右移，结果写入 `rd`。

编码



编码值

```
0x001b0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
rotr.w $a0, $a1, $a2      # $a0 = rotr($a1, $a2)
```

语义伪代码

```
GR[rd] = RotateRight(GR[rj][31:0], GR[rk])
```

---

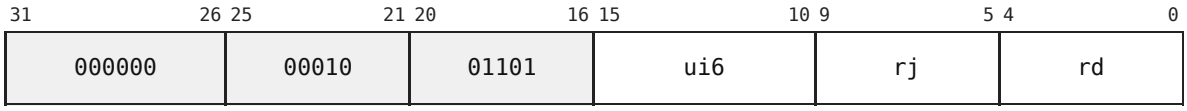
rotri.d `rd, rj, ui6`

`GR[rd] = RotateRight(GR[rj][63:0], ui6)`

立即数循环右移（双字）（Rotate Right Immediate Doubleword） · 2RI5/6 型 · LA64

将 `rj` 低 64 位循环右移立即数位，结果写入 `rd`。

编码



编码值 `0x004d0000 | ((ui6 & 0x3f) << 10) | (rj << 5) | rd`

汇编示例

```
rotri.d $a0, $a1, 1      # $a0 = rotr($a1, 1)
```

语义伪代码

```
GR[rd] = RotateRight(GR[rj][63:0], ui6)
```

---

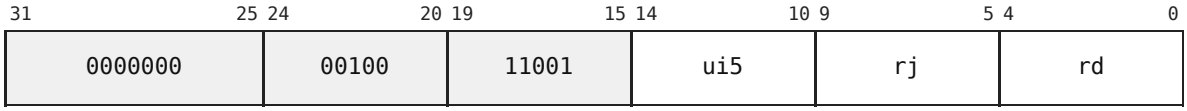
rotri.w `rd, rj, ui5`

`GR[rd] = RotateRight(GR[rj][31:0], ui5)`

立即数循环右移（字）（Rotate Right Immediate Word） · 2RI5/6 型 · LA32 / LA64

将 `rj` 低 32 位循环右移立即数位，结果写入 `rd`。

编码



编码值 `0x004c8000 | ((ui5 & 0x1f) << 10) | (rj << 5) | rd`

汇编示例

```
rotri.w $a0, $a1, 1      # $a0 = rotr($a1, 1)
```

语义伪代码

```
GR[rd] = RotateRight(GR[rj][31:0], ui5)
```

S · 减法、移位、存储与系统指令

本章中的存储地址为 `GR[rj] + si12` 或 `GR[rj] + GR[rk]`；`offs` 如无特别说明均为汇编中的字节偏移。

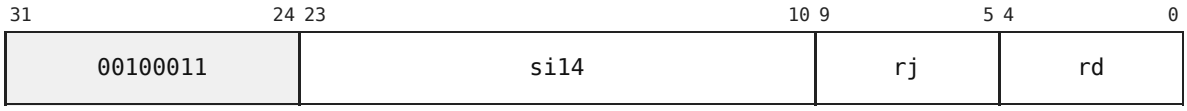
sc.d rd, rj, si14

`if (LLbit) [GR[rj]+si14] = GR[rd]; GR[rd] = success`

条件存储（双字）（Store Conditional Doubleword） · 2RI14 型 · LA64

LLbit 为 1 时将 `rd` 64 位写入 `rj + si14`，并把成功标志写入 `rd`。

编码



编码值

```
0x23000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
sc.d $a0, $a1, 0      # if (LLbit) [$a1] = $a0; $a0 = success
```

语义伪代码

```
if (LLbit) {
    Memory[GR[rj] + si14] = GR[rd]
    GR[rd] = 1
} else {
    GR[rd] = 0
}
```

---

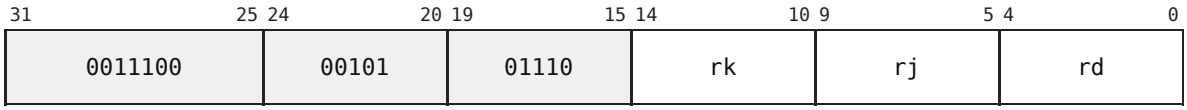
sc.q rd, rk, rj

`if (LLbit) [GR[rj]] = {GR[rk], GR[rd]}; GR[rd] = success`

条件存储（128 位）（Store Conditional Quadword） · 3R 型 · LA64

LLbit 为 1 时把 `{rk, rd}` 组成的 128 位数据写入 `rj` 地址，并把成功标志写入 `rd`。

编码



编码值

```
0x38570000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
sc.q $a0, $a1, $a2      # if (LLbit) [$a2] = {$a1,$a0}; $a0 = success
```

语义伪代码

```
if (LLbit) {
    Memory[GR[rj]] = {GR[rk], GR[rd]}
    GR[rd] = 1
} else {
    GR[rd] = 0
}
```

---

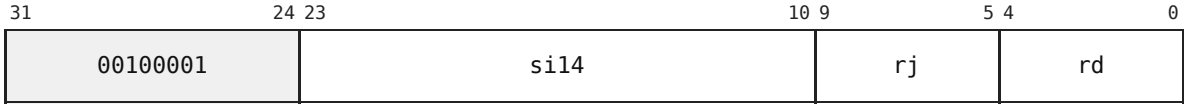
sc.w rd, rj, si14

`if (LLbit) [GR[rj]+si14] = GR[rd][31:0]; GR[rd] = success`

条件存储（字）（Store Conditional Word） · 2RI14 型 · LA32 / LA64

LLbit 为 1 时将 `rd` 低 32 位写入 `rj + si14`，并把成功标志写入 `rd`。

编码



编码值

```
0x21000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
sc.w $a0, $a1, 0      # if (LLbit) [$a1] = $a0; $a0 = success
```

语义伪代码

```
if (LLbit) {
    Memory[GR[rj] + si14] = GR[rd][31:0]
    GR[rd] = 1
} else {
    GR[rd] = 0
}
```

---

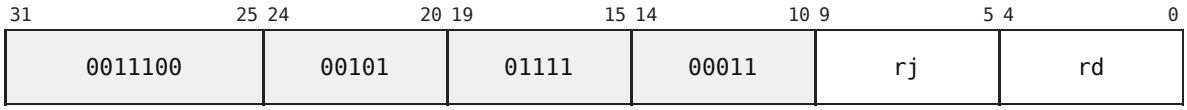
screl.d rd, rj

`ReleaseBarrier(); if (LLbit) [GR[rj]] = GR[rd]; GR[rd] = success`

释放条件存储（双字）（Store Conditional Release Doubleword） · 2R 型 · LA64

带释放语义的条件存储：先执行释放屏障，再按 LLbit 写入并返回成功标志。

编码



编码值

```
0x38578c00 | (rj << 5) | rd
```

汇编示例

```
screl.d $a0, $a1      # release; if (LLbit) [$a1] = $a0; $a0 = success
```

语义伪代码

```
ReleaseBarrier()
if (LLbit) {
    Memory[GR[rj]] = GR[rd]
    GR[rd] = 1
} else {
    GR[rd] = 0
}
```

---

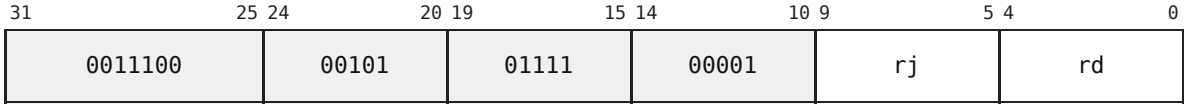
screl.w rd, rj

ReleaseBarrier(); if (LLbit) [GR[rj]] = GR[rd][31:0]; GR[rd] = success

释放条件存储（字）（Store Conditional Release Word） · 2R 型 · LA32 / LA64

带释放语义的条件存储：先执行释放屏障，再按 LLbit 写入并返回成功标志。

编码



编码值 `0x38578400 | (rj << 5) | rd`

汇编示例

```
screl.w $a0, $a1      # release; if (LLbit) [$a1] = $a0; $a0 = success
```

语义伪代码

```
ReleaseBarrier()
if (LLbit) {
    Memory[GR[rj]] = GR[rd][31:0]
    GR[rd] = 1
} else {
    GR[rd] = 0
}
```

---

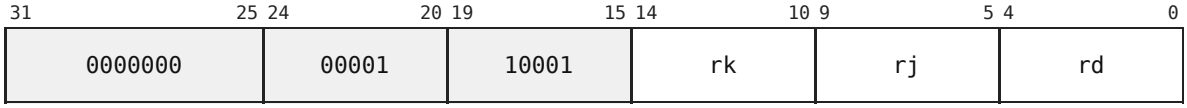
sll.d rd, rj, rk

GR[rd] = ShiftLeft(GR[rj][63:0], GR[rk])

逻辑左移（双字）（Shift Left Logical Doubleword） · 3R 型 · LA64

将 `rj` 低 64 位按 `rk` 低位进行左移（双字），结果写入 `rd`。

编码



编码值 `0x00188000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
sll.d $a0, $a1, $a2      # $a0 = sll($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftLeft(GR[rj][63:0], GR[rk])
```

---

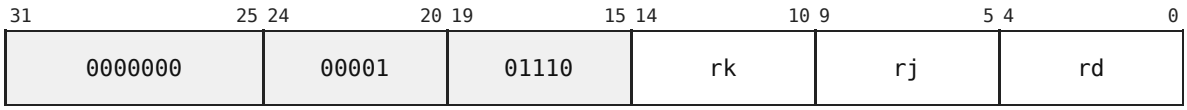
sll.w rd, rj, rk

GR[rd] = ShiftLeft(GR[rj][31:0], GR[rk])

逻辑左移（字）（Shift Left Logical Word） · 3R 型 · LA32 / LA64

将 `rj` 低 32 位按 `rk` 低位进行左移（字），结果写入 `rd`。

编码



编码值 `0x00170000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
sll.w $a0, $a1, $a2      # $a0 = sll($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftLeft(GR[rj][31:0], GR[rk])
```

---

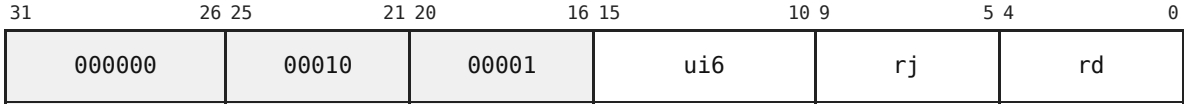
slli.d rd, rj, ui6

GR[rd] = ShiftLeft(GR[rj][63:0], ui6)

立即数逻辑左移（双字） （Shift Left Logical Immediate Doubleword） · 2RI5/6 型 · LA64

将 `rj` 低 64 位按立即数 `ui6` 辑左移（双字），结果写入 `rd`。

编码



编码值

```
0x00410000 | ((ui6 & 0x3f) << 10) | (rj << 5) | rd
```

汇编示例

```
slli.d $a0, $a1, 1      # $a0 = sll($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftLeft(GR[rj][63:0], ui6)
```

---

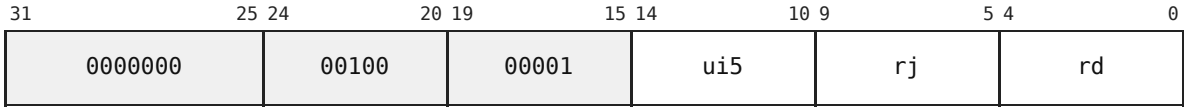
slli.w rd, rj, ui5

GR[rd] = ShiftLeft(GR[rj][31:0], ui5)

立即数逻辑左移（字） （Shift Left Logical Immediate Word） · 2RI5/6 型 · LA32 / LA64

将 `rj` 低 32 位按立即数 `ui5` 辑左移（字），结果写入 `rd`。

编码



编码值

```
0x00408000 | ((ui5 & 0x1f) << 10) | (rj << 5) | rd
```

汇编示例

```
slli.w $a0, $a1, 1      # $a0 = sll($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftLeft(GR[rj][31:0], ui5)
```

---

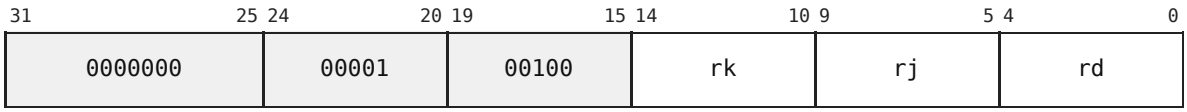
slt rd, rj, rk

GR[rd] = (signed(GR[rj]) < signed(GR[rk])) ? 1 : 0

有符号小于置位 （Set on Less Than） · 3R 型 · LA32 / LA64

有符号比较 `rj < rk`，真则 `rd=1`，否则 `rd=0`。

编码



编码值

```
0x00120000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
slt $a0, $a1, $a2      # $a0 = ($a1 < $a2)
```

语义伪代码

```
GR[rd] = (signed(GR[rj]) < signed(GR[rk])) ? 1 : 0
```

---

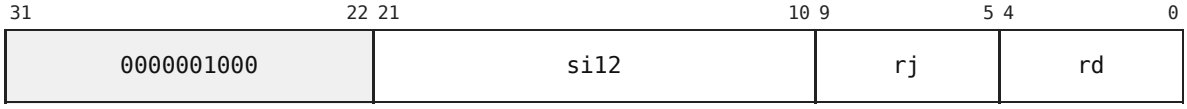
slti `rd, rj, si12`

`GR[rd] = (signed(GR[rj]) < signed(si12)) ? 1 : 0`

立即数有符号小于置位（Set on Less Than Immediate） · 2RI12 型 · LA32 / LA64

将 `rj` 与符号扩展的 `si12` 有符号比较，真则 `rd=1`。

编码



编码值

```
0x02000000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
slti $a0, $a1, 0      # $a0 = ($a1 < 0)
```

语义伪代码

```
GR[rd] = (signed(GR[rj]) < signed(si12)) ? 1 : 0
```

---

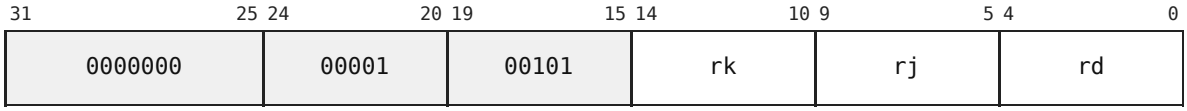
sltu `rd, rj, rk`

`GR[rd] = (unsigned(GR[rj]) < unsigned(GR[rk])) ? 1 : 0`

无符号小于置位（Set on Less Than Unsigned） · 3R 型 · LA32 / LA64

无符号比较 `rj < rk`，真则 `rd=1`，否则 `rd=0`。

编码



编码值

```
0x00128000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
sltu $a0, $a1, $a2      # $a0 = ($a1 <u $a2)
```

语义伪代码

```
GR[rd] = (unsigned(GR[rj]) < unsigned(GR[rk])) ? 1 : 0
```

---

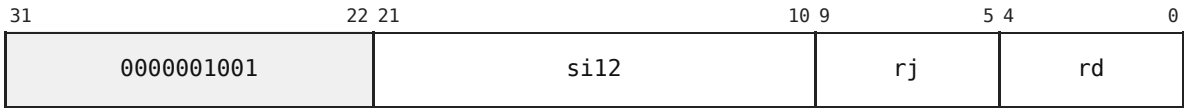
sltui `rd, rj, si12`

`GR[rd] = (unsigned(GR[rj]) < unsigned(si12)) ? 1 : 0`

立即数无符号小于置位（Set on Less Than Unsigned Immediate） · 2RI12 型 · LA32 / LA64

将 `rj` 与符号扩展的 `si12` 无符号比较，真则 `rd=1`。

编码



编码值

```
0x02400000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
sltui $a0, $a1, 0      # $a0 = ($a1 <u 0)
```

语义伪代码

```
GR[rd] = (unsigned(GR[rj]) < unsigned(si12)) ? 1 : 0
```

---

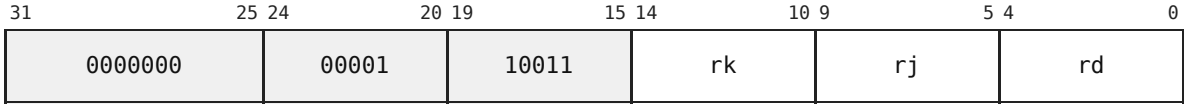
sra.d rd, rj, rk

GR[rd] = ShiftRightArithmetic(GR[rj][63:0], GR[rk])

算术右移（双字） （Shift Right Arithmetic Doubleword） · 3R 型 · LA64

将 rj 低 64 位按 rk 低位进行右移（双字），结果写入 rd。

编码



编码值

```
0x00198000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
sra.d $a0, $a1, $a2      # $a0 = sra($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftRightArithmetic(GR[rj][63:0], GR[rk])
```

---

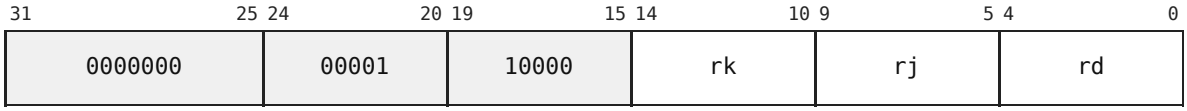
sra.w rd, rj, rk

GR[rd] = ShiftRightArithmetic(GR[rj][31:0], GR[rk])

算术右移（字） （Shift Right Arithmetic Word） · 3R 型 · LA32 / LA64

将 rj 低 32 位按 rk 低位进行右移（字），结果写入 rd。

编码



编码值

```
0x00180000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
sra.w $a0, $a1, $a2      # $a0 = sra($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftRightArithmetic(GR[rj][31:0], GR[rk])
```

---

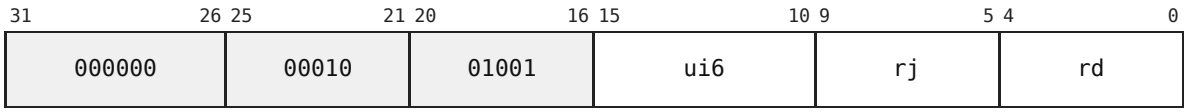
srai.d rd, rj, ui6

GR[rd] = ShiftRightArithmetic(GR[rj][63:0], ui6)

立即数算术右移（双字） （Shift Right Arithmetic Immediate Doubleword） · 2RI5/6 型 · LA64

将 rj 低 64 位按立即数 ui6 术右移（双字），结果写入 rd。

编码



编码值

```
0x00490000 | ((ui6 & 0x3f) << 10) | (rj << 5) | rd
```

汇编示例

```
srai.d $a0, $a1, 1      # $a0 = sra($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftRightArithmetic(GR[rj][63:0], ui6)
```

---

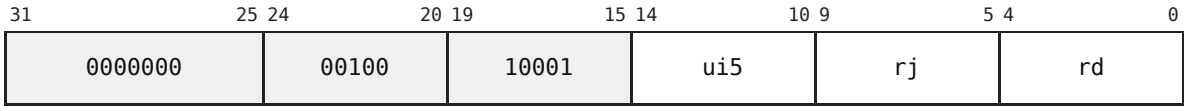
srai.w `rd, rj, ui5`

`GR[rd] = ShiftRightArithmetic(GR[rj][31:0], ui5)`

立即数算术右移（字） （Shift Right Arithmetic Immediate Word） · 2RI5/6 型 · LA32 / LA64

将 `rj` 低 32 位按立即数 `ui5` 术右移（字），结果写入 `rd`。

编码



编码值

```
0x00488000 | ((ui5 & 0x1f) << 10) | (rj << 5) | rd
```

汇编示例

```
srai.w $a0, $a1, 1      # $a0 = sra($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftRightArithmetic(GR[rj][31:0], ui5)
```

---

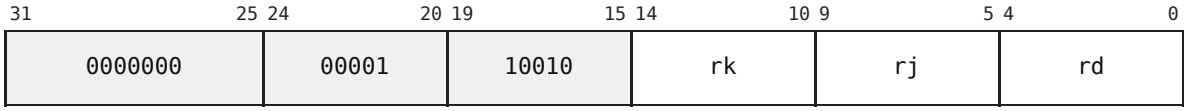
srl.d `rd, rj, rk`

`GR[rd] = ShiftRightLogical(GR[rj][63:0], GR[rk])`

逻辑右移（双字） （Shift Right Logical Doubleword） · 3R 型 · LA64

将 `rj` 低 64 位按 `rk` 低位进行右移（双字），结果写入 `rd`。

编码



编码值

```
0x00190000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
srl.d $a0, $a1, $a2      # $a0 = srl($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftRightLogical(GR[rj][63:0], GR[rk])
```

---

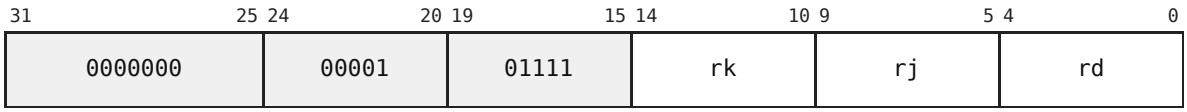
srl.w `rd, rj, rk`

`GR[rd] = ShiftRightLogical(GR[rj][31:0], GR[rk])`

逻辑右移（字） （Shift Right Logical Word） · 3R 型 · LA32 / LA64

将 `rj` 低 32 位按 `rk` 低位进行右移（字），结果写入 `rd`。

编码



编码值

```
0x00178000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
srl.w $a0, $a1, $a2      # $a0 = srl($a1, $a2)
```

语义伪代码

```
GR[rd] = ShiftRightLogical(GR[rj][31:0], GR[rk])
```

---

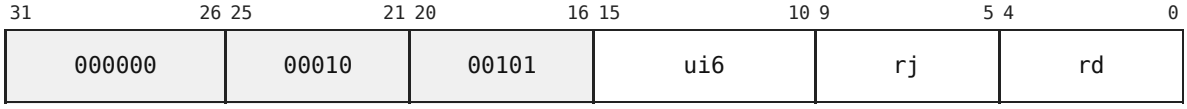
srli.d rd, rj, ui6

GR[rd] = ShiftRightLogical(GR[rj][63:0], ui6)

立即数逻辑右移（双字） （Shift Right Logical Immediate Doubleword） · 2RI5/6 型 · LA64

将 `rj` 低 64 位按立即数 `ui6` 辑右移（双字），结果写入 `rd`。

编码



编码值

```
0x00450000 | ((ui6 & 0x3f) << 10) | (rj << 5) | rd
```

汇编示例

```
srli.d $a0, $a1, 1      # $a0 = srl($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftRightLogical(GR[rj][63:0], ui6)
```

---

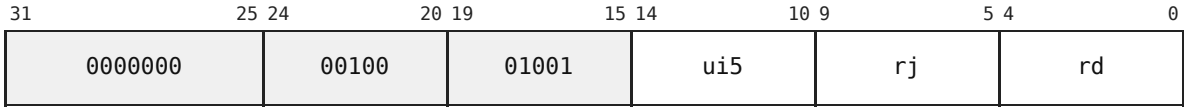
srli.w rd, rj, ui5

GR[rd] = ShiftRightLogical(GR[rj][31:0], ui5)

立即数逻辑右移（字） （Shift Right Logical Immediate Word） · 2RI5/6 型 · LA32 / LA64

将 `rj` 低 32 位按立即数 `ui5` 辑右移（字），结果写入 `rd`。

编码



编码值

```
0x00448000 | ((ui5 & 0x1f) << 10) | (rj << 5) | rd
```

汇编示例

```
srli.w $a0, $a1, 1      # $a0 = srl($a1, 1)
```

语义伪代码

```
GR[rd] = ShiftRightLogical(GR[rj][31:0], ui5)
```

---

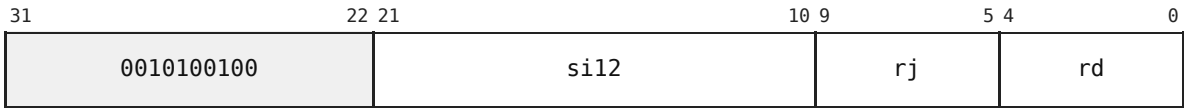
st.b rd, rj, si12

Memory[GR[rj]+si12, 8] = GR[rd][7:0]

存储（字节） （Store Byte） · 2RI12 型 · LA32 / LA64

将 `rd` 的低 8 位存储到 `rj + si12`。

编码



编码值

```
0x29000000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
st.b $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si12, 8] = GR[rd][7:0]
```

---

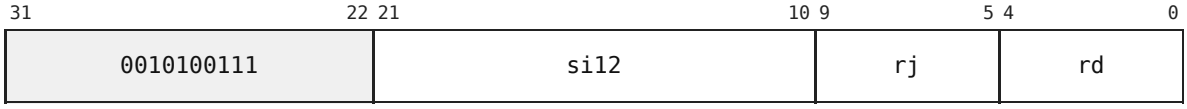
st.d rd, rj, si12

Memory[GR[rj]+si12, 64] = GR[rd][63:0]

存储（双字）（Store Doubleword） · 2RI12 型 · LA64

将 `rd` 的低 64 位存储到 `rj + si12`。

编码



编码值

```
0x29c00000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
st.d $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si12, 64] = GR[rd][63:0]
```

---

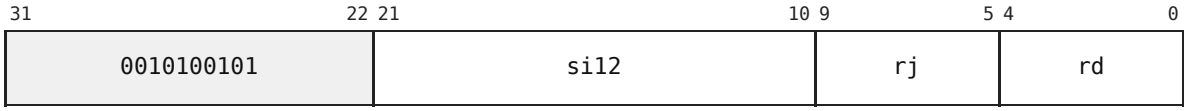
st.h rd, rj, si12

Memory[GR[rj]+si12, 16] = GR[rd][15:0]

存储（半字）（Store Halfword） · 2RI12 型 · LA32 / LA64

将 `rd` 的低 16 位存储到 `rj + si12`。

编码



编码值

```
0x29400000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
st.h $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si12, 16] = GR[rd][15:0]
```

---

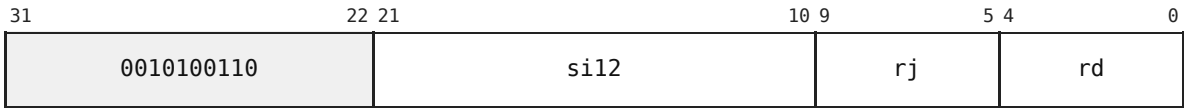
st.w rd, rj, si12

Memory[GR[rj]+si12, 32] = GR[rd][31:0]

存储（字）（Store Word） · 2RI12 型 · LA32 / LA64

将 `rd` 的低 32 位存储到 `rj + si12`。

编码



编码值

```
0x29800000 | ((si12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
st.w $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si12, 32] = GR[rd][31:0]
```

---

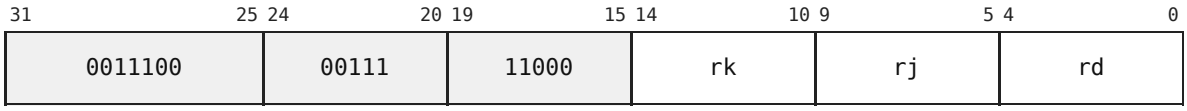
stgt.b rd, rj, rk

`if (GR[rj] > GR[rk]) Memory[GR[rj], 8] = GR[rd][7:0] else raise BCE`

存储（大于检查，字节）（Store Greater Than Byte） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时存储字节，否则触发边界检查异常。

编码



编码值 `0x387c0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stgt.b $a0, $a1, $a2      # if ($a1 > $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] > GR[rk])
    Memory[GR[rj], 8] = GR[rd][7:0]
else
    RaiseException(BCE)
```

---

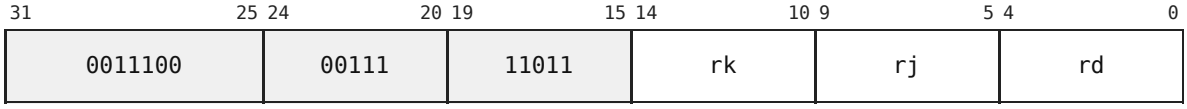
stgt.d rd, rj, rk

`if (GR[rj] > GR[rk]) Memory[GR[rj], 64] = GR[rd][63:0] else raise BCE`

存储（大于检查，双字）（Store Greater Than Doubleword） · 3R 型 · LA64

检查 `rj > rk`，满足时存储双字，否则触发边界检查异常。

编码



编码值 `0x387d8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stgt.d $a0, $a1, $a2      # if ($a1 > $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] > GR[rk])
    Memory[GR[rj], 64] = GR[rd][63:0]
else
    RaiseException(BCE)
```

---

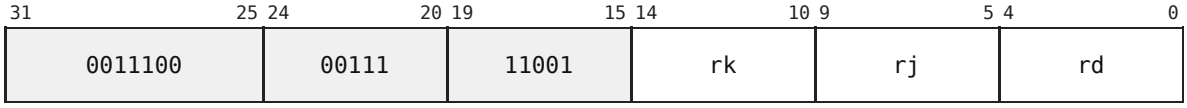
stgt.h rd, rj, rk

`if (GR[rj] > GR[rk]) Memory[GR[rj], 16] = GR[rd][15:0] else raise BCE`

存储（大于检查，半字）（Store Greater Than Halfword） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时存储半字，否则触发边界检查异常。

编码



编码值 `0x387c8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stgt.h $a0, $a1, $a2      # if ($a1 > $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] > GR[rk])
    Memory[GR[rj], 16] = GR[rd][15:0]
else
    RaiseException(BCE)
```

---

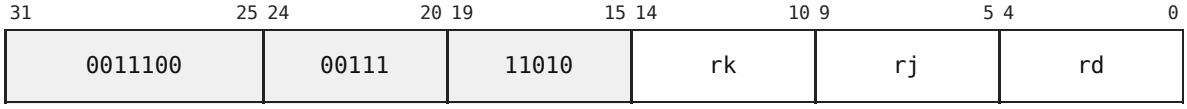
stgt.w rd, rj, rk

`if (GR[rj] > GR[rk]) Memory[GR[rj], 32] = GR[rd][31:0] else raise BCE`

存储（大于检查，字）（Store Greater Than Word） · 3R 型 · LA32 / LA64

检查 `rj > rk`，满足时存储字，否则触发边界检查异常。

编码



编码值 `0x387d0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stgt.w $a0, $a1, $a2      # if ($a1 > $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] > GR[rk])
    Memory[GR[rj], 32] = GR[rd][31:0]
else
    RaiseException(BCE)
```

---

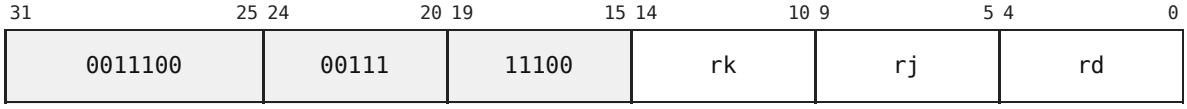
stle.b rd, rj, rk

`if (GR[rj] <= GR[rk]) Memory[GR[rj], 8] = GR[rd][7:0] else raise BCE`

存储（小于等于检查，字节）（Store Less Equal Byte） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时存储字节，否则触发边界检查异常。

编码



编码值 `0x387e0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stle.b $a0, $a1, $a2      # if ($a1 <= $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    Memory[GR[rj], 8] = GR[rd][7:0]
else
    RaiseException(BCE)
```

---

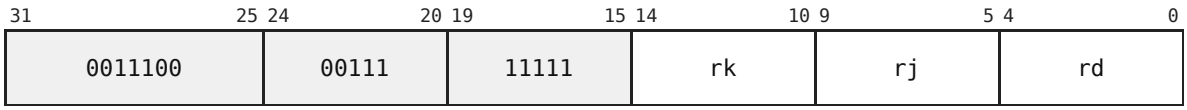
stle.d rd, rj, rk

`if (GR[rj] <= GR[rk]) Memory[GR[rj], 64] = GR[rd][63:0] else raise BCE`

存储（小于等于检查，双字）（Store Less Equal Doubleword） · 3R 型 · LA64

检查 `rj <= rk`，满足时存储双字，否则触发边界检查异常。

编码



编码值 `0x387f8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stle.d $a0, $a1, $a2      # if ($a1 <= $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    Memory[GR[rj], 64] = GR[rd][63:0]
else
    RaiseException(BCE)
```

---

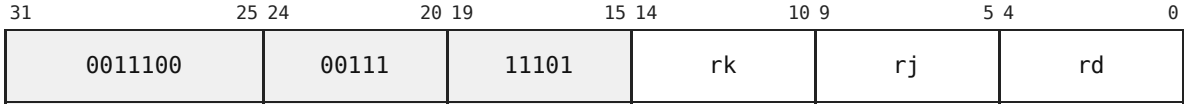
stle.h rd, rj, rk

`if (GR[rj] <= GR[rk]) Memory[GR[rj], 16] = GR[rd][15:0] else raise BCE`

存储（小于等于检查，半字）（Store Less Equal Halfword） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时存储半字，否则触发边界检查异常。

编码



编码值 `0x387e8000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stle.h $a0, $a1, $a2      # if ($a1 <= $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    Memory[GR[rj], 16] = GR[rd][15:0]
else
    RaiseException(BCE)
```

---

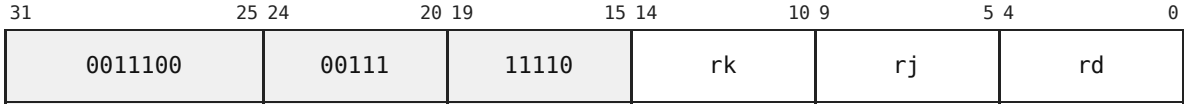
stle.w rd, rj, rk

`if (GR[rj] <= GR[rk]) Memory[GR[rj], 32] = GR[rd][31:0] else raise BCE`

存储（小于等于检查，字）（Store Less Equal Word） · 3R 型 · LA32 / LA64

检查 `rj <= rk`，满足时存储字，否则触发边界检查异常。

编码



编码值 `0x387f0000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
stle.w $a0, $a1, $a2      # if ($a1 <= $a2) [$a1] = $a0
```

语义伪代码

```
if (GR[rj] <= GR[rk])
    Memory[GR[rj], 32] = GR[rd][31:0]
else
    RaiseException(BCE)
```

---

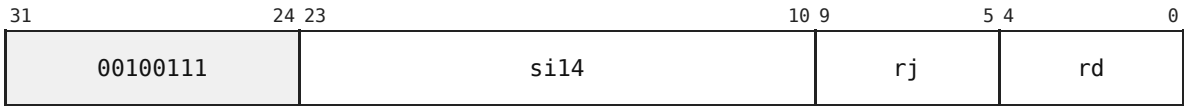
stptr.d rd, rj, si14

`Memory[GR[rj]+si14] = GR[rd][63:0]`

指针存储（双字）（Store Pointer Doubleword） · 2RI14 型 · LA64

将 `rd` 的低 64 位写入 `rj + si14`，常与 `addu16i.d` 配合。

编码



编码值 `0x27000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd`

汇编示例

```
stptr.d $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si14] = GR[rd][63:0]
```

---

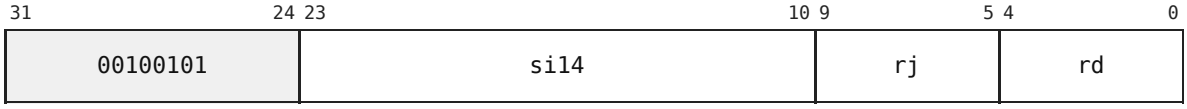
stptr.w `rd, rj, si14`

Memory[GR[rj]+si14] = GR[rd][31:0]

指针存储（字）（Store Pointer Word） · 2RI14 型 · LA32 / LA64

将 `rd` 的低 32 位写入 `rj + si14`，常与 `addu16i.d` 配合。

编码



编码值

```
0x25000000 | ((si14 & 0x3fff) << 10) | (rj << 5) | rd
```

汇编示例

```
stptr.w $a0, $a1, 0      # [$a1] = $a0
```

语义伪代码

```
Memory[GR[rj]+si14] = GR[rd][31:0]
```

---

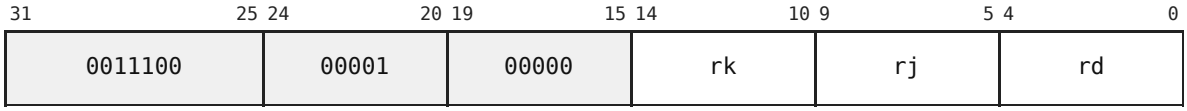
stx.b `rd, rj, rk`

Memory[GR[rj]+GR[rk], 8] = GR[rd][7:0]

索引存储（字节）（Indexed Store Byte） · 3R 型 · LA32 / LA64

将 `rd` 的低 8 位存储到 `rj + rk`。

编码



编码值

```
0x38100000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
stx.b $a0, $a1, $a2      # [$a1+$a2] = $a0
```

语义伪代码

```
Memory[GR[rj]+GR[rk], 8] = GR[rd][7:0]
```

---

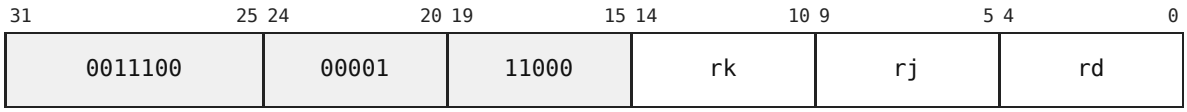
stx.d `rd, rj, rk`

Memory[GR[rj]+GR[rk], 64] = GR[rd][63:0]

索引存储（双字）（Indexed Store Doubleword） · 3R 型 · LA64

将 `rd` 的低 64 位存储到 `rj + rk`。

编码



编码值

```
0x381c0000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
stx.d $a0, $a1, $a2      # [$a1+$a2] = $a0
```

语义伪代码

```
Memory[GR[rj]+GR[rk], 64] = GR[rd][63:0]
```

---

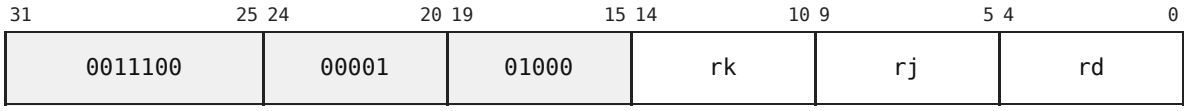
stx.h rd, rj, rk

Memory[GR[rj]+GR[rk], 16] = GR[rd][15:0]

索引存储（半字）（Indexed Store Halfword） · 3R 型 · LA32 / LA64

将 `rd` 的低 16 位存储到 `rj + rk`。

编码



编码值

```
0x38140000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
stx.h $a0, $a1, $a2      # [$a1+$a2] = $a0
```

语义伪代码

```
Memory[GR[rj]+GR[rk], 16] = GR[rd][15:0]
```

---

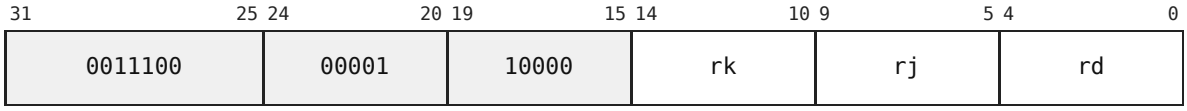
stx.w rd, rj, rk

Memory[GR[rj]+GR[rk], 32] = GR[rd][31:0]

索引存储（字）（Indexed Store Word） · 3R 型 · LA32 / LA64

将 `rd` 的低 32 位存储到 `rj + rk`。

编码



编码值

```
0x38180000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
stx.w $a0, $a1, $a2      # [$a1+$a2] = $a0
```

语义伪代码

```
Memory[GR[rj]+GR[rk], 32] = GR[rd][31:0]
```

---

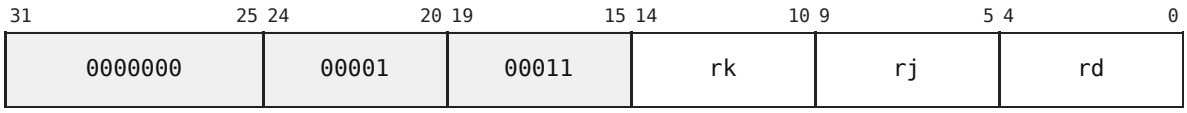
sub.d rd, rj, rk

GR[rd] = GR[rj] - GR[rk]

双字减法（Subtract Doubleword） · 3R 型 · LA64

将 `rj` 减去 `rk`，结果写入 `rd`。

编码



编码值

```
0x00118000 | (rk << 10) | (rj << 5) | rd
```

汇编示例

```
sub.d $a0, $a1, $a2      # $a0 = $a1 - $a2
```

语义伪代码

```
GR[rd] = GR[rj] - GR[rk]
```

---

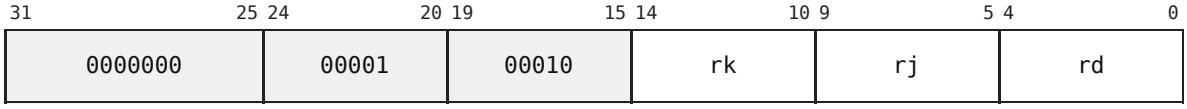
sub.w `rd, rj, rk`

`GR[rd] = SignExtend(GR[rj][31:0] - GR[rk][31:0], GRLEN)`

字减法 （Subtract Word） · 3R 型 · LA32 / LA64

将 `rj` 低 32 位减去 `rk` 低 32 位，结果符号扩展后写入 `rd`。

编码



编码值 `0x00110000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
sub.w $a0, $a1, $a2      # $a0 = $a1 - $a2
```

语义伪代码

```
GR[rd] = SignExtend(GR[rj][31:0] - GR[rk][31:0], GRLEN)
```

---

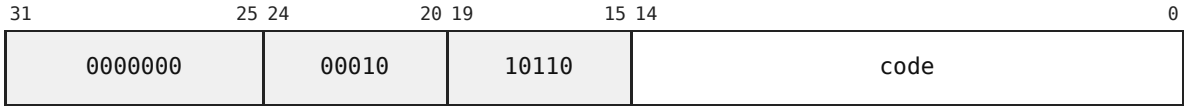
syscall `code`

`raise SYS(code)`

系统调用 （System Call） · I15 型 · LA32 / LA64

立即触发系统调用异常（SYS），`code` 作为异常处理参数。

编码



编码值 `0x002b0000 | ((code & 0x7fff) << 0)`

汇编示例

```
syscall 0      # 触发系统调用
```

语义伪代码

```
RaiseException(SYS, code)
```

T · TLB 维护指令

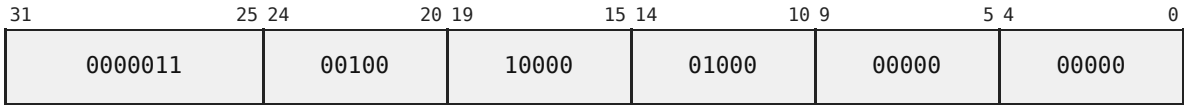
tlbclr

`TLBClear()`

TLB 清除 （TLB Clear） · 专用格式 · LA32 / LA64

清除 TLB 中与当前 ASID 相关的全局/非全局表项。

编码



编码值 `0x06482000`

汇编示例

```
tlbclr      # TLB 清除
```

语义伪代码

```
TLBClear()
```

---

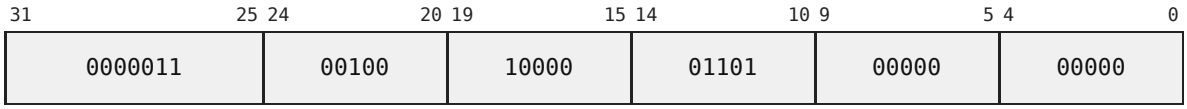
tlbfill

TLBFill()

TLB 填充 （TLB Fill） · 专用格式 · LA32 / LA64

按 CSR.TLBIDX 随机选择表项并写入 CSR.TLBEHI/TLBELO。

编码



编码值 0x06483400

汇编示例

```
tlbfill      # TLB 填充
```

语义伪代码

```
TLBFill()
```

---

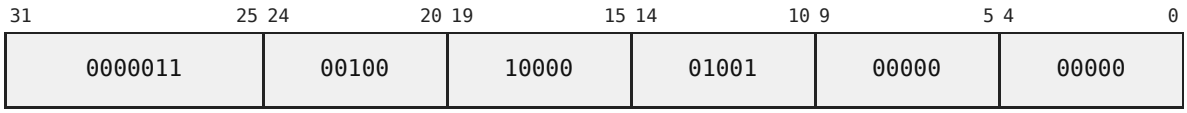
tlbflush

TLBFlush()

TLB 刷新 （TLB Flush） · 专用格式 · LA32 / LA64

刷新 TLB 中所有非全局表项。

编码



编码值 0x06482400

汇编示例

```
tlbflush      # TLB 刷新
```

语义伪代码

```
TLBFlush()
```

---

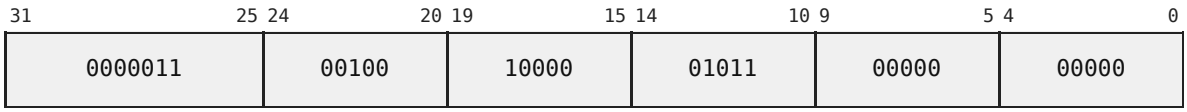
tlbrd

TLBRead()

TLB 读出 （TLB Read） · 专用格式 · LA32 / LA64

读取 CSR.TLBIDX 指向的 TLB 表项到 CSR.TLBEHI/TLBELO。

编码



编码值 0x06482c00

汇编示例

```
tlbrd      # TLB 读出
```

语义伪代码

```
TLBRead()
```

---

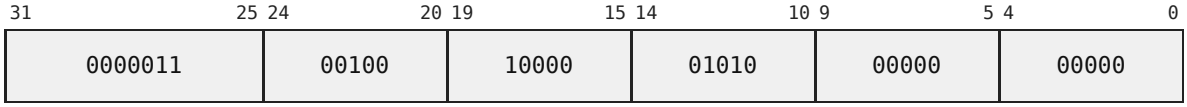
tlbsrch

`TLBSearch()`

TLB 查找 （TLB Search） · 专用格式 · LA32 / LA64

按 CSR.TLBIDX/ASID 等参数搜索 TLB，并更新索引。

编码



编码值 `0x06482800`

汇编示例

```
tlbsrch      # TLB 查找
```

语义伪代码

```
TLBSearch()
```

---

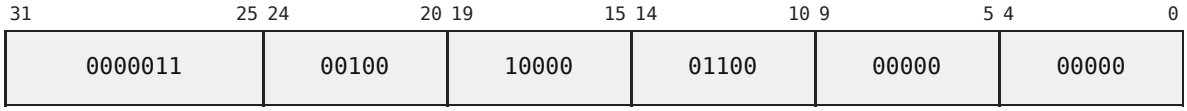
tlbwr

`TLBWrite()`

TLB 写入 （TLB Write） · 专用格式 · LA32 / LA64

将 CSR.TLBEHI/TLBELO 内容写入 CSR.TLBIDX 指向的 TLB 表项。

编码



编码值 `0x06483000`

汇编示例

```
tlbwr      # TLB 写入
```

语义伪代码

```
TLBWrite()
```

X · 异或指令

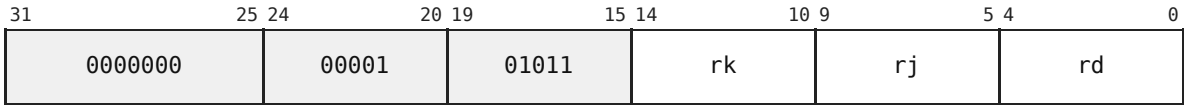
xor `rd, rj, rk`

`GR[rd] = GR[rj] ^ GR[rk]`

按位异或 （Bitwise XOR） · 3R 型 · LA32 / LA64

将 `rj`、`rk` 按位异或，结果写入 `rd`。

编码



编码值 `0x00158000 | (rk << 10) | (rj << 5) | rd`

汇编示例

```
xor $a0, $a1, $a2      # $a0 = $a1 ^ $a2
```

语义伪代码

```
GR[rd] = GR[rj] ^ GR[rk]
```

---

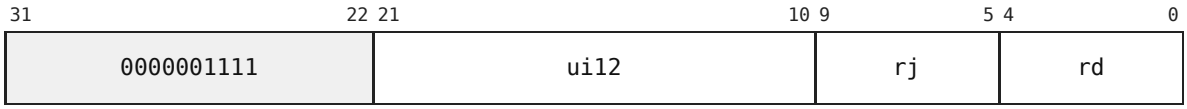
xori rd, rj, ui12

GR[rd] = GR[rj] ^ ZeroExtend(ui12, GRLEN)

立即数按位异或（Bitwise XOR Immediate） · 2RI12 型 · LA32 / LA64

将 `rj` 与零扩展的 12 位立即数按位异或，结果写入 `rd`。

编码



编码值

```
0x03c00000 | ((ui12 & 0xfff) << 10) | (rj << 5) | rd
```

汇编示例

```
xori $a0, $a1, 0xff      # $a0 = $a1 ^ 0xff
```

语义伪代码

```
GR[rd] = GR[rj] ^ ZeroExtend(ui12, GRLEN)
```